3D Spatial Construction

Contents

Abstract . . . 3
Introduction . . . 3
Three commonly misused notions of ”realism” . . . 4
Corpus Selection and Coding Method . . . 4
Background and the Input--Output Contract . . . 5
Data and Conditioning Layer . . . 5
Generation or Estimation Layer . . . 5
Scene Representation Layer . . . 5
Geometric Assetization Layer . . . 6
Physical Proxy Layer . . . 6
Engine Runtime Layer . . . 6
Validation and Security Layer . . . 7
Technical Lineage: New Representations Expand Capabilities Without Eliminating Older Layers 7
Geometry-First Stage . . . 7
Neural Implicit and Neural Rendering Stage . . . 7
Explicit Differentiable Primitive Stage . . . 7
Feed-Forward Geometry Foundation Model Stage . . . 7
Generative Video and World Model Stage . . . 8
The Parallel Backbone of Procedural Rules and Constraints . . . 8
Persistent 3D State and the Production Asset Stage . . . 8
The Unified Input--Internal State--Output Contract . . . 9
Classification Design and Coverage Audit . . . 9
Classification and Verification Figures . . . 10
L0--L5 Method Families . . . 10
L0: Video Generation and Action-Conditioned Frame Worlds . . . 10
L1: NeRF, Neural Rendering, and Queryable Appearance of 3DGS . . . 21
L2: Camera, Depth, Point-Map, and Point-Cloud Spatial Samples . . . 25
L3: Explicit Surfaces, Production Meshes, and Procedural DCC/CAD . . . 32
L4: Collision Surfaces, Distance Proxies, and Navigation Surfaces . . . 48
L5: Game Engines and Persistent Interactive Worlds . . . 55
Cross-Family Synthesis and Selection Guide . . . 61
Classification and Validation Figures . . . 62
First Contradiction: Observational Constraints and Generative Priors . . . 62
Second Contradiction: Image Loss and Surface Queries . . . 62
Third Contradiction: Per-Scene Optimization and Amortized Inference . . . 63
Fourth Contradiction: Frame-by-Frame Generation and Persistent Spatial State . . . 63
Fifth Contradiction: From Observed Unknowns to Explicit Design Intent . . . 64
Categorizing by core unknowns is more stable than categorizing by paper names . . . 64
Seven key transitions define complete spatial construction . . . 65
From design intent to parameterized geometry and scene graph . . . 65
From pixels to cameras and spatial samples . . . 65
From spatial samples to queryable appearance . . . 65

From spatial samples or radiance representations to surfaces . . . 65
From surfaces to production meshes . . . 65
From visual meshes to collision and navigation . . . 66
From assets to a persistent running world . . . 66
The correct order for method selection . . . 66
Datasets, Metrics, and Evaluation Evidence . . . 66
Categorization and Validation Figures . . . 66
Questions and Boundaries of the Analysis . . . 69
Selection of the Public Benchmark Layer and the Data Contract . . . 69
Main Analysis Layer . . . 69
Resource Numbers and Cross-Verification . . . 69
Descriptive Within-Benchmark Synthesis . . . 69
Direction-Unified Improvement Relative to a Common Baseline . . . 69
Paired Description of the Original Configurations and +LPM . . . 70
Leave-One-Dataset-Out Sensitivity . . . 70
Why the Strict Random-Effects Meta-Analysis Was Not Run . . . 70
Correlation Analysis . . . 71
Method . . . 71
Results . . . 71
Missingness, Dependency, and Bias Audit . . . 71
Local Lightweight Reproduction of the Surface/Collision Proxy . . . 72
Input, Processing, and Output Contract . . . 72
Three Constructions with Different Capabilities . . . 72
Mesh Quality and the Collision/Hit-Point Protocol . . . 72
Actual Run Status and Limits . . . 73
Deterministic Dual-Mesh Reproduction of Procedural Modeling . . . 73
Integrated Interpretation . . . 74
Auditable Entry Points . . . 74
Primary Sources for This Chapter . . . 74
Applications and Deployment Mapping . . . 74
Classification and Verification Figures . . . 75
High-Fidelity Digitization of Real Scenes . . . 75
Rapid World Generation from a Single Image or Text . . . 75
Interactive Video Worlds and Agent Training . . . 76
Games, XR, and Content Production . . . 76
Robot Simulation and Digital Twins . . . 76
Rule-Based Spaces, Modular Levels, and Batch Synthetic Data . . . 76
Choosing the Primary Representation by the Final Query . . . 77
Decision Rules . . . 77
Attacks, Defenses, Future Trends, and Limitations . . . 78
Classification and Verification Figures . . . 78
Attack Methods: From Observation Poisoning to Geometric Assets, Collision Navigation, and Engine Hosts . . . 78
Evolution of defense approaches and methods: from ”cleaning the input” to ”certifiable world contracts” . . . 84
Future trends: from ”can generate” to ”maintainable, verifiable, branchable” . . . 89
Evidence and reproduction limitations . . . 91
Conclusion . . . 91
Open Materials and Migration Audit . . . 92
Throughout the text, citations point preferentially to papers, official technical reports, project pages, standards, or official engine documentation. Product materials only attest to public interfaces and official statements. Where the model structure, training data, or independent benchmark is not public, those parts are not written up as verified performance. . . . 92

Appendix — Post-cutoff update (2026-08-07 □ 2026-09-26) 92

A.1 — 3D spatial construction: no substantive progress on the agenda 92
A.2 — Related movement 92
A.3 — Cross-repository note: institutional consequence of the OpenAI–Hugging Face incident . . 93
A.4 — How to use this appendix 93

Abstract

3D spatial construction is not the generation of 2D imagery that looks 3D. It converts observations or design intent, layer by layer, into queryable representations, production geometry, physical proxies and persistent runtime state. As of August 7, 2026, this survey compiles 173 retrieval records, 253 source entry points, 247 normalized titles, 40 attack-defense contracts and 108 benchmark long-table records. Its sole primary classification axis is the capability closed loop from L0 imagery to the L5 persistently running world. On that axis it uniformly analyzes video models, action-conditioned world models, World Labs Marble, SfM/MVS/TSDF, NeRF and neural SDF, 3DGS, point-cloud surface reconstruction, mesh assetization, procedural DCC/CAD, collision navigation and game engine construction. The synthesis shows that generation, reconstruction and rule-based modeling solve different unknowns. Visual usability cannot substitute for geometry, collision, navigation or state contracts. Attacks can propagate along coordinates, representations, assets and runtime. Defense therefore requires layered verification gates. No strict random-effects meta-analysis was run, because there is only one independent comparative study and variance is missing. Correlations serve only as exploratory description. The local Python geometry and dual-mesh mechanisms have been run. The Blender, Houdini, CAD and game engine hosts have still not been executed. A local full-text corpus, dual screening and a concrete submission template are lacking, so delivery status is limited to a compilable classification survey draft.

\newpage

Introduction

The core problem of spatial construction is not whether an image has a three-dimensional look. It is whether the system can connect provenance, coordinates, representation, geometry, physical proxies and runtime state into a verifiable transformation chain.

In this survey, 3D spatial construction means turning a set of observations or generation conditions into a spatial system that can be used sustainably. Such a system does not necessarily reach the same depth in every project. It must at least make its delivery contract explicit. Does it output only a video clip, or a scene that can be queried from any camera? Does it have measurable geometry? Can it become a topologically stable surface? Can it support collision, navigation and dynamics? Can a game engine, XR or a robot simulator load it reliably?

The definition deliberately keeps the following results apart.

A video model generates a pixel sequence with parallax and motion. A world model can continuously generate the next frame according to actions. NeRF or 3DGS can render high-quality images from new cameras. A point cloud provides a large number of surface samples in three-dimensional coordinates. A mesh provides triangular faces with connectivity relations. A collision proxy allows objects to maintain stable contact in a physics solver. A navigation mesh can describe the reachable region under a given agent size, slope and step constraint.

Everyday language can call all of these objects ”3D”, but their inputs, states and verifiable properties differ. Placing them side by side in a leaderboard produces erroneous conclusions. For example, the original NeRF paper optimizes a function from known camera images to continuous density and view-dependent color, and the paper's direct goal is novel view synthesis. It does not treat watertight meshes or collision bodies as native outputs. Original NeRF paper [@srcNERF-2020] 3D Gaussian Splatting (3DGS) represents a scene as a set of optimizable, rapidly projectable anisotropic Gaussians and achieves real-time novel view synthesis. Those Gaussians have no topological relations of a triangle mesh. Original 3DGS paper [@src3DGS-2023-N005] SuGaR requires an additional surface alignment regularizer and Poisson reconstruction. That requirement

shows precisely that going from high-quality Gaussian rendering to an editable mesh remains a separate line of work. SuGaR [@srcN006]

Three commonly misused notions of ”realism”

Visual realism answers whether a rendered image looks like a real photograph. PSNR, SSIM, LPIPS, FID/FVD and human preference usually measure different facets of this layer.

Geometric realism answers whether surface positions, depth, normals, camera and topology are close to the reference. The usual metrics are absolute/relative depth error, camera rotation/translation error, Chamfer distance, F-score, normal consistency, completeness and accuracy.

Physical realism answers whether objects can evolve according to auditable contact, friction, mass, constraints and dynamics. Stable frame rates and a lack of visual interpenetration are no substitute for contact error, penetration depth, tunneling rate, energy drift, reachability rate and task success rate.

A method may be strong at the first layer and average at the second, yet produce no output at the third. This survey will not normalize the metrics of the three layers and combine them into a ”composite realism” score. Such a total would hide genuine engineering gaps.

\newpage

Corpus Selection and Coding Method

This section rewrites the retrieval, screening, extraction, analysis and validation records that the earlier project actually executed into an evidence method. It does not back-infer a systematic review process that does not exist. The evidence cutoff date is August 7, 2026. The main retrieval window starts in 2016, and no start year is set for foundational methods such as SfM, volumetric fusion, surface reconstruction and collision detection.

The merged retrieval log contains 173 queries. The source master table contains 253 unique entry IDs and 247 normalized titles. It uses 6 groups of multiple-entry records sharing a title for version or entry reconciliation. The screening log contains 255 decisions, of which 253 were included and 2 same-name entities were excluded. Source entry points, unique works and independent studies are three different denominators.

The inclusion condition is direct participation in 3D generation, reconstruction, representation, assetization, procedural modeling, physical proxies, engine execution or security. At least one of the method, interface, public results, code or runtime artifacts must also be verifiable. Product pages only confirm public capabilities and export contracts. Abstracts, search snippets and secondhand paraphrase cannot support precise mechanism or performance conclusions.

Qualitative synthesis takes the method or system family as its unit. It codes the primary classification by L0--L5 output capability. The quantitative module takes paper--method configuration--dataset--metric as its unit. It explicitly preserves the dependencies of same-paper, shared-table and paired configurations. The attack-defense module additionally has 40 contract records. Their fields include threat model, attack capability, impact, observable signals, defense assumptions, adaptive evaluation and evidence boundary.

A strict random-effects meta-analysis was not run. The current same-protocol quantitative layer has only one independent comparative study, and the 108 long-table records lack per-scene variance, standard error or confidence interval. What was run is within-table descriptive synthesis, missingness and dependency auditing, and exploratory correlation analysis. These results describe the current corpus and do not constitute causal or cross-study population effects.

Reproduction status is recorded in four tiers. The local pure-Python geometry and deterministic dual-mesh mechanisms actually ran and passed. The Blender and Houdini examples passed only syntax or static checks. The CAD, engine, USD and synthetic-data hosts were not executed. For Marble, only a spot check of the public file contract was completed, with no reproduction of weights, the paid API or the internal algorithm.

This project has no local full-text PDF corpus, no two-person independent screening agreement and no formal bias assessment tool. This survey is therefore an auditable classification survey draft, and it does not use the strict systematic review label. The complete retrieval strategy is stored in normalized/search_strategy.md. The source, migration and gap ledgers are stored in the source, validation and evidence directories.

\newpage

Background and the Input--Output Contract

This section unifies the seven-layer pipeline and the input--internal state--output contract. It provides a common interface for the subsequent taxonomy.

Data and Conditioning Layer

This layer specifies what the system can see and what counts as a known quantity.

Observations consist of single image, multi-image, video, panorama, RGB-D and LiDAR/point cloud. Known quantities include intrinsics, extrinsics, timestamps, IMU, and scale or GPS. Conditioning consists of text, sketch, coarse 3D layout, object list and style. It also consists of user actions, agent pose, historical frames, memory and control targets.

Input conditions directly determine identifiability. A single image does not provide enough observations to uniquely determine occluded regions and absolute scale, and any full back side or unseen space involves prior-driven inference. If multi-image input lacks overlap, texture or stable exposure, matching and pose also degrade. If a dynamic video treats object motion as camera motion, it contaminates the entire geometry chain.

Generation or Estimation Layer

Generative models and reconstruction algorithms diverge at this layer.

The generation route learns a conditional distribution and provides plausible but not guaranteed-real content in unobserved regions. The estimation route uses observation constraints to recover pose, depth, correspondences and surfaces. The hybrid route uses generative priors to fill invisible regions and then makes the result queryable through multi-view or explicit geometric constraints.

Classical incremental SfM recovers sparse geometry and cameras through feature matching, geometric verification, camera registration, triangulation and bundle adjustment. The corresponding COLMAP paper [@srcG003] DUSt3R instead directly regresses uncalibrated image pairs into point maps and then performs global alignment. It lowers the barrier of camera priors, but going from point maps to production meshes still requires post-processing. DUSt3R [@srcG011] VGGT jointly outputs cameras, depth, point maps and point trajectories in a single forward computation. That joint output represents the development direction of 3D geometry foundation models. Its output is still geometric attributes and point-based representations, not engine assets with UV, collision and navigation completed automatically. VGGT [@srcVGGT-2025]

Scene Representation Layer

The representation determines which queries are cheap and which edits are difficult.

Voxel/TSDF/SDF holds occupancy or signed distance to the surface, which suits fusion, isosurface extraction and distance queries, but high-resolution dense grids consume memory. Point cloud/point map/surfel preserves sampling positions, colors, normals or confidence directly, and incremental fusion is convenient, but there is no closed surface or topology. NeRF/neural implicit field is continuous and differentiable and good at appearance fitting, while per-scene optimization and surface extraction are usually expensive. 3DGS uses explicit Gaussians, trains and renders fast, and offers strong transparency and view-dependent appearance, but geometry, topology and collision still require constraints or conversion. A triangle mesh has a mature

ecosystem for engines, DCC, rasterization and physics, while topology, UV, materials and LOD need maintenance. A hybrid representation lets meshes handle topology, animation and collision while Gaussians/neural textures handle high-fidelity appearance, a compromise of considerable practical value today. A latent world state is convenient for autoregressive prediction and action-conditioned generation, but if geometry cannot be exported or queried, it cannot replace an asset representation.

Geometric Assetization Layer

This layer turns ”data that can be displayed” into ”assets that can be maintained”. The typical steps are as follows.

Unify coordinate systems, units, orientation, cameras and scene origin. Remove outliers and low-confidence floating geometry. Estimate normals and keep their orientation consistent. Obtain a triangle mesh via alpha shape, Ball Pivoting, Poisson, TSDF or neural surfaces. Delete small connected components, fill holes, handle self-intersections, run manifold checks and correct normals. Simplify, retopologize, build LODs, unwrap UVs, and bake textures and materials. Export and validate according to target formats such as glTF/GLB, USD, FBX, OBJ, PLY and SPZ.

Open3D's official surface reconstruction documentation clearly reflects the algorithmic assumptions. Ball Pivoting depends on oriented normals and a suitable ball radius. Poisson solves a regularization problem to obtain a smooth surface, and the octree depth determines detail and resource consumption. Open3D surface reconstruction [@srcOPEN3D-SURFACE-P006] So ”point cloud to mesh” is not a parameter-free format conversion. It is a reconstruction step, and it carries assumptions about scale, density, noise and topology.

Physical Proxy Layer

Visual meshes aim at the image. Physical proxies aim to answer contact and reachability queries in a stable, cheap and conservative way. The two should usually be separated.

Static buildings can use simplified triangle meshes or height fields. Dynamic rigid bodies preferably use boxes, spheres, capsules, convex hulls or multiple convex pieces. Complex concave objects can undergo approximate convex decomposition or adopt SDF/voxel distance queries. Characters usually use stable proxies such as capsules rather than skintight high-poly meshes. The walkable ground surface is determined by proxy radius, height, slope, steps and clearance. It cannot be directly equated with visible ground triangles.

Unity's official documentation explicitly states that Mesh Collider can fit complex shapes more closely than primitives but is usually more expensive. Convex Mesh Colliders have a triangle count limit, and mesh cooking also cleans up degenerate triangles and preprocesses runtime queries. Unity Mesh Collider [@weba9de639ddac5] Unreal draws the same line: simple collision (primitives/convex hulls) and complex collision (triangle meshes) are two independent query representations. Unreal simple/complex collision [@srcUE-COLLISION-C006] That split is not an incidental engine limitation. It is a fundamental trade-off of real-time physics among stability, memory, build time and precision.

Engine Runtime Layer

Once a scene enters the engine, several further jobs still need to be completed.

Convert coordinates, scale, handedness and axes. Set PBR materials, color space, transparency, two-sided and shadow rules. Implement LOD, meshlet/Nanite-style geometry streaming, occlusion culling and texture streaming. Configure collision layers, triggers, continuous collision detection and physics materials. Build the navigation mesh, dynamic obstacles, off-mesh links and agent configuration. Manage scene partitioning, world origin rebasing, network replication and save/load. Isolate scripts, shaders, plugins and the asset supply chain.

Recast's public implementation voxelizes the input triangle mesh, filters out non-walkable voxels, partitions regions and regenerates a polygon navigation mesh. Tiled navmesh supports re-baking and streaming for

large scenes. Recast Navigation [@srcRECAST-NAV-V001] Godot's documentation further emphasizes that navigation and rendering/physics are independent systems. The navmesh is an approximate walkable area for a specific agent, and it cannot be expected to fit the ground precisely. If a character needs to stick to the ground precisely, it must also rely on physics or other ground queries. Godot NavigationMesh [@webf-bcb09318894]

Validation and Security Layer

Each layer needs its own gate.

Input gates check file type, size, provenance, license, hash, camera and timestamp consistency. Reconstruction gates check pose loop closure, reprojection, depth/point cloud error, confidence and anomalous frames. Representation gates check novel view, geometry, temporal and unobserved-region uncertainty. Mesh gates check finite numbers, triangle/vertex caps, degenerate faces, self-intersection, manifold, watertightness, connected components, normals, UV. Collision gates check proxy error, penetration, continuous collision, stacking, resting jitter and query latency. Navigation gates check connectivity rate, unreachable points, narrow passages, slope/steps and dynamic replanning. Engine gates check import warnings, frame budget, peak memory, streaming, script/shader allowlists and sandbox.

Only when these gates are explicitly recorded can a project know where a failure occurred. That failure may be ”the model did not generate”, ”the geometry is incorrect”, ”the asset is unusable” or ”the runtime configuration is wrong”.

Technical Lineage: New Representations Expand Capabilities Without Eliminating Older Layers

Geometry-First Stage

Photogrammetry, SfM/MVS, SLAM, and RGB-D fusion built the backbone that turns correspondences, pose, and depth into point clouds and meshes. Geometric quantities mean something definite, errors can be measured, and the results land close to CAD or engine assets. That is the advantage. The disadvantage is dependence on texture, lighting, calibration, view coverage, and dynamic scene handling. Fusion methods such as TSDF average multi-frame depth into voxels as truncated distances, so stable surfaces are easy to extract. Memory, though, constrains resolution and spatial extent.

Neural Implicit and Neural Rendering Stage

NeRF treats a scene as a continuous radiance field. That significantly improved novel-view quality and opened branches for acceleration, dynamics, reflection, semantics, editing, and surface reconstruction. Its core value is that rendering error optimizes the scene function directly. Its core limitations are per-scene optimization, dense ray queries, and the ambiguity that runs from density to surface. Neural SDF/occupancy fields write geometry into implicit functions and improve mesh extraction, but they still struggle with materials, lighting, scale, and thin structures.

Explicit Differentiable Primitive Stage

3DGS marks a leap in training and rendering efficiency, built on explicit anisotropic Gaussians and differentiable splatting. Explicit centers, scales, rotations, opacity, and spherical harmonic appearance make culling, compression, and streaming easier. Yet ”explicit” does not equal ”surfaced”. Occlusion relationships and color optimization are enough to produce beautiful images, but they do not guarantee that every Gaussian lies on the true surface. Follow-up work such as SuGaR, 2DGS, depth-supervised GS, and mesh-aligned GS is filling precisely this gap.

Feed-Forward Geometry Foundation Model Stage

DUSt3R, MASt3R, VGGT, and their successors partially compress camera, matching, depth, and point map estimation into the forward inference of a shared Transformer. Those steps previously required several mod-

ules and iterative optimization. These models significantly lower the barrier for casually captured input. They also bridge rapid 3D extraction from video generation results. Geometry foundation model outputs, however, still require validation on confidence, scale, drift across long sequences, dynamic objects, and extrapolated regions. A fast point cloud is still not a final game asset.

Generative Video and World Model Stage

Video diffusion has improved content diversity and camera control. Autoregressive world models add user actions to state transitions, so frames can respond in real time. The research version of Genie learns a controllable environment from videos that carry no action labels. It relies on spatiotemporal tokenization, autoregressive dynamics, and a latent action model [@srcGDM-GENIE]. Genie 3's official public page reports 720p, 24 fps, and a duration of several minutes. It explicitly lists limitations. These include a limited action space, limited interaction duration, multi-agent interaction, and accuracy of real locations. This round found no public technical report or weights for Genie 3 as of the survey's cutoff date [@srcGDM-GENIE3-W09]. The native output of such models is still an action-conditioned pixel sequence. The user-experienced ”walking around in a 3D world” therefore does not automatically imply a downloadable static mesh, a collision body, or a deterministic physics state.

The Parallel Backbone of Procedural Rules and Constraints

3D spatial construction does not begin with photogrammetry or generative models alone. OpenSCAD, parametric CAD, Blender/Houdini scripting, Geometry Nodes, procedural level generation, and engine editor automation take dimensions, sketch constraints, CSG trees, node graphs, object rules, and random seeds straight into geometry and scene graphs. This route is older than neural representations, yet it still matters for batch content, digital twins, synthetic data, and agent-driven modeling. Known structure does not have to be inferred back from pixels. Wall widths, door openings, and part relations can therefore be preserved explicitly. Stable object IDs are a further step, and they become constructible only when the schema derives IDs from semantic paths and stable parameters. Tool default numbering, exported face indices, or ”the Nth face” provide no such guarantee. The route describes design intent, and it does not automatically prove on-site reality.

The procedural route also reveals two different meanings of the word ”generation”. Probabilistic generative models sample content from a learned distribution. Even with conditioning held fixed, reproducing the result may require the sampling strategy and the model version. A procedural generator instead evaluates from explicit rules. Once its parameters, seed, and toolchain are fixed, it should return the same normalized asset. The two can be combined. Language or video models propose layout and style. DCC/CAD code then fixes measurable dimensions and topological constraints. Geometry libraries perform differential validation. Engine scripts finally build collision, navigation, and runtime state. Code can produce a mesh directly. That does not mean the mesh has correct units, manifold topology, collision semantics, or trustworthy script provenance. Later layers still accept these contracts independently.

Persistent 3D State and the Production Asset Stage

Representative work from 2025–2026 shows that explicit or latent 3D state beyond pixel history is re-entering world models. PERSIST writes the environment, camera, and renderer into a latent 3D scene, and it directly targets the spatial forgetting and long-horizon inconsistency caused by limited context [@srcPERSIST-2026]. WorldGen starts from text and combines layout reasoning, procedural generation, 3D generation, and object-level decomposition. It moves the goal from ”viewable” to ”traversable and interactive” [@srcWORLDGEN-2026].

Marble takes another path, a product-type one. Its official documentation states that it accepts text, a single image, multiple images, panoramas, video, and coarse 3D structures. It can export SPZ/PLY at about 2 million or 500,000 Gaussians, high-quality GLB meshes, and coarse collision GLB (Marble overview [@srcWL-MARBLE-OVERVIEW]; Marble export specification [@srcWL-MARBLE-EXPORT-M04]). More critically, the documentation explicitly distinguishes the highest-fidelity splats, visual meshes of about 600,000/1,000,000 triangles, and collision meshes of about 100,000–200,000 triangles. It acknowledges that visual meshes may

show holes, clumps, and floating faces on thin structures, transparent/reflective surfaces, sky, and uncovered regions (Marble mesh export [@srcWL-MARBLE-MESH]). This is a very concrete piece of product evidence for this survey's central argument. Visual, surface, and physical proxies remain three different assets, even if one system integrates generation, reconstruction, and export in a single interface.

The Unified Input--Internal State--Output Contract

The unified contract does not compress methods into a single total score. Instead it keeps asking five questions. What observables does the input provide? What does the internal state store? What queries can the native output answer? What assumptions does training or solving depend on? Which asset layers are still missing in the end? Video generation starts from text, image, or shot conditions and predicts frames in a pixel or latent video state. It usually still lacks pose, scale, surfaces, and physical assets. Action-conditioned world models add actions and memory. If that state cannot be queried and exported, deterministic geometry still cannot be obtained on that basis. SfM/MVS and RGB-D/TSDF begin with overlapping observations, cameras, or depth. What they recover natively is cameras, point clouds, depth, or isosurfaces. They still have to deal with uncovered regions, dynamics, texture, and assetization.

Feed-forward geometry models move the estimation of cameras, depth, point maps, or point trajectories into the network forward pass and amortize it there. That reduces the per-scene solving cost, but global scale, long-sequence loop closure, and the final surface still require validation. NeRF natively answers color and density queries along rays, and 3DGS natively answers efficient projection and synthesis queries. Neither obtains stable topology automatically just because rendering quality is high. Procedural DCC/CAD, by contrast, constructs B-rep, meshes, and scene hierarchies directly from parameters, constraints, CSG trees, or node graphs. It natively answers ”what asset should be generated given the rules”. It does not automatically answer whether the asset matches real observations, whether the host is safe, or whether collision is correct. Point clouds store spatial samples but not face connectivity. Meshes store vertex--edge--face adjacency relations, but manifoldness, UV, materials, and LOD need maintenance. The game engine then organizes these assets, physical proxies, and logic into a runnable scene.

The color coding in the figure indicates only ”native output”, ”obtainable through explicit conversion”, or ”not natively provided”. It does not indicate high or low performance, and it cannot be summed along a row. The subsequent analysis always uses this input--state--output--gap contract. It asks which solving step each work actually replaces, what kind of output is measured, and which responsibilities are left to the next layer.

\newpage

Classification Design and Coverage Audit

The survey classifies along one primary axis only, defined by the verifiable spatial capability that a method natively closes. L0 is pixels and frame sequences, delivering images, video, or action-conditioned observations. L1 is queryable appearance, and it requires that the same scene can still be rendered given a new camera. L2 is measurable spatial samples, delivering cameras, depth, point maps, point clouds, or surfels. L3 is explicitly editable surfaces and production assets, delivering meshes, SDF/TSDF isosurfaces, or versioned visual assets. L4 is collidable and navigable agents. L5 is persistent interactive worlds with object identity, state, scripting, physics, streaming, saving, and rollback.

Judgment looks first at whether the output can be independently queried and verified. Video with consistent camera motion does not automatically reach L1. A radiance representation that can render novel views does not automatically reach L2 or L3. 3DGS carrying explicit three-dimensional centers is not equivalent to an explicit surface. A visual mesh with triangles does not automatically reach L4. A world model that can predict the next frame under action conditioning does not automatically reach L5. Every promotion must add a new artifact contract or state contract.

Multi-labeling records only secondary mechanisms, such as explicit or implicit representation, per-scene optimization or feed-forward inference, observation-driven or generation-driven, static or dynamic, closed-source product or open implementation. A hybrid system may span multiple layers. The main text, though,

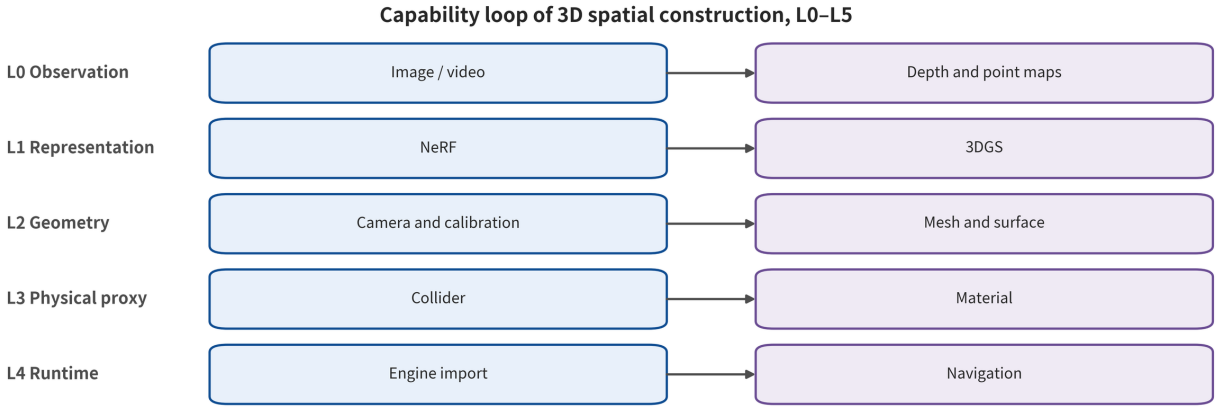
assigns it to the key interface of first closure, and then states for the later layers what conversions are still required.

Boundary cases of the coverage audit include Marble, Video2Game, feed-forward geometry foundation models, and procedural DCC/CAD. Public exports show that Marble can cover visual representation, mesh, and collision proxy, but the algorithm behind its generation is unknown. The value of Video2Game lies in the asset conversion chain, not merely in generating more frames. Procedural modeling moves straight from design intent to L3 or L4, so it should not be forced into the pixel reconstruction route.

This main axis does not promise that categories are mutually exclusive. It promises that the decision rules are stable. Categories are defined by verifiable outputs, and method names are only examples. A system that does not expose export, query, or state interfaces remains at the lowest layer that has been demonstrated. It cannot be inferred upward on the basis of demo footage.

Classification and Verification Figures

The figures below come from structured data or from generation scripts of prior projects. The captions also give the readable scope. The main text does not use comparison tables to replace argumentation.



The 3D spatial construction capability ladder and method family classification. The figure encodes categories by verifiable outputs and query capabilities, not by model names. It is a graphical expression of this survey's primary classification axis.

\newpage

L0--L5 Method Families

This section regroups the method families along the same input--internal state--output contract. For every family the section states the unknown it resolves, the verifiable artifact it produces, the supervision and optimization it relies on, and how failure propagates to the next layer.

L0: Video Generation and Action-Conditioned Frame Worlds

L0 natively delivers images, videos, or action-conditioned observation sequences. Camera conditioning, 3D caches, and latent states can improve consistency. Yet if the output is only frames, queryable appearance, spatial samples, and persistent assets still cannot be inferred.

The conclusion of this chapter comes first. Researchers did not bring video models into 3D spatial construction because ”continuous imagery is inherently three-dimensional”. They brought them in because cameras,

rays, depth, point clouds, and external caches were progressively connected into the generation process. The order of evolution can be summarized as follows. Numerical poses control the overall motion. Ray conditioning expands poses down to pixels. Depth back-projection and point cloud rendering then provide pixel-level geometric constraints. Finally, the newly generated observations go to 3DGS, mesh, and collision construction modules for consolidation. Every step adds a query capability, and every step also writes the previous step's error into the next layer.

The paper versions and public pages described in this chapter were verified as of August 6, 2026. The formulas explicitly distinguish generic expressions from paper-specific implementations. In particular, one must not generalize every camera-controllable video model into ”using Plücker rays”. MotionCtrl uses pose matrices directly. AC3D adopts the Plücker camera representation. ViewCrafter and GEN3C go further and use explicit point cloud rendering as a condition.

What separates a world model from ordinary video generation is the closed loop, not the name. An action has to change the internal state, and that state has to determine what is observed next. The RSSM used for control shares neither an optimization objective nor an output contract with the autoregressive token model used for playable video, the diffusion model that directly predicts the next observation, the JEPA that plans in an embedding space, or the persistent state model that explicitly maintains 3D world frames. This chapter organizes them by ”what the state is” rather than listing papers one by one.

What must be solved is not ”generating a video” but three kinds of non-identifiability The video generator receives text or a single starting image, a target camera trajectory, and optional depth information. It must output a set of frames

$$\hat{X}_{1:T} \sim p_\theta \left(X_{1:T} \mid X_{\text{ref}}, C_{1:T}, c_{\text{text}}, c_{\text{geo}} \right).$$

This is the generic probabilistic form of camera-conditioned video generation. It matches no single paper's original formulation. Here $C_t = (K_t, R_t, t_t)$ denotes intrinsics and extrinsics, and c_{geo} may be empty, or may be depth, point cloud rendering, optical flow, or multi-view conditions. This problem has at least three layers of non-identifiability.

First, a single RGB image cannot determine depth and absolute scale uniquely. Objects of different sizes, placed at different distances, can produce the same perspective projection. Second, occluded regions yield no observation at all, so the generator can only complete them from training priors. The completed back side may be perceptually plausible, yet it need not be identical to the real scene. Third, pixel motion can come from camera motion, from object motion, or from the two superimposed. If the ”pose-annotated videos” in the training data are almost all static real-estate videos, the model can easily mislearn camera conditioning as a signal to ”suppress scene dynamics.”

Therefore, the video route's native output contract is usually only a time-indexed sequence of RGB frames or latents. It can answer ”what does it look like along a given path.” It cannot directly answer the depth at a point, wall thickness, closed topology, collision normals, or whether a character can pass through a doorway. The capability level truly moves upward only when the system additionally outputs queryable depth, point clouds, 3DGS or meshes, and passes independent geometric checks.

The common foundation of video diffusion: denoising completes, conditioning constrains Most related work builds on latent video diffusion. What follows is a generic diffusion expression, not an implementation unique to any one method. An encoder first compresses a real video X into a latent variable $z_0 = E(X)$. Gaussian noise is then added at a random time τ:

$$z_\tau = \alpha_\tau z_0 + \sigma_\tau \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

If the network predicts noise, its common training objective is

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{z_0, \epsilon, \tau} \left[\left\| \epsilon - \epsilon_\theta(z_\tau, \tau, c) \right\|_2^2 \right],$$

Here the condition c may include text, the first frame, camera parameters, and geometric rendering. Some papers also adopt x_0 prediction, velocity prediction, or rectified flow. Those choices change the prediction target and the sampling trajectory, but the core is the same: "applying a condition to a noisy state and progressively restoring the video." If the condition is only a soft embedding during training, the denoiser can compromise between the visual prior and the condition. So "a pose was input" does not equal "projection geometry is satisfied pixel by pixel."

Camera conditions can be divided into three levels of granularity. The first is a whole-frame pose matrix. A typical form flattens each frame's 3×4 rotation--translation matrix and feeds it into a temporal module. The second is per-pixel rays. Adopt the world-to-camera convention $x_c = R_t X_w + t_t$ and let the pixel homogeneous coordinate be $\tilde{u}$. The camera center and the world-frame ray direction can then be written as

$$o_t = -R_t^\top t_t, \qquad d_t(u) = \frac{R_t^\top K_t^{-1} \tilde{u}}{\|R_t^\top K_t^{-1} \tilde{u}\|_2}.$$

Plücker rays are often written as $r_t(u) = (d_t(u), o_t \times d_t(u))$. This is a generic ray definition. Methods such as CameraCtrl, VD3D, and AC3D adopt this family of representations, but MotionCtrl does not construct Plücker rays per pixel. A ray indicates where a pixel looks from and where it looks toward. It does not indicate at which depth the ray intersects a surface.

The third is to back-project depth and color into explicit 3D points and then render from the target camera. This generic back-projection of a source pixel u, and the warp to the target view, still use the above camera convention:

$$X_w(u) = R_t^\top \left(D_t(u) K_t^{-1} \tilde{u} - t_t \right), \qquad \tilde{u}' \sim K_{t'} \left(R_{t'} X_w(u) + t_{t'} \right).$$

A practical implementation must also perform z-buffering, frustum culling, and occlusion masking. Otherwise points behind the source view will incorrectly cover the foreground of the target view. Depth warp provides one more assumption than a pure ray embedding, namely "where along the ray," and can therefore lock camera motion more strongly. It also turns depth errors directly into 3D position errors.

A generic geometry-conditioned video pipeline can be written as the algorithm below. It describes only the structure this method family shares. It does not represent the complete code of any single paper.

The text pipeline or pseudocode in the original manuscript

```
Input: reference images or sparse views, camera parameters, target camera trajectory
1. Encode the reference images; if explicit geometry is needed, estimate depth and confidence.
2. Back-project valid RGB-D pixels to world coordinates, keeping the source view and confidence.
3. Render color, depth, and a visibility mask from each target camera.
4. Encode the renderings, the first frame, and the text as video denoising conditions.
5. Sample or distill to obtain video frames along the target path.
6. Filter inconsistent frames using reprojection, cyclic trajectories, and multi-view matching.
7. Feed only the frames that pass screening into point cloud fusion, 3DGS, or mesh construction.
Output: generated frames; optional depth, point cloud cache, and a downstream reconstruction representat
```

Failure propagation is also determined by this chain. Pose errors make ray directions wrong. Ray or depth errors misalign the warp. The denoiser will then "fix" the misalignment to look real, and the reconstructor may in turn solidify such visual patching into fake surfaces. Therefore, video metrics, pose re-estimation metrics, and 3D geometry metrics must be reported separately.

MotionCtrl: first separate camera motion and object motion from video motion The primary problem that MotionCtrl [@srcV01] solves is not 3D reconstruction but control-signal entanglement. Conventional video conditioning often mixes camera movement and object movement into a single optical flow. Inside a latent video diffusion model, MotionCtrl adds a Camera Motion Control Module (CMCM) and an Object

Motion Control Module (OMCM) as separate modules. Each type of motion can then be controlled on its own or in combination.

Its paper-specific implementation works in the following steps. First, CMCM receives a per-frame camera pose sequence of length L, $RT_t = [R_t \mid t_t] \in \mathbb{R}^{3\times4}$. It first replicates the original $L\times12$ pose values along the spatial dimension into a camera condition of $H \times W \times L \times 12$. Second, this condition is concatenated along the channel dimension with the output of the first self-attention module in the temporal Transformer. A fully connected layer then projects it back to the original number of channels as the input to the second self-attention part. In other words, the projection in the paper happens after broadcasting and concatenation, rather than first encoding the 12-dimensional pose and then broadcasting. Third, OMCM rewrites the object trajectory given by the user into a sparse displacement field. If the positions of a trajectory point in adjacent frames are (x_{i-1}, y_{i-1}) and (x_i, y_i), the paper defines

$$u_i = x_i - x_{i-1}, \qquad v_i = y_i - y_{i-1},$$

and fills zeros at pixels that are not traversed, forming a trajectory tensor of $L \times H \times W \times 2$. Fourth, a convolution and downsampling network extracts multi-scale features from this tensor. It injects them only into the corresponding spatial layers of the denoising U-Net encoder. Fifth, CMCM is trained first. The base model and CMCM are then frozen to train OMCM, which avoids the later-trained camera module destroying the already-learned object trajectory control.

In this evolution, the step matters because it separates ”global camera motion” and ”local object motion” at the network interface. The control is still soft, however. The pose matrix is broadcast for the whole frame. It does not tell each pixel which ray it corresponds to, and it does not give the surface intersection. Therefore, when a scene contains large parallax, when occlusion appears, or when the camera combination is out of the training distribution, the model can produce roughly correct background movement. It cannot guarantee that every structure projects according to the same 3D scene.

Its output contract is still pixel video. The camera condition of CMCM cannot directly derive depth, point clouds, or collision bodies. The evolution after MotionCtrl naturally turns to two questions. How can pose conditioning be expanded into per-pixel rays? And at what time, and in which layers of the denoising process, should the condition be injected?

AC3D: camera motion is low-frequency structure, and the condition should not disturb all denoising layers AC3D [@web00a2d5e10b6d] conducts a finer-grained mechanistic analysis of video DiT. It does not simply ”switch to Plücker rays.” It first answers when camera information appears in denoising time and network depth, and then narrows the scope of condition injection accordingly.

The paper-specific pipeline is as follows. First, the method builds a per-pixel Plücker camera representation from each frame's extrinsics and intrinsics. A fully convolutional encoder turns it into camera tokens at the same resolution as the video tokens. Second, a lightweight DiT branch processes the camera tokens and adds them before the main video DiT blocks, while the main branch is allowed to give cross-attention feedback to the camera tokens. Third, the authors analyze the motion spectrum of the generated video. They find that camera movement mainly forms low-frequency motion and is determined at a very early stage of the rectified-flow reverse trajectory. Based on this observation, training noise is concentrated in the early interval $[0.6, 1]$, and inference applies camera conditioning only in the first 40% of the reverse denoising interval. This is the paper-specific schedule of AC3D; it should not be generalized into a fixed ratio for all diffusion models. Fourth, the authors perform linear pose probing on 32 video DiT blocks and find that camera information is most obvious in the middle layers. They therefore inject the condition only in the first 8 blocks, which avoids mutual interference between the already-formed camera representation and subsequent appearance and dynamic features. Fifth, in addition to RealEstate10K, about twenty thousand clips of ”fixed camera but dynamic scene” video are added. The camera branch thereby stays active on dynamic content. This also reduces the data bias that ”having camera annotation means a static scene”.

AC3D explains the key change from MotionCtrl to the next generation of methods. Controllability no longer depends only on what the condition is. It also depends on where the condition is injected along the denoising

time axis and the network-layer axis. It improves the conflict between camera following and video quality. It still does not form an explicit surface. Even if the generated video allows ParticleSfM to recover a relatively accurate trajectory, that only shows that the frames carry estimable camera motion. Scale, post-occlusion geometry, topology, and collision remain unknown.

Therefore, the failure of AC3D propagates downstream as follows. If the model ignores high-frequency camera changes, the parallax in the generated frames is smaller than that of the given trajectory. Subsequent SfM then estimates a compressed baseline, the triangulated depth becomes correspondingly too large, and the scale of the final point cloud and mesh is distorted. To block this propagation, the next step cannot be to keep adjusting the embedding alone. It must turn the coarse 3D projection directly into a pixel condition.

ViewCrafter: letting the coarse point cloud handle the camera and video diffusion handle hole filling ViewCrafter [@srcV07] advances the task from "camera-controllable video" to "novel view generation conditioned on coarse 3D". Its basic division of labor is clear. Point cloud rendering provides structure that can be aligned under the target camera. Video diffusion repairs the holes, stretching, and unobserved regions of the point cloud rendering.

The paper-specific algorithm consists of six steps. First, a dense stereo model such as DUSt3R estimates a point map, cameras, and an initial point cloud P^{ref}. Its input is a single image or sparse images. Second, under the target camera sequence $C_{1:L}$, the point cloud is rendered into a coarse video $P_{1:L}$. Here P denotes the point cloud rendering condition in the paper, not a probability. Third, a point-conditioned video diffusion model is trained to learn

$$X_{1:L} \sim p_\theta \left(X_{1:L} \mid I^{\text{ref}}, P_{1:L} \right),$$

This is the conditional distribution form given by the ViewCrafter paper. The reference image provides a reliable appearance anchor. The coarse rendering provides a per-pixel camera constraint, and the denoising model completes the missing regions. Fourth, to expand the browsing range, the system samples multiple candidate poses near the current camera and renders the visibility mask of each candidate. A utility function then selects the next best view that can reveal more uncovered regions. Fifth, it generates a new video along the interpolated trajectory from the current pose to the next best pose. The generated new views then go back to the dense stereo model to update the point cloud. Sixth, it repeats "plan camera--render coarse point cloud--diffusion completion--update point cloud". Finally, it uses the extended point cloud to initialize 3DGS and supervises 3DGS optimization with synthesized views.

This loop adds the idea of "actively supplementing observations", but it also introduces an important bootstrapping risk. The utility of a candidate viewpoint comes from the visibility of the current point cloud. If the initial depth is wrong, the planner may treat erroneous regions as uncovered regions. The diffuser then generates a visually plausible patch, and the stereo model turns it into new points. The error therefore escalates from appearance hallucination to three-dimensional fact. A higher point cloud cleanup threshold can reduce flying points but increases holes. A lower threshold retains more structure and also retains more erroneous points.

ViewCrafter's native generation output is still novel view videos and an updated point cloud. 3DGS is a subsequent optimization result. It does not natively provide watertight meshes, object identity, collision proxies, or physical properties. In this evolution it marks the first upgrade of the camera condition, from "describing rays" to "directly rendering coarse geometry". The subsequent GEN3C further turns this coarse geometry into an explicit external memory in long video.

GEN3C: replacing limited pixel history with an updatable 3D cache GEN3C [@srcV08] targets two problems. Multi-view conditions may contradict one another, and a limited context forgets content that leaves the frame in long videos. Its answer is a spatiotemporal 3D cache that holds the colored point cloud of every time step and every input view. Past content can then be re-rendered from a new camera instead of being recalled from latent memory alone.

The paper's own representation is an $L \times V$ point cloud array $P_{t,v}$. For the RGB image at time t and view v the method estimates depth, then applies the back-projection formula of Section 3.2 to build a colored point cloud. Single-image generation creates one cache element and copies it along time. Static multi-view gives each view its own cache. Dynamic multi-view makes each synchronized video stream a column of the cache. When the camera is not given, the paper's implementation can estimate it with DROID-SLAM.

Given a user camera C_t, the point cloud renderer outputs

$$(I_{t,v}, M_{t,v}) = R(P_{t,v}, C_t),$$

Here $I_{t,v}$ is the coarse color and $M_{t,v}$ is the coverage mask. The mask explicitly marks which positions in uncovered regions "need generation". Multi-view fusion does not first force all point clouds into a single geometry. Different depth estimates and lighting would make direct stitching produce ghosting. GEN3C instead encodes each view rendering with a frozen VAE, $z^v = E(I^v)$, downsamples the mask, and multiplies it element-wise with the rendered latent. That product is concatenated with the noisy latent of the target video. Its paper-specific fusion can be summarized as

$$z^{v\prime} = \mathsf{InLayer}\left(\mathsf{Concat}(z^v \odot M^{v\prime}, z_\tau)\right), \qquad z' = \mathsf{MaxPool}_{v=1}^{V} z^{v\prime}.$$

Cross-view max-pooling removes any need to fix the number of views or their permutation order. A fine-tuned video diffusion model then generates the target video under these rendering conditions.

Long videos use block-wise autoregression. Adjacent blocks overlap by one frame. Once a block is generated, depth is estimated only for the last frame of the block. The paper's supplementary material does not simply trust the scale of monocular depth. It solves for the scale s and offset b over the region that the old cache covers:

$$(s^*, b^*) = \arg\min_{s,b} \left\| (sD + b - D_{\mathsf{cache}}) \odot M \right\|_2^2.$$

The aligned RGB-D is then back-projected and appended to the cache. The next block renders its condition from that updated cache. This is the closed loop of "generate first, write back, then query".

The 3D cache adds a new capability, revisiting. After a wall leaves the frame, its sampled points can still be queried and rendered under a new viewpoint. The same cache also carries a risk of persistent contamination. Depth scale alignment constrains s, b only in regions the cache already covers, so newly appearing regions may still be wrong. If the temporal correspondence of dynamic objects is incorrect, ghosting is left at multiple positions in the cache. Depth blending at occlusion edges forms floating layers. Max-pooling can aggregate multi-view features, but it does not resolve geometric conflicts on its own.

GEN3C natively outputs a video and an internal point cloud cache, not a deliverable mesh. To enter the asset layer it must export a point cloud that carries camera, time, confidence, and provenance, then perform fusion or 3DGS optimization. If only the final video is exported, the interface loses the spatial value that the external memory provides.

ReconX: first filling in observations with generation, then solidifying them into 3DGS with confidence ReconX [@srcD01] recasts the ambiguity of sparse-view reconstruction as a video interpolation and extrapolation problem. What sets it apart from ViewCrafter is how explicitly it states the final goal. The target moves from "generating controllable novel views" to "using these novel views to optimize 3DGS".

The paper's algorithm starts from two or more sparse images that may have no known poses. First, DUSt3R predicts point maps and confidence for image pairs, and global alignment yields a unified point cloud $P = {p_i}_{i=1}^{N}$. Second, farthest point sampling selects a small number of query anchors. Those anchors aggregate raw point cloud features through cross-attention, which encodes a variable-length point cloud into a fixed-length three-dimensional context $F(P)$. Third, the first and last reference images become the endpoints of

video interpolation. The image condition and the three-dimensional context are each injected into the spatial cross-attention of the video U-Net. ReconX's paper-specific fusion is

$$F_{\text{out}} = \text{Softmax}\left(\frac{QK_g^\top}{\sqrt{d}}\right)V_g + \lambda_F \,\text{Softmax}\left(\frac{QK_F^\top}{\sqrt{d}}\right)V_F,$$

The image condition supplies the first term. The second term comes from the three-dimensional point cloud condition. Diffusion training still uses the noise prediction objective. Fourth, the model generates multiple video frames between the reference views and along the outer paths. Fifth, DUSt3R runs again, this time to match the generated frames pairwise. That yields global alignment and confidence among the generated views. Sixth, 3DGS starts from the point maps of the input endpoints and is then optimized on all generated frames.

ReconX does not count every generated pixel as equally valid ground truth. Let the pixel confidence of the i-th frame be C_i. Its paper-specific 3DGS objective weights the color, SSIM, and LPIPS errors by confidence:

$$\mathcal{L}_{\text{conf}} = \sum_i C_i \left[\lambda_{\text{rgb}}\mathcal{L}_1(\hat{I}_i, I_i) + \lambda_{\text{ssim}}\mathcal{L}_{\text{ssim}}(\hat{I}_i, I_i) + \lambda_{\text{lpips}}\mathcal{L}_{\text{lpips}}(\hat{I}_i, I_i)\right].$$

This is safer than treating generated frames as noise-free photographs. Even so, "low matching confidence" does not equal "the generated content is definitely wrong". Nor does it guarantee that regions which are semantically consistent but geometrically false will receive low weight. Suppose the video model keeps generating the same nonexistent window. Multi-frame matching may then assign it high confidence, and 3DGS will stably solidify the fake structure.

ReconX upgrades its output contract to a renderable 3DGS, which can answer appearance queries from arbitrary viewpoints. It still does not close topology, explicit surface normals, collision bodies, or physical materials. To connect to the next layer, it should retain, for each Gaussian, the provenance of whether it comes from real input or generated completion. It should also set different thresholds and review protocols for the two types of regions before surface extraction.

VideoScene: starting from coarse 3D and skipping the early steps of recovering structure from pure noise VideoScene [@srcD02] attacks the speed bottleneck that appears when video diffusion takes part in three-dimensional reconstruction. Its "one step" refers to the distilled video generation step. It does not refer to obtaining, in one step, an engine asset that has already passed physical validation.

The algorithm first receives two images and a camera. The pose may be given, or estimated by COLMAP or DUSt3R. It then uses a feed-forward sparse-view 3DGS model such as MVSplat to construct a coarse scene. Along the interpolated camera trajectory it renders a structurally consistent but possibly blurry video X_0^r. Traditional diffusion starts from pure noise. VideoScene instead encodes the coarse rendering into the latent space and adds noise at an intermediate timestep

$$x_t^r = \alpha_t x_0^r + \sigma_t \epsilon,$$

and then trains a student model to approximate how the teacher maps t_{n+1} to t_n. Its paper-specific consistency distillation objective is written as

$$\mathcal{L}_D = \mathbb{E}\left[d\left(f_\theta(x^r_{t_{n+1}}, t_{n+1}), f_{\theta^-}(\hat{x}^\phi_{t_n}(x^r_{t_{n+1}}), t_n)\right)\right],$$

where ϕ is the teacher and θ^- is the exponential moving average student. The dynamic denoising policy network DDPNet treats candidate jump timesteps as contextual bandit actions. When the coarse rendering is good, it chooses less noise to preserve structure. When the coarse rendering is distorted, it allows more noise for repair. Too much noise, however, will erase the 3D prior.

Compared with ReconX, the route moves toward ”feed-forward 3D initialization + local correction by a generative model + distillation acceleration”. It avoids making the denoiser reinvent the entire scene from pure noise. Yet it may still let visual completion cross real obstacles. The paper's supplementary material shows a case where the two input views differ greatly in semantics. There the generated camera path passes directly through a closed door. That case shows that a continuous transition in the pixel sequence carries no feasible path constraints. It also shows that DDPNet selects only visual denoising strength, not a collision planner.

Therefore, VideoScene's intermediate output should still go through pose re-estimation, reprojection, and visibility checks. Only then should it reach InstantSplat or another 3DGS reconstruction. Any clip that ”passes through a door but looks continuous” must be removed before NavMesh and collision construction. Its evidence level must not be raised just because generation is fast.

Video2Game: what truly closes the spatial construction chain is asset conversion, not more video frames Video2Game [@srcD04] is not a direct successor of the video diffusion chain above. It deals with the systems engineering problem of going from a real single video to a game environment. Yet it clearly shows which steps are still missing between ”a renderable representation” and ”an interactive asset”. For that reason it serves as the endpoint reference for this evolutionary chain.

Its paper-specific pipeline is divided into three layers. First, train a large-scene NeRF from posed video. Beyond density and color, add surface normals, sky, and monocular geometric cues. These improve how stably surfaces can be extracted from the radiance field. Second, run Marching Cubes on the NeRF density and post-process the mesh. UV unwrapping bakes multi-channel neural textures into texture maps. At runtime, a small GLSL shading MLP combines texture features and the view direction to recover view-dependent appearance. Third, split the scene into objects, either by semantics or by bounding box, and extract a mesh for each object on its own. Where surfaces are newly exposed, complete their textures by nearest neighbor.

The interaction layer does not directly use the high-fidelity visual mesh as a unified collision surface. Each rigid-body entity is represented as

$$e_i = (\mathcal{G}_i^{\text{vis}}, \mathcal{G}_i^{\text{col}}, m_i, f_i, T_i),$$

This is an engineering notation for the paper's interface, not the paper's original equation. Here $\mathcal{G}_i^{\text{vis}}$ is the visual mesh, $\mathcal{G}_i^{\text{col}}$ is the collision proxy, m_i and f_i are mass and friction, and T_i is the object transform. The paper supports four classes of collision geometry: box, sphere, convex polyhedron, and triangle mesh. Complex convex proxies can be generated with V-HACD. Mass and friction can be set by hand. The paper also discusses estimating them with a language model, but those estimates are not measured ground truth.

This pipeline shows why the output contract must be accepted layer by layer. NeRF optimization answers appearance and density. Marching Cubes yields a rasterizable surface. UV and neural textures yield real-time appearance. Object decomposition yields independent transforms. Only collision proxies and physical parameters yield contact and dynamics. No step can be automatically derived from the previous step. In particular, an erroneous NeRF density produces an erroneous isosurface. A simplified collision body expands or shrinks the traversable space. Erroneous mass and friction estimates make the collision response inconsistent with the real world.

Evolution logic of this chapter and the interface to the next layer Placed on the same line of evolution, these methods show that each generation replaces a different link. MotionCtrl separates the global pose and the local trajectory from ordinary video motion. AC3D further decides when and at which layers the camera condition should take effect. ViewCrafter replaces purely numerical poses with coarse point-cloud rendering, obtaining pixel-level constraints. GEN3C turns the point cloud into an updatable, revisitable external cache. ReconX solidifies newly generated observations into 3DGS through confidence weighting. VideoScene uses coarse 3D and distillation to reduce generation cost. Video2Game shows that after 3DGS or NeRF, surfaces, textures, objects, collision, and a runtime contract are still required.

Therefore, the minimal interface this chapter delivers to the reconstruction and asset layer should include the following. Camera intrinsics and extrinsics, plus the coordinate convention. Provenance labels for real

frames and generated frames. Per-frame depth, confidence, and occlusion mask. The provenance and time of each primitive in the point cloud or 3DGS. Markers of unobserved regions. Closed-loop reprojection residuals. And the constraint that downstream must not directly treat the visual representation as collision ground truth. Chapters 6–10 cover SfM, MVS, TSDF, surface extraction, and collision construction. Those steps are exactly what turns these incomplete observations into auditable spatial assets.

State, action, and observation must be kept separate The general state transition in a partially observable environment can be written as

$$s_{t+1} \sim p(s_{t+1} \mid s_t, a_t), \qquad o_t \sim p(o_t \mid s_t), \qquad r_t \sim p(r_t \mid s_t, a_t).$$

This is the general form of a POMDP and of world models. The true state s_t is often unobservable. From the history $h_t = (o_{\leq t}, a_{<t})$ the model can only build an approximate state. The fundamental difference between the various routes is where this approximation is placed. Dreamer places it in a recurrent latent state. Genie places it in discrete video tokens and historical actions. GameNGen and DIAMOND condition on a finite frame history to generate the next observation. V-JEPA 2 predicts the future in feature space. PERSIST explicitly evolves latent three-dimensional world frames, cameras, and a renderer.

Suppose the state exists only in a finite pixel history. Then doors, boxes, or terrain that leave the field of view slide away with the context window. If the state is a compact latent variable, it can support reward prediction and policy learning. An engine, however, cannot necessarily query it for "whether the door is open". If the state is object or three-dimensional scene data, only then can the system stably support saving, branching, revisiting, multi-user sharing, and collision queries.

RSSM and Dreamer: compressing history for imagination-based planning, not exporting 3D assets The Nature paper on DreamerV3 [@srcW03] builds on a recurrent state-space model (RSSM). Its paper-specific structure is

$$\begin{gathered} h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}), \\ z_t \sim q_\phi(z_t \mid h_t, x_t), \\ \hat{z}_t \sim p_\phi(\hat{z}_t \mid h_t), \\ \hat{r}_t \sim p_\phi(\hat{r}_t \mid h_t, z_t), \\ \hat{c}_t \sim p_\phi(\hat{c}_t \mid h_t, z_t), \\ \hat{x}_t \sim p_\phi(\hat{x}_t \mid h_t, z_t). \end{gathered}$$

h_t is the deterministic recurrent memory. z_t is the stochastic discrete representation, corrected by the current observation. The prior $p(\hat{z}_t \mid h_t)$ lets the model roll out when no future observation exists. The reward head, the continue-flag head, and the observation decoder head force the state to retain what the task needs. Training can be summarized as a combination of reconstruction, reward, continue prediction, and the prior–posterior KL. DreamerV3 adds stabilization techniques such as KL balance, free bits, symlog, and return normalization. The objective below is a conceptual reorganization of the paper's structure; it does not reproduce every coefficient item by item:

$$\mathcal{L}_{\text{WM}} = \mathcal{L}_x + \mathcal{L}_r + \mathcal{L}_c + \beta_{\text{dyn}} D_{\text{KL}}(\text{sg}\, q \| p) + \beta_{\text{rep}} D_{\text{KL}}(q \| \text{sg}\, p).$$

At run time the algorithm first writes real interactions into the replay buffer. It then obtains a posterior state from real observations. From some posterior state it rolls out multi-step "imagination trajectories" using only the prior dynamics and the actor. The critic estimates the returns of those trajectories. The actor improves accordingly. The algorithm then collects new data with the real environment again. This loop makes the model learn a compressed state useful for decision-making.

The RSSM's advantage is also its limitation for spatial construction. To predict reward, it can discard textures, distant rooms, or precise wall thickness that do not affect the current task. The policy may also exploit model error, finding high-return paths in imagination that do not exist in the real environment. Even if the decoded

frames are clear, (h_t, z_t) is not an editable mesh or a collision field. The Dreamer route therefore delivers no scene assets to a spatial system. What it delivers is "the latent state, reward, and termination prediction given an action." Where persistent space is needed, a geometry or object database must be connected as an independent state source.

Genie: discovering discrete control from video without action labels, but the state is still carried by token history Internet video has no action labels. Genie [@srcGDM-GENIE] addresses that problem with three parts, all explicitly disclosed in the paper: a spatiotemporal video tokenizer, a latent action model, and an autoregressive Dynamics Model.

Stage one compresses the video $x_{1:T}$ into discrete tokens $z_{1:T}$ with a spatiotemporal VQ-VAE. In stage two the latent action model looks at the history frames and the next frame. An encoder infers a continuous action, then quantizes it into a small VQ codebook. The decoder reconstructs the next frame from the history and that action alone. The decoder cannot see future frames. The action code must therefore summarize the most explanatory change from the current frame to the next. The paper's experiments limit the latent action vocabulary to 8, so that a person can explore its semantics the way one learns a new gamepad. That number is Genie's experimental setting, not a universal rule for all latent action models.

The third stage is a decoder-only MaskGIT Dynamics Model. It receives historical video tokens and stop-gradient latent action embeddings, and predicts the next frame tokens. Its paper-specific main supervision is

$$\mathcal{L}_{\text{dyn}} = -\sum_{t=2}^{T} \sum_{j \in \mathcal{M}_t} \log p_\theta(z_{t,j} \mid z_{<t}, z_{t,\setminus \mathcal{M}_t}, a_{<t}),$$

where $\mathcal{M}_t$ is the randomly masked token positions. During training the paper masks a relatively high ratio of middle-frame tokens at random. At inference it generates the next frame by multiple rounds of MaskGIT sampling. The latent action model is discarded at inference, except for the codebook. The user selects discrete action codes directly, and the Dynamics Model then autoregressively generates video.

Relative to the RSSM, this step changes the objective. The latent state no longer mainly serves the value function; it serves controllable observation generation. Latent actions let even unlabeled video learn interaction. However, one action code may mix "walk left," "camera pans left," and "background change" into a single discrete category, and the data distribution sets its semantics. Video tokens contain causal history, but they carry no stable object ID, map coordinates, inventory variable, or collision body. What happens after the character leaves the frame can only be guessed from the limited context and the model prior.

Genie therefore demonstrates that "low-dimensional changes that a human can control can be discovered from video." It does not demonstrate that "the game rules or the three-dimensional level have already been recovered." Research next turns to two paths. One lets a diffusion model predict the next observation directly from labeled actions, pursuing real time and visual detail. The other moves persistent state out of the pixel history.

GameNGen: predicting the next frame in real time with action-conditioned diffusion, and using noise augmentation to resist autoregressive drift GameNGen [@web685daea96198] trains in two stages on DOOM. First it trains a reinforcement learning agent to play the original game, keeping that agent's action–observation trajectories from the random stage to the proficient stage. Then it adapts Stable Diffusion v1.4 into a next-frame model. Actions no longer pass through text prompts. Each key combination maps to a token, which replaces the original text cross-attention. Past frames are encoded by the VAE and concatenated with the current noisy latent along the channel dimension.

The paper uses velocity parameterization. Its specific loss is

$$\mathcal{L} = \mathbb{E}_{t,\epsilon,T}\left[\left\| v(\epsilon, x_0, t) - v_{\theta'}(x_t, t, \phi(o_{i<n}), A_{\text{emb}}(a_{i<n})) \right\|_2^2\right].$$

Training uses teacher forcing, so the history frames are real game frames. At inference the history gradually becomes the model's own output, and the two produce a distribution shift. GameNGen's key correction adds Gaussian noise of random strength to the history frame latents during training and tells the network the noise level. The model then learns to recover from a slightly corrupted history. This is not persistent state, only a kind of training against rollout error. The paper reaches real-time generation with a 64-frame context and a small number of DDIM steps. It also explicitly observes that predicted trajectories diverge rapidly from real ones, because of tiny velocity differences.

GameNGen can maintain phenomena such as health, ammo, doors, and enemies in the frame. That does not mean these quantities exist as queryable variables. They may be merely implicit results of pixel patterns and limited history. The same visual frame may map to different ammo, enemy states, or map positions. The model has no external key-value to tell them apart. The output contract is the next observation conditioned on actions, not a level file, script state, or collision geometry.

DIAMOND: the diffusion observation model preserves detail, while reward and termination are still handled by independent models DIAMOND [@web3b350556ce83] applies diffusion to reinforcement learning world models. The motivation is that an overly compressed discrete latent may lose small objects, score items, and obstacle details that are important to the policy. DIAMOND does not adopt Genie's discrete video token dynamics. It learns the conditional next-observation distribution directly.

In a POMDP, DIAMOND approximates the state with a finite history and trains a conditional EDM denoiser. The paper-specific objective can be written as

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left\|D_\theta(x^\tau_{t+1}, \tau, x^0_{\le t}, a_{\le t}) - x^0_{t+1}\right\|_2^2\right].$$

EDM preconditioning mixes the noisy next observation with the U-Net prediction according to the noise strength. Past observations are concatenated along the channel dimension, and actions enter the residual blocks through adaptive group normalization. The paper solves the reverse process with the Euler method. It emphasizes long-rollout stability under very few function evaluations. Reward and termination are scalar tasks, so a separate CNN-LSTM model R_ψ predicts them. The actor-critic is then trained entirely in the imagination of this world model. The real environment only collects data at intervals.

Relative to GameNGen, this structure adds a closed loop of reward, termination, and policy for reinforcement learning. It also demonstrates that better visual detail may improve control. But observation diffusion, the reward model, and the termination model stay separate, and the three may contradict one another. The frame shows the reward object still present, yet the reward head has already issued the reward. The enemy in the frame has disappeared, yet the termination head still believes the episode continues. Facts outside the history window are likewise not explicitly stored. DIAMOND therefore still belongs to ”limited pixel history + action-conditioned observation generation”, and it cannot deliver three-dimensional state directly to the geometry layer.

V-JEPA 2: planning without generating pixels, but latent goal distance is not geometric distance V-JEPA 2 [@srcW11] takes another route. It does not ask the model to reconstruct all pixels. Instead it predicts occluded or future video representations in a joint embedding space. Action-agnostic pretraining first lets the encoder learn motion and semantics from large-scale video. The encoder is then frozen, and a roughly 300-million-parameter, block-causal action-conditioned predictor, V-JEPA 2-AC, is trained on DROID robot data. It then autoregressively predicts the next-frame representation from the history representation, the end-effector state, and the action.

Planning uses Equation 5 of the paper. The current image and the goal image are encoded as z_k, z_g. The predictor P rolls out candidate action sequences, and the energy is given by

$$E(\hat{a}_{1:T}; z_k, s_k, z_g) = \left\|P(\hat{a}_{1:T}; s_k, z_k) - z_g\right\|_1, \qquad a^*_{1:T} = \arg\min_{\hat{a}_{1:T}} E.$$

The paper samples candidates from a Gaussian action distribution with the Cross-Entropy Method. It keeps the low-energy elite trajectories to update the mean and variance, and executes only the first action each time. It replans after observing the new image. This is closed-loop MPC, not one-shot generation of an entire robot action segment.

The JEPA route has one advantage over pixel diffusion: it does not spend computation reconstructing texture detail that is irrelevant to the task. Latent-space planning is also faster than video diffusion planning. Its limitations are equally concrete. The L1 in the energy is a representation distance, not meters, collision distance, or joint safety margin. If the encoder is insensitive to transparent obstacles, thin ropes, or precise gripper clearances, low-energy trajectories may still fail physically. The paper also notes sensitivity to camera position and the accumulation of error over long rollouts. V-JEPA 2-AC outputs future representations and candidate action energies, not a renderable three-dimensional world.

How errors roll along the world model and enter spatial assets RSSM error first appears as divergence between the prior $p(z_t \mid h_t)$ and the true posterior. The reward head and the actor are then optimized jointly on that erroneous latent state. Genie's error appears when erroneous tokens are treated as the history of the next frame, so latent action semantics drift as rollout proceeds. GameNGen and DIAMOND pass their errors directly into the next observation condition, and small artifacts in the frame may be amplified frame by frame. V-JEPA 2's error distorts the distance between future representations and the goal representation, which leads the CEM to select the wrong actions. An explicit three-dimensional state model may write a single erroneous generation into the long-term world frame, which makes the error more persistent than a pixel artifact.

Propagation cannot be prevented by better single-step PSNR alone. Each of these must be tested separately: single-step prediction, long rollouts, multiple action branches, revisiting after leaving, multiple cameras for the same state, object permanence, collision decisions, deterministic replay. A world model that outputs content to a reconstruction or engine layer must at minimum carry the state version, action history, random seed, visible region, generated region, confidence, and rollback point. Visual observation can only ever be one projection of state. It must not, conversely, become the sole basis for collision and scripting.

L1: NeRF, Neural Rendering, and Queryable Appearance of 3DGS

For L1 the promotion gate is whether the appearance of the same scene can still be queried from a new camera. The comparison here covers NeRF, the rendering side of neural SDFs, and the original 3DGS. High rendering quality, real-time speed, or explicit Gaussian centers do not equal an explicit surface.

The Core Contradiction of Representation Evolution: Pixel Fitting Does Not Uniquely Determine the Surface Classical MVS picks depths for pixels first, then fuses points or voxels. Implicit neural representations write the whole space as a continuous function. Through differentiable rendering, all sampled points jointly explain the image. That can share statistical information in sparse regions, and it also avoids fixing the grid resolution in advance. Yet it produces a new unidentifiability. A single set of training images can be explained jointly in several ways: by a thin surface, thick density, view-dependent color, or multiple semi-transparent layers. 3DGS in turn replaces the many MLP queries along each ray with sortable explicit ellipsoidal primitives, markedly improving training and rendering speed. Yet it defines no mesh adjacency for the primitives. Anyone tracing this evolution must ask two separate questions: ”how does it render” and ”how does it define a surface.”

NeRF: Discrete Integration Along Rays, Not Attaching Pixels Directly to 3D Points The original NeRF paper and its project page [@srcNERF-2020] represent the mapping from position and direction to volume density and color with a network F_θ:

$$(\sigma, c) = F_\theta(\gamma(x), \gamma(d)),$$

$x \in \mathbb{R}^3$ is the spatial position and d is the viewing direction; γ denotes a high-frequency positional encoding. The original design makes density a function of position alone. Color, by contrast, depends on position and direction together. A camera ray $r(t) = o + td$ is sampled at $t_1 < \cdots < t_N$ between the near and far bounds, where $\delta_i = t_{i+1} - t_i$. Discrete volume rendering in the NeRF paper can be written as

$$\hat{C}(r) = \sum_{i=1}^{N} T_i \alpha_i c_i, \qquad \alpha_i = 1 - \mathsf{exp}(-\sigma_i \delta_i), \qquad T_i = \prod_{j<i} (1 - \alpha_j) = \mathsf{exp}\left(- \sum_{j<i} \sigma_j \delta_j \right).$$

α_i is the discrete probability that light terminates within this small segment. T_i is the transmittance accumulated before that segment is reached. Training is supervised by pixel color, for example

$$\mathcal{L}_{\mathsf{rgb}} = \sum_{r \in \mathcal{R}} \| \hat{C}(r) - C^{gt}(r) \|_2^2.$$

The original NeRF runs a hierarchical coarse sampling pass, then performs importance resampling according to the coarse network weights. Occupancy grids, hash encodings, and proposal networks appear in later implementations. They are acceleration or parameterization evolutions, and should not be written back as components of the original paper. At runtime the method holds at least a camera/ray batch, sample points, network parameters, spatial bounds, and sampling/acceleration state. The basic procedure is:

Indented procedure or pseudocode in the original draft

```
Generate rays o,d from the pixel and the camera
→ sample t_i between the near and far bounds to obtain x_i=o+t_i d
→ query the network for sigma_i and c_i
→ compute alpha_i, the accumulated transmittance T_i, the composite color, and the expected depth
→ compare with the ground-truth RGB and back-propagate
→ update the network and the optional camera parameters
```

NeRF can output color from arbitrary views, plus expected depth and density samples. Density is not a signed distance, however. Provided $\hat C$ is correct, optimization may accept floating density, thick surfaces, or view-dependent color that masks geometric errors. Beyond the sparse views, the function extrapolates color. Choosing a density threshold arbitrarily and running Marching Cubes is another surface estimate: shift the threshold and the shell inflates, splits, or disappears. No natural inside/outside exists. The native NeRF output contract should therefore be written as "radiance field + camera + bounds + sampling/acceleration structure + rendered depth and uncertainty." It must not be written as a "collidable mesh."

VolSDF and NeuS: Two Different SDF Differentiable Rendering Mechanisms A neural network $f_\theta(x)$ can represent the SDF, and that is one way to make the surface a first-class object of the model. The surface is then defined as

$$\mathcal{S} = x \mid f_\theta(x) = 0.$$

That sign separates inside from outside. The normal follows as $n = \nabla f / \|\nabla f\|$. A true signed distance function satisfies $\|\nabla f\|_2 = 1$ almost everywhere, which is why an Eikonal regularizer is often added

$$\mathcal{L}_{\mathsf{eik}} = \mathbb{E}_{x \sim \Omega} \left(\|\nabla_x f_\theta(x)\|_2 - 1 \right)^2.$$

Eikonal is a general constraint on SDFs. The density/opacity mappings below belong instead to individual papers. The two must not be conflated.

VolSDF [@srcN003] turns the SDF into volume density through the cumulative distribution of a Laplace distribution. One of the paper's sign conventions allows the map to be written as

$$\sigma(x) = \alpha \Psi_\beta(-f_\theta(x)),$$

Here Ψ_β is the zero-mean Laplace CDF with scale β. Density magnitude is governed by α, and the surface transition bandwidth by β. NeRF-style volume rendering integration is still used afterwards. VolSDF induces density from the SDF, and it offers an error-bound idea for sampling. Near the zero surface, however, it still explains the image through a density of finite width.

NeuS [@srcN002] does not simply reuse the Laplace density above. It derives weights with better unbiasedness from two sources: the logistic distribution and the variation of the SDF along the ray. Expressed with the logistic CDF Φ_s, its discrete mechanism can construct adjacent samples as

$$\alpha_i = \max\left(\frac{\Phi_s(f_i) - \Phi_s(f_{i+1})}{\Phi_s(f_i)}, 0\right),$$

Color is then composited with the forward transmittance. This simplified form rests on conventions about the orientation of the SDF, and about local monotonicity along the ray. The actual NeuS also estimates interval endpoints through the SDF gradient, and it carries an annealing strategy. Its core purpose is to give the main weight to the interval that crosses the zero level set. It also reduces the problem of a density peak that is systematically biased to one side of the surface. The label "VolSDF with a different CDF" loses the derivational differences between the two papers.

Both families usually combine RGB, mask, Eikonal, sparse depth, or normal terms. When the camera is inaccurate, the pose can also be jointly fine-tuned. Once the SDF is available, one queries f on a bounded three-dimensional grid or an adaptive octree, runs Marching Cubes on the zero isosurface, and computes gradient normals. Compared with extracting a surface from a NeRF density threshold, this is more definitional. It is not automatically true, however. Eikonal favors regular, smooth distance fields, and thin sheets smaller than the sampling scale may be erased. An incorrect camera can be absorbed jointly by the smooth surface and the color network. Unobserved back sides may be filled in as a closed shell. When the training bounds are too small, the bounding box truncates the isosurface. A neural SDF's output contract must be accompanied by the coordinate scale, sampling bounds, inside/outside sign, zero-surface threshold, training mask, observation coverage, gradient quality, and surface-extraction resolution.

Original 3DGS: Covariance Projection, Forward Alpha Compositing, and Adaptive Densification The original 3D Gaussian Splatting paper [@src3DGS-2023-N005] drops the continuous MLP in favor of a set of anisotropic Gaussians. Each k-th primitive carries a mean $\mu_k \in \mathbb{R}^3$, a rotation R_k, a scale s_k, an opacity parameter o_k, and spherical-harmonic color coefficients. To keep the covariance positive semidefinite, the paper parameterizes it by scale and rotation:

$$\Sigma_k = R_k \,\mathrm{diag}(s_k^2) R_k^\top.$$

The camera transform comes first, and the perspective projection is linearized at the mean. Let W denote the linear part of the world-to-camera transform, and J_k the Jacobian of the projection function at the camera-space mean. The screen-space covariance is then approximated as

$$\tilde{\Sigma}'_k = J_k W \Sigma_k W^\top J_k^\top, \qquad \Sigma'_k = \left[\tilde{\Sigma}'_k\right]_{1:2,1:2}.$$

In practice, the screen ellipse keeps the upper-left 2×2 block of this matrix, and adds numerical stabilization such as a minimum screen footprint. Once these low-pass details are ignored, the elliptical Gaussian response and the effective alpha at a pixel x can be written as

$$G_k(x) = \exp\left[-\tfrac{1}{2}(x - \mu'_k)^\top (\Sigma'_k)^{-1}(x - \mu'_k)\right], \qquad a_k(x) = \mathrm{sigmoid}(o_k) G_k(x).$$

Gaussians that cover the same tile are sorted by view-point depth. Color is then composited forward, front to back:

$$C(x) = \sum_{k \in \mathcal{N}_x} c_k(d) a_k(x) \prod_{j<k} (1 - a_j(x)).$$

Viewing direction enters through the spherical harmonics, which let $c_k(d)$ vary. The original paper initializes the means from SfM sparse points. It then optimizes position, scale, rotation, color, and opacity under a combined loss of L_1 on RGB and D-SSIM. The GPU rasterizer culls by the view frustum and the screen bounding box, and writes the Gaussian instances into the key--value lists of the tiles they cover. It sorts by "tile ID + depth," then forward-composites and backpropagates in per-tile thread blocks. Its data structures are the Gaussian parameter arrays, tile ranges, sort keys, and optimizer state. No edge table or face table exists.

A fixed number of primitives struggles to cover flat regions and high-frequency detail at once. The original method therefore performs adaptive density control at intervals, driven by view-space position gradients and scale. Smaller primitives with high gradients can be cloned. Larger primitives with high gradients can be split into smaller ones. Primitives with low opacity, or an abnormally large size, are pruned, and opacity is periodically reset for redistribution. The mechanism can be written as:

Indented procedure or pseudocode in the original draft

```
Initialize Gaussian(mu, scale, rotation, opacity, SH) from SfM points
for each iteration:
frustum cull and assign to tiles
sort by tile_id and depth
front-to-back alpha compositing
compute L1 and D-SSIM, and back-propagate to update all parameters
accumulate view-space gradients, visibility, and scale statistics
at densification steps:
clone small primitives with high gradients
split large primitives with high gradients
prune low-opacity or pathologically large primitives
```

Densification is a rendering-quality mechanism. It is also a source of resource risk and geometric instability. Incorrect poses, or training images that contradict each other, leave persistent pixel residuals; the optimizer may then memorize individual views with more floating Gaussians. Elongated ellipsoids can cover color along a ray while corresponding to no real thin surface. Sky and reflections form distant or large-scale primitives. Aggressive pruning deletes thin lines and semi-transparent objects. Weak pruning lets GPU memory, sort volume, and rendering time grow. The 3DGS output contract should therefore include the Gaussian parameters, spherical-harmonic order, coordinate/scale, cameras, training-image version, densification/pruning configuration, Gaussian count and resource curves, visibility, and low-confidence regions. Storing primitives explicitly is not the same as storing a surface explicitly. That a file is viewable as a PLY does not prove that its topology or collision is usable.

Chapter 7 Output Contract and the Link to Surfacing from Point Clouds NeRF's native capability is continuous novel-view querying. Neural SDFs add an explicit zero level set, and Gaussians add real-time explicit differentiable rendering. From all of them one can export depth, normals, and surface samples, but they become a candidate mesh only after threshold, coverage, and topology validation. The next layer needs Chapter 7 to deliver: sample point positions, normals, and confidence; which cameras or primitives each point/face comes from; the observed regions and the filled-in regions; the implicit field bounds and the isovalue threshold; a reproducible version of the Gaussians or the network; and the coordinates, scale, and defect report of the visual mesh.

The next chapter no longer assumes that these points connect naturally into a surface. It asks instead "how to infer a surface from unordered, noisy samples of uneven density whose normals may point the wrong way."

BPA, Poisson, alpha shape, and TSDF/Marching Cubes differ essentially in the assumptions they make about local sampling, global closure, and unknown space.

L2: Camera, Depth, Point-Map, and Point-Cloud Spatial Samples

L2 natively delivers measurable spatial samples such as cameras, depth, point maps, point clouds, or surfels. Acceptance focuses on scale, pose, confidence, multi-view consistency, and free- versus unknown-space semantics, not on the number of points or on whether a file can be loaded.

Observability: Reconstruction Is First a Constraint Problem, Not a Model-Name Problem The i-th image is I_i, with camera intrinsics K_i and world-to-camera extrinsics $T_{cw,i} = [R_i|t_i]$; the 3D points are $X_j \in \mathbb{R}^3$. The ideal pinhole projection takes the form

$$u_{ij} = \pi\left(K_i(R_i X_j + t_i)\right),$$

where u_{ij} is the 2D coordinate of point j in image i, and $\pi([x, y, z]^\top) = [x/z, y/z]^\top$. One pixel yields a single ray, and cannot give the distance along that ray. Two pure-rotation images share no triangulation baseline. Without a known object size, a stereo baseline, IMU/GNSS, or a depth anchor, a monocular sequence can be recovered only up to an overall similarity transformation. So "1 unit" in the output of a classical reconstruction cannot be treated as "1 meter" in an engine without calibration. Reflective and transparent surfaces break the assumption that the same 3D point has a stable appearance. Dynamic objects break the static-world assumption.

An auditable reconstruction task must first declare whether the intrinsics are known, whether distortion is corrected, whether the images are synchronized, and whether the scene is approximately static. It must also state whether an absolute scale exists, how much unobserved area is allowed, and whether the downstream need is novel views, measured surfaces, or collision assets. The algorithm that follows does no more than make those assumptions concrete.

Incremental SfM: How Features, Geometric Verification, Triangulation, and Bundle Adjustment Connect Structure-from-Motion Revisited [@srcG003] represents incremental SfM, which is not "a point cloud obtained from a single network inference." It is a graph algorithm with fallbacks and repeated optimization. Its core steps are as follows.

Step 1: Detect local features and build candidate matches. Within each image, detect keypoints u_{ik} that are relatively stable in scale and rotation, and compute descriptors d_{ik}. Candidate image pairs can be produced by temporal adjacency, GPS, global bag-of-words retrieval, or exhaustive matching. For a candidate pair (i, j), the nearest-neighbor ratio and mutual nearest-neighbor tests filter out obviously wrong descriptors. What is stored here is not a dense tensor. It is an image table, keypoint arrays, descriptor matrices, and a sparse edge table of image pairs. Wrong matches also sit very close together in descriptor space when textures are highly repetitive, so a large number of matches does not mean strong geometric constraints.

Step 2: Verify two-view geometry with RANSAC. When the cameras are uncalibrated, estimate the fundamental matrix F. When they are calibrated, the essential matrix E can be estimated. Correct matches should satisfy the epipolar constraint

$$\tilde{u}_j^\top F \tilde{u}_i \simeq 0.$$

In practice, the geometric distance is commonly approximated by the Sampson error:

$$e_{\text{samp}} = \frac{(\tilde{u}_j^\top F \tilde{u}_i)^2}{(F\tilde{u}_i)_1^2 + (F\tilde{u}_i)_2^2 + (F^\top \tilde{u}_j)_1^2 + (F^\top \tilde{u}_j)_2^2}.$$

The original RANSAC paper [@srcG012] describes one general mechanism. Draw a minimal sample, fit a model, count inliers against a threshold, then re-estimate from all inliers. Let the minimal sample size be s and the inlier ratio be w. To draw at least one all-inlier sample with probability p, the theoretical number of iterations is approximately $N = \log(1-p)/\log(1-w^s)$. A low inlier ratio therefore makes computation grow sharply. The same expression also shows that a fixed pixel threshold must be adjusted with resolution and noise. Pure rotation, planar scenes and extremely small baselines may make a homography fit better than an essential matrix. A system that does not compete among degenerate models will mistake image pairs that cannot be triangulated for good initializations.

Step 3: Select the initial image pair and triangulate. Decomposing the essential matrix yields candidate R,t$. The four solutions are ambiguous. The one with the most positive depths resolves that ambiguity. For a feature track $\mathcal{O}_j$ formed by multiple images, a linear DLT gives an initial 3D point. Minimizing the multi-view reprojection error then refines it. A triangulation angle that is too small makes depth extremely sensitive to pixel noise. One that is too large may bring appearance changes and occlusion. Practical systems check positive depth, angle, reprojection error and track length together. They do not keep a point as soon as two rays intersect.

Step 4: Register new cameras and extend the observation graph. For an unregistered image, collect 2D–3D correspondences between its 2D features and existing 3D points, then solve for R_i, t_i with PnP-RANSAC. Once registration succeeds, continue triangulating tracks that have not yet become points, matching the new image against its already-registered neighbors. The next image is usually chosen as the one with enough visible 3D points that are well distributed in space. Looking only at the number of correspondences concentrates points in one corner of the image and yields a numerically unstable pose.

Step 5: Local and global bundle adjustment. The expression below is a generic BA objective. It is not equivalent to all the robustification, camera models and scheduling details of any particular software package:

$$\min_{R_i,t_i,K_i,X_j} \sum_{(i,j)\in\mathcal{O}} \rho\left(\left\|u_{ij} - \pi\left(K_i(R_iX_j + t_i)\right)\right\|^2_{\Sigma_{ij}^{-1}}\right).$$

$\mathcal{O}$ is the edge set of the bipartite observation graph. π includes the chosen perspective and distortion models. Σ_{ij} is the pixel measurement covariance, and ρ is a robust kernel such as Huber or Cauchy. The optimization variables carry gauge freedom in overall scale, rotation and translation. An implementation must therefore fix the first camera and the scale, remove the corresponding degrees of freedom, or add an identifiable prior. Otherwise the Hessian is singular. Distortion parameters and focal length can both be free at once. If the viewing angles are insufficient, the two will compensate for each other. The Jacobian matrix is sparse, because each observation depends on only one camera and one point. Engineering implementations usually eliminate point variables first via the Schur complement, then solve the smaller camera normal equations. Camera blocks, point blocks, observation edges, robust weights and residual statistics are therefore formal data structures, not debugging information.

The incremental loop can be written as:

Indented workflow or pseudocode in the original manuscript

```
features = detect_and_describe(images)
candidate_pairs = retrieve_pairs(images)
for each pair:
matches = mutual_ratio_match(pair)
model, inliers = ransac_F_or_E(matches)
if nondegenerate(model, inliers):
add_verified_edge(pair, inliers)
seed = choose_seed_by_baseline_and_coverage()
initialize_poses_and_triangulate(seed)
while an image has enough 2D-3D correspondences:
pose = pnp_ransac(next_image)
if pose passes quality gates:
```

```
triangulate_new_tracks()
local_bundle_adjustment()
prune_by_reprojection_angle_and_cheirality()
else:
quarantine_image()
global_bundle_adjustment_with_gauge_fix()
```

How failures propagate. A wrong descriptor correspondence that passes through RANSAC forms a wrong track. A wrong track shifts the PnP pose. During triangulation, the shifted pose in turn interprets originally correct pixels as wrong depths. Bundle adjustment may reduce the overall residual while ”jointly explaining” systematic errors by moving cameras and points, especially under repeated facades, rolling shutter and unmodeled distortion. Once downstream MVS treats these cameras as ground truth, the same plane falls at different depths in different views. Fusion then produces double walls or thick shells. The output contract of SfM must therefore include the coordinate system and scale state, plus each camera's intrinsics and extrinsics with quality/uncertainty. It must also record each point's 3D coordinates, color, track length, triangulation angle and reprojection statistics, along with the observation edges and the rejected images. A standalone PLY sparse point cloud is an incomplete contract.

MVS: From Plane Sweep to Cost Volume, Then From Depth Maps to Fused Surface Samples SfM resolves ”where the cameras are” and a small number of stable points. Only MVS estimates the depth of each reference pixel under known cameras. Classical plane sweep selects a set of fronto-parallel or general spatial planes for the reference camera. Consider a plane in reference camera r with normal n and distance hypothesis d_k. A plane-induced homography can transform source view s into the reference view. Under one common coordinate convention, the form is

$$H_{s\leftarrow r}(d_k) = K_s \left(R_{sr} + \frac{t_{sr} n^\top}{d_k} \right) K_r^{-1}.$$

If the literature uses the opposite plane equation, normal or camera transformation direction, the signs in the expression change. An implementation must therefore write the coordinate convention into its synthetic-plane tests, and must not copy the matrix verbatim. For each pixel x and depth layer k, the colors or features of multiple source images are sampled through $H(d_k)$ onto the same hypothesized 3D point. SSD, NCC or learned feature variance is then computed. At the correct depth, the multi-view appearance should be more consistent:

$$C(x,k) = \frac{1}{N} \sum_s \left(f_s(H_{s\leftarrow r}(d_k)x) - \bar{f}(x,k) \right)^2 .$$

This is the generic cost-volume expression of plane sweep. It does not mean that all MVS methods use the same features, aggregation or depth regression. Under its world-to-camera parameter convention, MVSNet [@srcG004] writes it specifically as

$$H_i(d) = K_i R_i \left(I - \frac{(t_1 - t_i) n_1^\top}{d} \right) R_1^\top K_1^{-1},$$

Camera 1 is the reference camera and n_1 is its principal axis. After unifying the convention, this is equivalent to the relative-pose form above. The paper warps deep features of the source views through this differentiable homography. Cross-view variance yields an $H \times W \times D \times C$ cost volume, which a 3D CNN regularizes. It then performs expectation regression on the depth probabilities and refines the result with the reference image. PatchMatch, traditional photometric propagation, cascaded cost volumes and subsequent Transformer MVS have different search and aggregation mechanisms, and cannot all be called MVSNet.

The depth estimate in discrete depth-probability implementations of the MVSNet type can be written as

$$\hat{d}(x) = \sum_{k=1}^{D} P(k \mid x) d_k.$$

Cost-volume GPU memory grows approximately with HWD. A depth range that is too wide wastes resolution. One that is too narrow excludes the true surface from the search space. Cascaded MVS uses coarse-level results to narrow the fine-level frustum interval. That is essentially allocating a limited depth sampling budget.

Each reference depth map is still not the final surface. Four types of checks are usually performed before fusion. The depth probability peak or entropy gives a photometric confidence. Reference points are projected into source views and back-projected, to check the reprojection position and relative depth difference. A visibility test excludes occlusions. Samples are merged only when multiple reference views support them. A depth d is back-projected through

$$X_w = T_{wc,r} \left(d K_r^{-1} \tilde{x} \right)$$

into a world point. Neighboring points are aggregated by spatial distance, normal and color, to obtain a dense point cloud or to be fed into a TSDF. Practical data organization includes depth/normal/confidence maps stored per reference view, view-selection lists, depth ranges and sampling strategies. It also keeps the visibility relation from pixels to supporting views, and provenance indices from fused points to the original observations.

How failures propagate. In textureless regions the cost volume is nearly flat along the depth direction, so the probability expectation outputs a distribution average rather than the true surface. Repeated textures produce multiple peaks. Specular surfaces vary with viewpoint, which makes features at the true depth less consistent instead. Occlusion boundaries mix foreground and background features into the same voxel. A confidence threshold that is too loose yields double layers at fusion. One that is too strict removes thin poles, edges and weakly textured walls. If the "novel views" completed by video diffusion are inconsistent across views, the cost volume will also forcibly explain them as geometry. MVS output must therefore distinguish direct observations, model completion and low-confidence regions, and must retain the number of view supports for each point.

RGB-D, TSDF, and SLAM: Fusing History Also Requires the Ability to Revise History RGB-D or LiDAR already provides a distance for each frame, but per-frame noise, holes and pose errors still require fusion. Curless–Levoy volumetric fusion [@srcG001] projects the spatial voxel center x into the current depth map. The convention adopted here is "positive when the predicted depth from the camera to the voxel is smaller than the observed surface depth":

$$s_i(x) = D_i(\pi(T_{cw,i} x)) - z_i(x), \qquad f_i(x) = \mathsf{clip}(s_i(x)/\mu, -1, 1).$$

D_i is the observed depth, $z_i(x)$ is the voxel's depth in camera coordinates, and μ is the truncation bandwidth. Some implementations define the opposite sign, but the zero level set is the same. When deriving normals and inside/outside, the sign convention must be stored along with them. The common per-voxel update of the historical TSDF F and weight W is

$$F'(x) = \frac{W(x)F(x) + w_i(x) f_i(x)}{W(x) + w_i(x)}, \qquad W'(x) = \mathsf{min}\left(W(x) + w_i(x), W_{\mathsf{max}}\right).$$

w_i may vary with sensing noise, incidence angle and depth, and not all implementations use constant weights. The neighborhood of $F = 0$ is the candidate surface. Under the convention of this paragraph, the observed truncation band in front of the surface is positive. Voxels that have never been updated are unknown space. Unknown is not free space, a semantics that ordinary point clouds do not have.

KinectFusion [@srcG002] performs ICP between the current depth and the predicted surface obtained by raycasting the global TSDF. A common point-to-plane tracking objective is

$$\min_{\xi} \sum_{k} w_k \left[n_k^{\top} \left(\exp(\hat{\xi}) p_k - q_k \right) \right]^2 ,$$

p_k is a point in the current frame, q_k, n_k are the corresponding model point and normal, and $\xi \in \mathfrak{se}(3)$ is the pose increment. After linearizing for small motion, a 6×6 normal equation is solved. A coarse-to-fine pyramid enlarges the region of convergence. The engineering loop is "preprocess depth predict the global model establish correspondences solve for pose quality gating fusion update the surface". When quality gating fails, fusion must stop. Otherwise a single tracking failure will be permanently written into the map.

A large space cannot be allocated as a dense cube, so voxel block hashing is common. A hash table maps integer block coordinates to fixed-size voxel blocks. Within each block it compactly stores the TSDF, weights, color and the most recent update time. An octree or sliding submap may also be used. Online SLAM further maintains keyframes, feature/dense constraints, a covisibility graph and a pose graph. After a loop closure is detected, the general objective of the pose graph can be written as

$$\min_{T_i} \sum_{(i,j)\in\mathcal{E}} \left\| \log \left(Z_{ij}^{-1} T_i^{-1} T_j \right) \right\|_{\Omega_{ij}}^2 ,$$

Z_{ij} is the measured relative pose and Ω_{ij} is the information matrix. The key point is that the old depth was written into the TSDF under the old pose, before the trajectory was corrected. Updating only the camera trajectory will not make a thick wall become thinner by itself. BundleFusion [@srcG006] demonstrates surface re-integration after global optimization. ElasticFusion [@srcG005] maintains surfels and a deformation graph, so that historical surfaces deform with the loop closure. The two follow different implementation routes, but both show that the map state must be able to be undone, deformed or recomputed.

How failure propagates. A depth bias changes the TSDF zero level set. Errors in normals and correspondences shift the ICP pose. The pose shift in turn aligns the next frame with the erroneous model, which forms positive feedback. A truncation band that is too wide erases thin walls and narrow gaps. One that is too narrow cannot form a continuous zero surface under noise. After weights saturate, new observations find it difficult to correct old errors. If a dynamic person is fused as a static surface, it leaves a ghost volume. A false loop closure may force two similar corridors to coincide. A complete output should include versioned poses, keyframes, replayable depth, voxel resolution, truncation distance, sign conventions, weights/coverage, free—occupied—unknown states, loop-closure edges, re-integration versions and the final isosurface.

Learned TSDF, DUSt3R, and VGGT: what feed-forward geometry changes is the initialization cost Learned reconstruction writes the data prior into the traditional state. NeuralRecon [@srcG009] back-projects multi-view features into a sparse 3D volume. It uses sparse convolutions and a GRU to update the hidden state across fragments, predicts local occupancy/TSDF, and then extracts surfaces with Marching Cubes. It partly avoids the accumulation of "per-frame depth error followed by independent fusion". If the voxels completed by the network and the direct observations are not kept in separate layers, however, training-domain bias will be treated as a definite surface. NICE-SLAM [@srcG010] uses multi-level local neural feature grids to represent geometry and color, and continues to optimize them during tracking and mapping. It replaces the fixed grid function of the TSDF with a learnable implicit function, yet still requires poses, keyframes and local/global consistency management.

DUSt3R [@srcG011] does not need camera intrinsics or extrinsics up front. For an image pair (I_i, I_j), it regresses two per-pixel pointmaps and their confidence directly. Each pixel no longer outputs only a scalar depth, but a 3D point in a reference frame of the image pair. The 3D relations among those pointmaps serve the matching role as well. For multiple images, every edge $e = (n, m)$ gives two pointmaps $X^{n,e}, X^{m,e}$ in

one local coordinate frame. The paper solves for a world-coordinate pointmap χ^v per image, and for a rigid transformation P_e and a positive scale σ_e per edge. Its global alignment objective is

$$\min_{\chi,P,\sigma} \sum_{e=(i,j)} \sum_{v\in i,j} \sum_{p} c_{e,p}^{v} \left\| \chi_p^v - \sigma_e P_e X_p^{v,e} \right\|_2, \qquad \prod_e \sigma_e = 1.$$

c is the network confidence. The constraint $\prod_e \sigma_e = 1$ rules out the trivial solution in which all scales shrink to zero at once. The same P_e maps both pointmaps of the pair to the world pointmap at once, so the predictions of a shared image must stay consistent across its different edges. The paper also gives an extension in which a pinhole model parameterizes χ^v and recovers the camera and depth. The core change compresses the past discrete chain of "descriptor matching essential matrix triangulation" into a feed-forward pointmap. Edges are then merged by 3D alignment rather than traditional 2D reprojection BA.

VGGT [@srcVGGT-2025] places multi-frame patch tokens and camera tokens into a shared Transformer. One forward pass then jointly predicts the camera, depth, pointmaps, and point tracks. The model reduces pairwise module boundaries, so sparse, low-texture inputs can also obtain strong initial values from the training prior. Its native data structure is still "frame-level camera tensors + pixel-level depth/pointmaps/confidence + tracks." It is not a half-edge mesh or a physical scene graph.

A more reliable production pipeline does not treat classical optimization and feed-forward models as alternatives. Instead it does this:

Indented workflow or pseudocode in the original manuscript

```
The feed-forward network outputs cameras, depth/point maps, trajectories, and confidence
→ build an observation graph from high-confidence trajectories and isolate low-confidence and dynamic pi
→ perform local BA / pose graph correction with reprojection, scale anchors, and loop-closure constraint
→ re-fuse depth or align the point maps according to the corrected poses
→ store the direct-observation mask and the model-completion mask separately
→ proceed to the implicit field, Gaussian, or point-cloud surfacing stage
```

This "feed-forward initialization + explicit constraint correction" explains how the optimization approach evolved. The network lowers the threshold for cold start and low-texture matching, while geometric optimization re-imposes the real observations of the current scene onto the result. The same framing also preserves honest boundaries. The training prior may fill a common wall over a real doorway. Confidence may be uncalibrated. The scale of long sequences will drift, and dynamic objects may produce mutually contradictory pointmaps. If these points are sent directly into Poisson, model hallucination will become a watertight fake wall. If they directly generate colliders, visual bias will escalate into gameplay errors.

The Chapter 6 output contract and the connection to the representation layer The formal deliverable of Chapter 6 should be a replayable reconstruction package, not a single model file. Such a package covers camera coordinate conventions and units, intrinsics, distortion, extrinsics, timestamps, and quality. It also carries raw image/depth references and hashes, sparse tracks, and an observation graph. Per-pixel depth, pointmaps, normals, confidence, and dynamic/completion masks follow. The package also holds the TSDF's voxel size, truncation band, weights, and unknown space, plus versions for global alignment, loop closure, BA, and re-integration. Its export is the point cloud/initial mesh with the provenance mapping.

What this layer adds is "queryable distance and multi-view geometry." It still has no stable appearance function, and it does not necessarily have a closed, oriented, editable surface. NeRF, neural SDF, and 3DGS arrive in the next chapter to enhance novel-view and surface estimation with continuous fields or differentiable primitives. They must inherit the camera, scale, confidence, and observation coverage. At the interface they must not zero out the uncertainty of Chapter 6.

What point clouds lack is not a triangle file but neighborhood, orientation, and free-space semantics A point cloud is usually written as $\mathcal{P} = (p_i, c_i, w_i, \ldots)$, where $p_i \in \mathbb{R}^3$, and it may carry color, confidence, time, and provenance. It does not capture "which points are adjacent," "which side is the interior," or

"whether there really is a surface between the points." Laser scanning, MVS, DUSt3R pointmaps, and 3DGS surface sampling have different noise distributions. Depth sensing is often biased along the line of sight. MVS produces mixed depth at occlusion edges. Feed-forward pointmaps may use a prior to complete low-confidence regions. Gaussian sampling, in turn, is controlled by the level-set threshold. Before surfacing, the provenance and the line of sight must be retained. Otherwise the algorithm can only guess at missing measurements from the spatial distribution of the points.

Neighborhood queries are the foundation of every subsequent algorithm. A static point cloud can use a k-d tree for k-nearest-neighbor or radius queries. When density varies strongly, the physical scale of a fixed k shifts with the region. A fixed radius will fail to obtain enough points in sparse places. Voxel hashing suits large-scale nearest-neighbor search and deduplication, and an octree suits multi-scale solving. Preprocessing should first remove NaN/Inf and points outside the sensor range. It should then filter by confidence, line-of-sight consistency, and statistical outlier score. Voxel downsampling should aggregate color, normal, time, and provenance rather than keep an arbitrary point. For multi-layer thin structures, one should also avoid letting a single voxel average the two layers in front and behind into a nonexistent middle surface.

Point count also cannot directly represent surface quality. A dense region may simply be where the camera stayed longer. A sparse region may be reflective rather than empty of objects. Subsequent methods need a common input contract. At minimum it must store position, optional color, the raw line of sight or camera ID, confidence, sampling time, local density, and the "direct observation/model completion" label.

Normal PCA: the smallest eigenvector gives the axis, the viewpoint or graph propagation gives the orientation Take the neighborhood $\mathcal{N}_i$ of point p_i. Compute the weighted centroid $\bar{p}_i$ and the covariance

$$C_i = \frac{1}{\sum_j w_{ij}} \sum_{j \in \mathcal{N}_i} w_{ij}(p_j - \bar{p}_i)(p_j - \bar{p}_i)^\top.$$

Let the eigenvalues be $0 \le \lambda_0 \le \lambda_1 \le \lambda_2$. The eigenvector v_0 for the smallest eigenvalue is the normal axis of the locally fitted plane, because the points vary least in that direction. The local curvature index is often taken as

$$\kappa_i = \frac{\lambda_0}{\lambda_0 + \lambda_1 + \lambda_2}.$$

When λ_0 and λ_1 are very close, the local region does not look like a stable plane, so the normal confidence should be reduced. A neighborhood that is too small leaves the normal following noise. One that is too large averages sharp corners and the two sides of a thin layer together. Normal stability should be checked at several physical radii. The scale should be chosen according to the sensor resolution, not by point count alone.

PCA determines only the axis. It does not say which of n and $-$n$ is the outer side. Where the camera center o_i is known for each point, one can enforce $n_i^\top(o_i - p_i) > 0$ on visible surfaces, so the normal points toward the camera. If the application's convention is that the outward normal points away from the camera, flip all of them as a whole. Multi-view points should be oriented by the most trustworthy observation or by line-of-sight voting, not by an arbitrary choice of first frame. When no viewpoint is available, one can build an adjacency graph and a minimum spanning tree whose edge cost is $1 - |n_i^\top n_j|$. Then flip adjacent normals from the root normal so that the dot product is positive, and use a known exterior point to decide the overall sign.

Normal propagation does not fail from local noise alone. When the two sides of a thin shell are mutual nearest neighbors in Euclidean distance, the graph will connect them across the gap. One side is then flipped incorrectly as a whole, and Poisson will propagate that orientation error into a large-scale spurious shell. The two sides of a sharp corner should start with different normals. Forcing global smoothing will round the edges. The normal stage must therefore output the neighborhood scale, the three eigenvalues,

and the orientation confidence. It must also report the source of the orientation, edge markers, and all flip events.

L3: Explicit Surfaces, Production Meshes, and Procedural DCC/CAD

L3 turns spatial samples, implicit fields, 3DGS, or design intent into editable explicit surfaces and production assets. The same geometric quality gate must apply to point-cloud-to-surface conversion, 3DGS surfacing, mesh topology and LOD/UV, and code-driven DCC/CAD. Marble is analyzed here as a multi-artifact product case.

The Marble part must adopt an evidentiary standard different from that of academic papers. As of August 6, 2026, the public World Labs materials are enough to verify product inputs, editing operations, and part of the generation time. They also verify the splat and mesh export specifications and the sample files. They do not publicly disclose the network architecture, training objective, training data, geometric benchmark, or physical benchmark in enough detail to reproduce the model. This chapter therefore discusses only the observable product contract and integration algorithms. It does not speculate about the vendor's internal model.

First Determine the Entity and Version: Marble Is a Product, Not a Public Algorithm Paper In this survey, Marble refers specifically to the world-generation product that the official World Labs Marble documentation describes [@srcWL-MARBLE-OVERVIEW]. It does not refer to the spatial-reasoning dataset of the same name, and it does not refer to a multi-agent software framework. On April 2, 2026, the official documentation listed Marble 1.1 and 1.1 Plus, and it retains 1.0 and 1.0 Draft. For the internal network of these versions, the public pages give no layer count, no parameter count, no loss function, and no training corpus. Therefore one cannot write video diffusion, NeRF, 3DGS optimization, or any known world-model architecture directly as Marble's internal pipeline.

What can be established is the external capability boundary. Vendor documentation states that the product can create explorable worlds from text, a single image, multiple images, 360-degree panoramas, short videos, and coarse three-dimensional structure. It also states that the product can perform panorama editing, expansion, variants, world composition, and camera recording. It states that Gaussian splats, visual meshes, collision meshes, and image/video assets can be downloaded. These are feature claims in vendor documentation. They are not equivalent to independently measured geometric accuracy.

Public Input Contract: The Closer the Information Is to Multi-View Capture, the Stronger the Spatial Constraints Text input carries semantics alone. It provides no true scale, no cameras, and no hidden surfaces. A single image adds one perspective observation. Depth and the regions behind occlusions remain unidentifiable. With multiple images, the user can specify front, back, left, and right, or use an automatic layout. Coverage can increase, but conflicts in focal length, exposure, and content may arise. A 360-degree panorama covers directions most completely, but a single-center panorama still lacks a translation baseline. Short videos provide continuous parallax, and the current official limit is within 100 MB. Chisel coarse three-dimensional structure explicitly provides layout constraints. This ordering is an analysis based on input observability. It is not an inference about Marble's internal weights.

The official pages give a typical product flow. It breaks into the following public I/O steps:

The user submits one or more permitted inputs and selects a product model version. The service first generates a panorama or a draft. At the panorama layer, the user can make local edits. The service then generates an explorable world. The official documentation currently estimates about 20 seconds for a draft and 5 minutes for a complete world. The times vary with the service version and load. The user can expand boundaries, generate variants, or compose multiple worlds in Studio. For export, the user chooses splats, visual meshes, collision meshes, a panorama, or recorded video.

No step here can prove that a particular neural network is used internally. From the exported primitives, all we can judge is that the final high-fidelity visual representation contains Gaussian splats. From the GLB

we can judge that the service provides triangle mesh assets. We cannot infer backward which diffusion, reconstruction, or meshing algorithm generated these assets.

Public Output Contract: Splats, Visual Meshes, and Collision Meshes Must Be Used Separately The official export specifications [@srcWL-MARBLE-EXPORT-M04] list four main asset classes. The panorama is a 2560×1280 equirectangular PNG. The Gaussian representation can be exported as an SPZ of about 2 million splats or of about 500,000 splats. A PLY export is also available at the same scale. SPZ is optimized for Marble rendering and file size, while PLY targets broader software compatibility. The Collider Mesh is a coarse GLB mesh with about 100,000 to 200,000 triangles. It is officially defined as intended for simple physics computation. High-quality visual meshes include a GLB with about 600,000 triangles and textures. Another GLB carries about 1 million triangles and vertex colors. According to the Mesh export documentation [@srcWL-MARBLE-MESH], generation takes up to about one hour, and typical files are about 100 to 200 MB.

The query contracts of these three representations differ. Splats answer camera queries for high-fidelity color and opacity, and they usually have no triangle topology. DCC and rasterization pipelines can edit, simplify, and UV-unwrap high-quality meshes. Such meshes are not guaranteed to be watertight, manifold, or physically correct. A collider mesh sacrifices visual detail for physics queries and cannot be used for high-quality rendering. The vendor also explicitly requires that visual rendering prefer splats or high-quality meshes, not the collider.

Coordinate conventions depend on the asset version. The export specifications page still carries the statement “default OpenCV coordinates”. The changelog [@webd179b99c4e6c] records a different rule. From December 11, 2025, the splats and meshes produced by world generation default to OpenGL. From January 1, 2026, users may choose OpenGL or OpenCV, and approximate scale and grounding handling were added. The two documents disagree in time, so integrators cannot apply any single fixed conversion unconditionally. Read the export task options together with the asset version. Confirm first that the source asset uses the OpenCV convention. Only then negate the Y and Z axes as documented, which converts that asset to OpenGL.

The Executable Marble Integration Algorithm Is Not Marble's Internal Generation Algorithm The pseudocode below shows how to integrate the public exports into an engine safely. It describes a client-side asset pipeline proposed by this survey. It is not Marble's internal model algorithm.

The text flow or pseudocode in the original manuscript

```
Input: Marble task ID, export manifest, target engine coordinate system and units
1. Save the model version, task time, input types, and export options.
2. Download SPZ/PLY, visual GLB, collider GLB, and textures; compute the byte sizes and SHA-256.
3. Parse each file and reject out-of-range indices, abnormal node hierarchies, missing buffers, and non-
4. Confirm OpenGL/OpenCV from the task record; perform the axis conversion only after confirmation.
5. Verify the "approximate scaling" with known scale anchors; the approximate scale must not be treated
6. Attach the splat to the visual rendering layer; attach the HQ GLB to the editable visual asset layer.
7. Copy the collider GLB to a separate physics layer, generate a BVH, and check for degenerate triangles
8. Rebuild the NavMesh from the validated collision layer according to the specific agent radius, slope,
9. Run penetration, falling, door-width, slope, revisit, and coordinate-orientation tests.
10. If any item fails, isolate the asset, keep the original files and logs, and do not automatically rep
Output: layered visual/physics assets, a provenance manifest, a validation report, and a rollback-capabl
```

The algorithm insists above all on separating the visual and physical layers. A visual mesh of about 600,000 faces, set directly as a dynamic triangle mesh collider, would raise broad-phase and narrow-phase costs. Holes and floating surfaces may also produce abnormal contacts. Rendering with the coarse collider of 100,000 to 200,000 faces, conversely, would lose materials and detail. A NavMesh cannot be generated directly from splats either. Walkability requires a continuous surface, slope, clearance and agent size.

Official Samples Can Prove the Format Is Usable, Not That the Geometry and Physics Are Correct This project previously ran a read-only format spot check on the `rustic-kitchen` sample that the official

export specifications page provides. The sample collider GLB parses as GLB v2 and contains 114,183 triangles, with no materials. The high-quality GLB contains 595,429 triangles plus materials and textures. The SPZ of about 500,000 splats is identifiable as a compressed SPZ file. The project keeps the full byte counts and digests in its source notes (open materials: `../sources/world_video_marble_notes.md`). These counts are of the same order of magnitude as the official specifications. They can prove that the file contract of this sample is parseable.

This spot check has no collision ground truth, scale benchmark, surface scan, navigation task or independent reference mesh. It therefore cannot prove that the collider error is small, that the visual mesh is watertight, or that the splats have physical meaning. It did not call a paid API or obtain model weights either. That also rules out calling it a reproduction of the Marble algorithm. The conclusion it supports is narrow: "the official sample matches the public format and the approximate scale statement at the file level."

How Disclosed Failure Modes Propagate to the Engine Layer The official Mesh export documentation explicitly lists the structures that are more prone to unevenness, holes, clumps and floating surfaces. Those are thin or complex structures, transparent or reflective surfaces, sky and background, and regions not adequately covered by the input. Their propagation paths are concrete. A missing thin railing makes holes appear in the visual mesh. If the collider is missing in the same place, a character may pass through. If the collider instead seals the hole, a passage visible in the image becomes impassable. A reflective floor reconstructed as an undulating surface causes jitter in falling bodies or vehicle contacts. Floating sky surfaces that enter the collision layer create invisible obstacles in the air. Clumps in uncovered regions generate walkable polygons when used for a NavMesh. Those polygons lead to platforms that do not exist.

The January 2026 changelog only says that assets are "roughly scaled and grounded," that is, approximately scaled and grounded. It cannot replace metric calibration. Robotics or digital-twin applications must recalibrate using known lengths, gravity direction and the ground plane. Triangles in a GLB do not mean that it contains mass, friction, restitution, joints and object semantics. Those properties still require external measurement, rules or manual configuration.

Defenses should be enforced in place at each interface. At the input end, check video continuity, camera shake, exposure jumps and dynamic objects. At the export end, validate the file structure, coordinates, non-finite values and resource limits. At the geometric end, check manifoldness, self-intersections, connected components, normals and scale. At the physics end, use an independent collision proxy for falling, penetration and continuous collision tests. At the navigation end, rebuild and replay paths for the agent radius and door width. These different units cannot be merged into a single "overall quality score."

Marble's Position in the Method Lineage and Its Interface to the Next Layer Judged by the public contract, Marble productizes several steps that were originally scattered. They are multimodal input, world generation and editing, Gaussian visual representation, and visual mesh and collision mesh export. It comes closer to production assets than models that output only video. It also supplies one more coarse physics proxy than reconstructors that output only splats. The public materials cannot yet prove, however, that it has queryable object state, deterministic dynamics, NavMesh, scripting logic or long-term interactive state. An officially exported collision mesh also does not mean that physics acceptance in the target engine has been completed.

Marble is therefore reasonably classified as "a product system that provides multimodal world creation and multi-layer asset export." It is not a public algorithm that can be reproduced from paper formulas. When it delivers to the next layer, splats, visual meshes and colliders should be treated as three independent versioned assets. That handover should also complete coordinate and scale confirmation, geometry repair, collision validation, NavMesh generation, object semantics, physics parameters and runtime safety gates. Its internal architecture should remain unknown until it is made public. It should not be filled in on its behalf from adjacent papers.

The four chapters that follow are not a set of unrelated papers. They describe one continuous spatial-construction chain. It starts from 2D observations with occlusion, noise and appearance ambiguity. From

those observations it first recovers cameras and measurable spatial samples. It then organizes the samples into queryable appearance or continuous surfaces. Finally it converts them into production meshes with topology, UV and multiple levels of detail. Two changes happen at the same time as the methods evolve. The representation expands from ”sparse points and discrete voxels” to ”continuous implicit fields, differentiable explicit primitives, and oriented meshes.” The solving moves from ”repeated optimization for each scene” toward ”learning a prior that gives an initialization in a single feed-forward pass, then locally correcting it with reprojection, closed loops, and surface constraints.” Later methods reduce some bottleneck of the previous generation. They do not automatically supply the asset contract that the previous generation lacked.

Throughout this survey, four kinds of output need to be distinguished. The first is cameras and observation maps. They answer which camera a given pixel came from, and which images observed the same point. The second is sampled geometry, including depth, point maps, point clouds and surface samples with confidence. The third is rendering representations, including NeRF and Gaussians. These first answer what is seen from a novel view. Only the fourth is a production surface, with vertex-edge-face adjacency, stable normals, material parameterization, level of detail and explicit defect markers. Many problems trace back to renaming one kind of output directly as the next. The result is an asset that “appears to already have 3D but cannot be used once it enters the engine.”

Gaussian surfacing: adding geometric constraints, not swapping in a different export button Gaussian surfacing methods evolve along three mechanisms. The first is to make 3D Gaussians adhere to a common surface. SuGaR [@srcN006] adds a surface-alignment regularizer to trained Gaussians, which makes them flatter and closer to the level set approximated by Gaussian density. It then samples points and normals from this field and reconstructs a mesh with Poisson. New Gaussians are bound to the triangle faces so that high-quality rendering and editing can continue. Here Poisson is a lossy geometric decision. Mesh quality depends on the level set, the normals and hole filling. Rendering PSNR does not guarantee it automatically.

The second mechanism changes the primitives themselves into local surface elements. 2DGS [@srcN007] replaces full 3D ellipsoids with 2D Gaussian disks. It defines depth more explicitly, through the intersection of rays with the local tangent plane. It also adds depth-distortion and normal-consistency regularizers. This reduces the degrees of freedom that stretch along the viewing direction, and it improves local geometry. The disk elements still share no topology, though. Intersecting disks, gaps, both sides of thin objects and unobserved regions still need a surface-generation algorithm to handle them.

The third mechanism defines or assists a continuous surface field from the Gaussians. Gaussian Opacity Fields [@srcN008] constructs an extractable level set from accumulated opacity. 3DGSR [@srcN009] combines implicit surface reconstruction with the Gaussian representation. PGSR [@srcN010] imposes geometric constraints on approximately planar regions. Together they push information beyond color fitting into the objective, in the form of depth, normal, plane or implicit-field terms. Achieving “looking right” through incorrect thickness then becomes harder. The price is extra weights, scales and surface-extraction thresholds. The pipeline must still pass through isosurface sampling, connectivity checks, hole repair and simplification.

How failures propagate. A tiny error in the input camera can make the same edge form two clusters of Gaussians. The surface regularizer compresses the two clusters into a double-layer mesh. Poisson then closes the gap between the two layers into an inner shell. Relying on the regularizer can flatten a low-texture wall, and a doorway may then be treated as missing data and filled in. The rendering Gaussians and the collision mesh may instead be simplified independently, without shared coordinates, object IDs and versions. The visual wall and the physical wall will then be misaligned. The surfacing output is therefore divided into at least three artifacts. They are raw/compressed rendering Gaussians, a visual mesh with provenance and confidence markers, and collision candidates to be validated separately later. The same coordinates, units, object tiles and content hash link the three.

BPA: building triangles from the local visibility of a rolling ball The original Ball-Pivoting Algorithm paper [@srcP002] assumes a sufficiently smooth source surface, sampled densely enough to be compatible with the ball radius r. Three points can form a candidate triangle if they can jointly support a ball of radius r. The ball must touch all three points, and no other samples may lie inside it. The algorithm first finds a seed triangle

whose orientation is compatible with the input normals, and puts its three edges into the active front. For a directed edge, the ball keeps touching both endpoints of the edge and rotates around that edge. The first new point it touches forms an adjacent triangle with that edge. The front is then updated until no edge can be pivoted. Multi-radius BPA uses small balls to recover detail, then larger balls to connect sparser regions.

Indented workflow or pseudocode in the original manuscript

```
Build a k-d tree and estimate consistent normals
for r in the sequence of radii from small to large:
find seed triangles that are not yet covered and whose supporting ball is empty
add the three directed edges to the active front
while the active front is non-empty:
take one directed edge e
query candidate points in the rolling-ball sweep neighborhood of e
select the point q that is touched first, keeps the ball empty, and is orientation-compatible
if q exists:
output triangle (e.start, e.end, q)
update or cancel the active edge
output boundary loops, unused points, and non-manifold conflicts
```

The data structures are a spatial index, an active half-edge front, the set of generated triangles, and point/edge usage state. The precondition for correctness is not that “any point cloud can roll out a surface.” It is sampling dense enough, noise small relative to r, consistent normals, and a local surface that can accommodate the ball. If r is smaller than the point spacing, the surface fragments. If r is larger than a real hole, the ball will cross the hole. If it is larger than the spacing of a thin layer, the ball will bridge the two layers. A noise point can become the pivoting contact point first. Wrongly oriented normals will reject correct triangles or produce flipped faces.

BPA does not solve a global equation. It preserves local holes and boundaries, and it does not naturally close unknown regions. This is both an advantage and a limitation. The output contract must carry the radius sequence and the boundary loops. It must also carry the proportion of uncovered points, the triangle aspect ratios, and the supporting points of each face. An open boundary may be a capture gap or a real doorway. It cannot be uniformly recorded as an error to be repaired.

Alpha shape: using the scale parameter of the Delaunay complex to decide which empty balls to keep Alpha shape begins with the 3D Delaunay tetrahedralization of a point set. Every Delaunay simplex has a circumsphere that contains no other sample points. Fix a scale parameter α, and keep those vertices, edges, triangles and tetrahedra whose circumsphere radius satisfies the threshold. The boundary of the kept complex is the alpha shape. Implementations parameterize this differently, using either the radius $r \leq \alpha$ or the squared radius $r^2 \leq \alpha$, so read the library convention. The CGAL 3D Alpha Shapes documentation [@srcP007] draws a further distinction between general and regularized results. Regularized results can remove isolated lower-dimensional simplices that belong to no 3D cell.

Alpha shape shares a ”ball scale” with BPA, but the two methods reason differently. BPA rolls the ball locally along an existing front. Alpha shape builds the global Delaunay complex first, then filters it by the empty-ball scale. A small α preserves detail but yields many fragments. A large α fills in grooves and eventually approaches the convex hull. Delaunay requires exact predicates or controlled symbolic perturbation on cospherical and nearly coplanar points. Direct tetrahedralization of a million points also usually requires far more memory than local BPA. Alpha shape suits one question particularly well: is a topology stable with respect to scale? A doorway that shows up in only a very narrow parameter range is not a stable asset feature. The original sightlines and images should then be consulted for verification.

A single ”best alpha” mesh should not be the only output of this method. It should also save how connected components, cavities, boundaries and volume change across the parameter sweep. Choosing alpha by tuning until it ”looks best” puts the selection process into the asset provenance as well.

Poisson: treating oriented points as the gradient of an indicator function to solve for a global implicit surface The original Poisson Surface Reconstruction paper [@srcP003] starts from the ideal solid indicator function $\chi(x)$, which jumps from inside to outside at the surface. Its gradient in the distributional sense is related to the surface normal field. Splat and smooth the normal of every oriented point into a spatial vector field $V(x)$, then solve

$$\min_{\chi} \int_{\Omega} \|\nabla\chi(x) - V(x)\|^2 dx.$$

The Euler--Lagrange equation of that objective gives the Poisson equation

$$\Delta\chi = \nabla \cdot V.$$

The actual algorithm represents $\chi = \sum_k x_k B_k$ with local basis functions on an adaptive octree. It assembles a sparse linear system $Ax =$b$, solves for the coefficients with a multiscale solver or an iterative method, then extracts the surface at the isovalue $\chi = \tau$. The octree depth controls the smallest cell and the memory. The number of samples per node affects adaptive refinement. Sample density estimation can help trim extrapolated surfaces supported by very few points. The original Screened Poisson paper [@srcP004] adds a point-value constraint beyond gradient fitting, which it abstracts as

$$\min_{\chi} \int \|\nabla\chi - V\|^2 dx + \lambda \sum_i (\chi(p_i) - \chi_0)^2,$$

This term brings the isosurface closer to the samples and reduces the over-smoothing of the original Poisson. A λ that is too large will also follow the noise. The continuous objective above explains the mechanism. The specific octree basis functions, the discretization of the screening term and the solver should come from the paper and the implementation version.

Indented workflow or pseudocode in the original manuscript

```
Input oriented points, confidence weights, and the reconstruction bounding box
→ build an adaptive octree and splat the normals into a vector field V
→ assemble the Laplacian sparse matrix A and the divergence right-hand side b
→ if Screened Poisson is enabled, add the point-value term
→ iteratively solve Ax=b and record the residual
→ determine the iso-value tau from function statistics at the samples
→ extract the iso-surface on the octree and clip by support density
→ output the mesh, per-vertex density, boundaries, connected components, and parameters
```

Poisson is correct only when the input has fairly consistent oriented normals and the user accepts the global smoothing and closure prior. It can bridge small gaps and suppress noise, and it tends to produce watertight surfaces. Without the original sightlines, however, there is no way to know whether a gap is a scanning hole or a door or window. A wrong normal does not merely flip one local triangle. It changes $\nabla\cdot V$, and the error propagates through the global equation into a large shell or a wrong inside/outside. The bounding box, octree depth and isovalue threshold also change volume and thin structures. Watertightness is a property of the algorithm, not evidence of scene authenticity. Surfaces filled in from low sample density must be marked separately. Their provenance must be retained as well, so that the surfaces can be verified against the original rays.

TSDF Marching Cubes: retaining free/unknown semantics before extracting the isosurface Suppose the point cloud comes from a sensor sequence that still retains cameras and depth. Reconstructing a TSDF from the observation SDF and weight formulas in Chapter 6 is then usually more auditable than discarding the sightlines and running Poisson. A TSDF distinguishes voxels in front of the surface that were

observed as free from voxels that were never seen. The fusion weights can also enter the surface-extraction confidence.

The original Marching Cubes paper [@srcM001] reads the signs of the eight corners of each regular voxel cell against the threshold τ. Those signs give an 8-bit index, which selects one of 256 configurations. When the scalar values at the endpoints x_a, x_b of an edge are f_a, f_b, the intersection point is the linear interpolation

$$x = x_a + \frac{\tau - f_a}{f_b - f_a}(x_b - x_a).$$

An edge cache must deduplicate the vertices that edges of adjacent cells share. Without it, the result is a triangle soup that is geometrically coincident but topologically disconnected. On saddle or face-ambiguous cells, classic table lookup may choose different connections. Consistent disambiguation rules can reduce the cracks that result. When resolution is insufficient, zero crossings on both sides fall in the same cell and thin walls disappear. Truncation and weights may in turn form unreliable isosurfaces in unobserved regions. Surface extraction should skip cells with insufficient weight or with unknown corners. It should also allow open boundaries in the output rather than filling the entire volume by default.

Marching Cubes data structures include regular volumes or sparse voxel blocks, a per-cell case index and a cross-cell edge vertex cache. The same structures also carry vertex position/normal arrays and triangle indices. Normals can come from the central-difference gradient of the TSDF. At block boundaries, however, neighboring blocks must be accessed or a consistent halo used; otherwise visual seams and incorrect face orientations appear.

Method Selection, Failure Propagation, and the Output Contract No single method of surfacing point clouds is optimal for all inputs. If genuine open boundaries are to be preserved and sampling is approximately uniform, multi-radius BPA can be tried first. If concave–convex topology is to be explored across scales, alpha shapes can be used. If consistently oriented points are available and globally smooth completion of unmeasured regions is acceptable, Poisson/Screened Poisson can be used. If depth, pose and line of sight are still available, TSDF plus Marching Cubes can preserve free–unknown semantics. Results from multiple methods can be inspected side by side. Majority voting still cannot replace observation, because the methods share the same input errors while their priors differ.

The error chain here is clear. Depth/pose noise changes point positions. Neighborhood scale interprets that noise as curvature, or blends thin layers together. Wrong normals make BPA reject correct connections and make Poisson flip shells globally. An oversized ball or alpha spans genuine holes. An overly deep octree chases noise, while an overly shallow octree loses detail. TSDF resolution and the truncation band erase thin objects. Marching Cubes ambiguity or edge-cache errors create cracks. A log that claims “automatic surfacing succeeded” but checks only that the file is non-empty has covered none of these mechanisms.

The surfacing stage should deliver the reference point cloud and its attributes, the spatial index configuration, and normal scale and orientation records. It should also report the algorithm used and all of its scale parameters, plus per-face support/observation density. A completion face mask, boundary loops, connected components and non-manifold edges belong in the delivery as well. So do an initial self-intersection screening and the bidirectional distance to the original point cloud. Surfaces extracted from a TSDF must also retain the voxel weights and unknown space. Surfaces extracted from Poisson must retain the octree depth, iso-value threshold and density clipping.

The output may by now already be a set of triangles, yet it is not necessarily a production mesh. Duplicate vertices, misoriented faces, T-junctions and self-intersections all remain. So do extremely thin triangles, semantically meaningless hole filling, missing UVs and excessively high face counts. The task of Chapter 9 is not to guess the surface again. It is to organize, repair, simplify and parameterize the candidate surface into a visual asset that can enter DCC tools and engines, without concealing these provenance risks.

The Half-Edge Structure Upgrades “What a Face Looks Like” to “How Faces Connect” Exchange files often represent a mesh with a vertex array $V \in \mathbb{R}^{n\times 3}$ and triangle indices $F \in \mathbb{N}^{m\times 3}$. However identical

their vertex coordinates are, two triangles remain topologically disconnected as long as the indices do not refer to the same vertex. Inappropriate welding does the reverse, gluing together geometrically close but semantically independent sheets. The first step of assetization is to build explicit adjacency from the triangle soup.

The half-edge structure splits each undirected edge into two oppositely directed half-edges. Each half-edge stores its origin, its twin half-edge, the next half-edge in the same face, and the face it belongs to, at a minimum. A vertex stores one outgoing half-edge, and a face stores one edge. Following next walks around a face, and following twin.next walks around a vertex. A boundary half-edge has no twin face, or connects to a dedicated boundary loop. For a closed two-dimensional manifold, each undirected edge must be incident to exactly two faces with opposite directions, and the vertex neighborhood must be homeomorphic to a disk. For a manifold with boundary, an edge may be associated with only one face, and the boundary must form a single sector at a vertex. Three faces sharing one edge are non-manifold, and so are two independent face fans meeting at only one vertex.

During construction, place the directed key (a, b) in a hash table. Look up (b, a) to build the twin. When the same undirected key appears more than twice, record the conflict. Do not pair two of them arbitrarily. At the same time, independent attribute indices must be stored. The two sides of a UV seam at the same position usually need different texture coordinates and tangents. Rendering formats may duplicate vertices as well. The topology layer can share geometric vertices. The rendering layer instead expands by the combination of position, normal, UV, material and skin. If the two layers are confused, “welding duplicate vertices” destroys hard edges and UVs.

A consistency audit can draw on the Euler characteristic $\chi = V - E+$F$, on connected components, on the number of boundary loops and on the number of incident faces per edge. A single Euler number cannot point to the location of a defect, however, nor can it determine whether a hole is genuine. The half-edge structure pays off because every later repair and edge collapse can query its topological impact locally.

Repair Order: First Diagnose Defects, Then Decide by Asset Intent Whether to Modify A reliable repair pipeline follows an order that runs from semantics-free steps to semantics-bearing ones.

Parse and numeric gate. Validate index ranges, NaN/Inf in vertices/normals/UVs, element counts, and the file budget. Apply axis, handedness, unit and transform hierarchy, then fix the reference space. Negative scale flips face winding, and non-uniform scale requires transforming normals with the inverse-transpose matrix. When an importer silently drops illegal faces, later reports must still be able to say what was lost. Degenerate and duplicate. Compute the triangle area $A_f = \frac{1}{2}\|(v_1 - v_0) \times (v_2 - v_0)\|$. Delete duplicate indices and faces whose area is below a threshold relative to the bounding box. Merge geometrically duplicate vertices within a controlled tolerance, but lock object boundaries, material edges, UV seams and skeleton seams. Set the tolerance according to units and local scale. The same absolute value cannot apply to a city and to a cup. Connectivity and orientation. Build the half-edge structure and enumerate connected components, boundary loops and non-manifold edges. For each manifold component, propagate consistent winding with BFS. Closed components can use signed volume.

$$V_s = \frac{1}{6} \sum_{(a,b,c)\in F} a \cdot (b \times c)$$

Determine the overall orientation, but for open surfaces the outside cannot be decided by volume alone. The original normals and the camera line of sight should be consulted.

Non-manifold handling. Distinguish duplicate shells, T-junctions, multi-face shared edges and contacts at a single vertex. Splitting vertices by face fan can restore local manifoldness. Another option is to revisit the reconstruction layer and recompute. A split like this makes the data structure legal, but it may create cracks in collision, so it must be recorded as a repair event. Holes and self-intersections. For boundary loops compute size, planarity, observation support and semantic labels. Only automatically fill small scan gaps that satisfy the policy. Doors, windows, pipe openings and garment edges must not be sealed shut in pursuit of watertightness. Small holes can be projected onto a local plane for triangulation, then smoothed and

projected back onto the constraints. Large holes should be handed to provenance-constrained hole filling. Use an AABB tree or BVH to find candidate triangle pairs. Exact triangle intersection then excludes shared adjacency and identifies genuine self-intersections. Recompute attributes. Decide shared or split normals according to hard-edge angle and material/UV seams, and generate tangents. Retain the original scan colors and material provenance. Finally, verify boundaries, non-manifold edges, self-intersections, connected components and bidirectional surface distance once more.

The MeshFix paper [@srcM004] and CGAL Polygon Mesh Repair [@srcM005-CGAL-REPAIR] supply tools for local defects. Those tools cannot know the semantics of a hole, however. Methods such as voxel watertightening can reconstruct a two-dimensional manifold from the boundary between occupied voxels and free space connected to the outside. They then project it back onto the reference surface. Topological legality improves, but a voxel minimum feature scale enters the result. The method may also seal structures that should have remained open. A repair report must not conflate three operations: “deleting illegal degenerate elements,” “restoring obvious breaks,” and “adding surfaces based on priors.” The last must not be disguised as a cleanup operation.

Repair also needs a stopping condition. Consider a component with many interpenetrating thin layers, an unknown inside/outside, and faces that carry no provenance. Repeated local patching will let the file pass manifold checks while it drifts ever further from the real scene. At that point, return to TSDF/point-cloud surfacing or re-acquire data rather than continue beautifying.

QEM Simplification: Why Edge Collapse Is Fast, and Why Semantic Constraints Must Be Locked Surface Simplification Using Quadric Error Metrics [@srcM002] writes each triangle's plane as $p = [a, b, c, d]^\top$, with $a^2 + b^2 + c^2 = 1$, and a point's homogeneous coordinates as $\bar{v} = [x, y, z, 1]^\top$. The squared distance from a point to the plane is

$$(p^\top \bar{v})^2 = \bar{v}^\top (pp^\top)\bar{v}.$$

A face therefore contributes a 4×4 symmetric matrix $K_p = pp^\top$. The quadric error matrix of a vertex v sums over its incident faces, $Q_v = \sum_{p \ni v} K_p$. The cost of collapsing a candidate vertex pair (v_i, v_j) to a new point $\bar v$ is

$$\Delta(v_i, v_j \to \bar{v}) = \bar{v}^\top (Q_i + Q_j)\bar{v}.$$

If the upper-left 3×3 block of $Q = Q_i + Q_j$ is invertible, the optimal position can be solved under the constraint that the homogeneous last component equals 1. If the block is singular, choose the lowest-cost option among finite candidates such as the two endpoints and the midpoint. The original paper also allows vertex pairs not necessarily connected by an existing edge, to promote aggregation. Engineering implementations aimed at topology-preserving assets usually restrict candidates to edges and additionally check local topology. The algorithm places all legal candidates in a min-heap. It pops the lowest cost each time, then checks the entry version and topological legality. The collapse follows, $Q_i + Q_j$ accumulates, and only the local adjacent edges are updated.

Indented Procedure or Pseudocode in the Original Manuscript

```
Compute the plane quadric Kp for each face
Accumulate Qv for each vertex and add boundary/feature constraints
For each edge allowed to collapse, compute the candidate position and cost, and push them into the min-h
while the face count is above the target and the minimum error is below the budget:
pop the lowest-cost edge whose version is still valid
if it violates the link condition or locked attributes, reject the edge
perform the collapse and update half-edges, faces, attributes, and Q
recompute the local candidates and write versioned heap entries
```

QEM's correctness presupposes that the local planar quadric distance can represent the geometric quality to be preserved. It does not automatically protect texture, semantics, or physical function. Without restrictions, boundaries shorten, sharp corners round off, two closely spaced thin layers may become connected, and door gaps narrow. Merging the two sides of a UV seam stretches the texture. Material edges and hard normals are lost, and skinning weight blending makes joints collapse. Common constraints forbid collapses across material, object, and semantic edges. Boundary points may move only along the boundary. Silhouettes and feature edges receive high-weight constraint planes. Others limit normal flips, minimum triangle area, local Hausdorff error, UV distortion, and bone weight change. The link condition is checked before collapsing, to avoid creating non-manifold geometry.

Collision-critical features must also be locked separately. A step of a few millimeters that QEM smooths away may be visually imperceptible, yet it changes whether a character can step over it. A narrow door that becomes wider lets a large agent pass through incorrectly. Visual LODs and collision proxies must not share a single simplification result driven only by triangle count.

LOD and HLOD: Converting Geometric Error into Screen-Space and Gameplay Error A single low-poly model is not a complete LOD. The offline stage generates $M_0, M_1, \dots, M_L$. Each level should record the bidirectional distance to the reference surface M_0. It should also record normal and silhouette differences, UV/material error, topological change, and face count. A world-space error ϵ_w at focal length (in pixel units) f and depth z has an approximate projected error

$$\epsilon_{px} \approx \frac{f\epsilon_w}{z}.$$

This is the basis for selecting LODs by screen-space error, which adapts better to different FOVs and resolutions than a fixed distance threshold. Switching should have a hysteresis band, so the camera does not repeatedly change levels near the threshold. Abrupt geometric changes can use short dithering or cross-fading, but transparency, shadows, and collision cannot necessarily be blended without artifacts. Progressive meshes can store the inverse of each edge collapse, the vertex split, and stream detail continuously. Cluster or meshlet streaming instead cuts the mesh into chunks with bounding volumes and error bounds, selected by visibility and budget.

HLOD packs multiple distant objects into proxy meshes with baked materials. Draw calls drop, but independent object IDs, animation, and occlusion relationships are lost. The mapping from HLOD back to the source objects must be preserved. It must also be made explicit when an interactive object switches back from the proxy to the real entity. Screenshots alone do not constitute acceptance of a visual LOD. Silhouettes, normal maps, shadows, reflections, skinned poses, and switch-frame peaks must be covered too. Collision and navigation should be versioned independently, and every visual asset update should run an alignment regression.

Each LOD level should set two budgets. A visual budget limits projection error and material distortion. A runtime budget limits vertices, triangles, meshlets, VRAM, and draw calls. Meeting only the latter three without the former is ”a faster wrong asset”. Meeting only the former while running out of VRAM at runtime is likewise not a deliverable asset.

UV, Tangents, and Texture Baking: The Last Step That Turns Geometry into a Shadable Asset Reconstructed meshes often carry per-vertex colors. Colors can also be queried from a NeRF/Gaussian. Neither route gives a stable two-dimensional parameterization. UV authoring answers four questions: where to cut seams, how to unwrap each chart, how to pack charts into a texture atlas, and which observations supply the textures on a new surface.

First, the face graph is cut into topological disk charts, based on high curvature, occlusion, material boundaries, and invisible regions. Unwrapping solves for each vertex $u_i \in \mathbb{R}^2$ and aims to reduce angular, area, or length distortion. Methods of the LSCM family pin at least two vertices and minimize the residual of the local conformal equations. No unwrapping can preserve every length and area perfectly at once, so each chart should report its angular/area distortion and flipped triangles. Charts are then scaled by texel density

and packed under a given padding. The padding must cover mipmap and compression filter dilation. Too little causes color bleeding at distance.

Texture baking starts at each surface texel and traces back to a three-dimensional point. That point is then projected onto candidate source images, or it queries the radiance field/Gaussians. Source image selection should trade off visibility, incidence angle, resolution, exposure, and dynamic occlusion. Multi-view colors require exposure compensation and seam blending. Completely unobserved faces must be labeled as inpainting/generation, and must not silently copy their neighborhood. Normal maps require a consistent tangent space, and a vertex's position, normal, UV, tangent, and handedness must match the target engine's conventions. Positions may be identical at a UV seam, but rendering vertices must be split.

The asset data structure is therefore more than positions and indices. It should include a topological half-edge mesh; vertex buffers laid out by render attribute; submesh/material slots; UV0 and optional lightmap UVs; tangents and normals; textures and color spaces; skeleton/skinning; object hierarchy and transforms; LOD/HLOD relationships; and provenance and repair records. Exchange formats such as glTF can serialize only the subset of render assets that their specifications support. Half-edge topology, collision semantics, LOD/HLOD relationships, and provenance and repair records usually still require dedicated extensions, `extras`, sidecars, or engine conventions. They must be verified by read-back with the target consumer. Writing custom fields cannot be equated with semantics being preserved.

Procedural Modeling: Constructing Space Directly from Parameters, Constraints, and Code Procedural modeling is not an "automation button" after reconstruction. It is a construction route standing alongside observational reconstruction and generative models. Reconstruction uses images, depth, or point clouds as evidence to solve for unknown cameras and surfaces. Generative models fill in plausible content from a data distribution. Procedural modeling instead treats dimensions, topology rules, assembly relationships, semantic constraints, and random seeds as known intent, and directly solves for a scene that satisfies the contract. This route often suits rooms, roads, pipelines, shelving, level modules, building facades, industrial parts, and synthetic training scenes. Their repeated structures and design rules are more concise than per-vertex coordinates. The cost is that what the code generates is "the world defined by the rules," not an automatic measurement of the real world. Wrong rules can generate wrong assets in bulk, stably and deterministically.

Let the parameter vector be θ, the host and dependency versions be v, the random seed be s, and the procedural builder be G. Then the output asset is not an abstract "model" but should be written as

$$A = G(\theta, s, v) = M_{visual}, M_{collision}, H, U, P, Q,$$

where M_{visual} is the visual mesh and $M_{collision}$ is the independent physics proxy. H is the object hierarchy and stable IDs, U is UV/material, P is provenance, parameters, and the build manifest, and Q is the verification result of geometry and runtime queries. The determinism requirement is not that the asset "looks the same". Once normalized parameters, versions, and seed are fixed, the key output hash or normalized geometry signature must be identical. If the host evaluation, floating-point kernel, or exporter version changes, v must also enter the artifact identity.

Blender Python: The Data API, BMesh, Modifiers, and the Evaluated Scene Are Not One Layer Blender embeds Python and exposes data blocks, scenes, objects, modifiers, nodes, import/export, and rendering interfaces through `bpy`. Scripts have two operational surfaces that are often conflated. `bpy.data` and the object/mesh data API modify data-blocks in Blender's main database, and they are better suited to implementing explicit, headless construction. Whether those edits are written to `.blend` depends on the save action, however, and the application script itself must also guarantee idempotence. `bpy.ops` calls the same operators as the user interface, and whether execution succeeds may depend on the active object, selection, mode, area, and view layer context. A batch script that leans heavily on implicit `bpy.context` may produce different results in an interactive window and in `--background` mode. Engineering defaults should therefore prefer creating and linking explicit data-blocks. Only when an operator must be reused should context be established explicitly and its `poll` conditions checked. Blender Python API [@web11c361c8ab15]

Direct mesh construction writes vertices, edges, and faces into `bpy.types.Mesh`. It can also drive connectivity operations on the editable topology of `bmesh`, such as split, collapse, dissolve, and extrude. The official BMesh documentation states explicitly that this API will not enforce the maintenance of all valid states on the script's behalf. Duplicate edges/faces, faces with fewer than three vertices, and inconsistent selection states are all the script's responsibility to clean up. After an edit-mode BMesh modification, the Mesh should be updated through `bmesh.update_edit_mesh`, and loop triangle tessellation recomputed as needed. A standalone BMesh is first written back with bm.to_mesh(mesh) , and then mesh.update() is called. Blender BMesh [@srcBDCC-003] So "the API call raised no error" is not a sufficient condition for mesh correctness. Every generation function must still check face indices, area, winding order, incidence relationships, boundary loops, transforms, and units.

Modifiers and Geometry Nodes introduce a second key distinction. The base mesh in the object data-block is not necessarily equal to the evaluated mesh of the dependency graph. Arrays, booleans, mirrors, subdivision, curves, Geometry Nodes instances, and the dependency graph change topology after evaluation. A correct contract must make explicit whether what is delivered is the base parametric model or the evaluated geometry. It must read and check the latter from the dependency graph's evaluated object. An exporter should be expected to output the corresponding evaluated result only when Apply Modifiers or an equivalent option is enabled. The target consumer's read-back must still be the standard. Otherwise the script counts 8 base cube vertices, while the exported file may contain tens of thousands of instances or subdivided faces. It may also still be the base mesh, because the option was off. Geometry Nodes is essentially a program graph with typed sockets, fields, attribute domains, instances, and control flow zones. Its node group version, input sockets, attribute names, whether instances are realized, random IDs, and seeds should be fixed, rather than saving only a screenshot of the nodes. Geometry Nodes Manual [@web11b0dbc34b98]

One reliable order for a headless Blender build runs as follows. Clear the scene or open a fixed template. Set units, axes, renderer, and color management. Create collections, objects, and meshes from parameters, then add and configure modifiers and node groups. Evaluate in the dependency graph. Derive visual, collision, and LOD geometry separately. Run geometry checks, export to a temporary path, and read the result back with the target importer. If the read-back passes, move the result atomically into the release directory. The command line should run in background mode, name the script path explicitly, and return a non-zero exit code on Python exceptions. Parse business parameters after `--`. Blender's command-line documentation also reminds us that arguments execute in the order they appear, and that loading a `.blend` overrides scene options set earlier. The command order is therefore part of the reproduction contract itself. Blender Command Line [@webab7044d6a1d4]

Text Workflow or Pseudocode in the Original Manuscript

```
blender --background --factory-startup \
--disable-autoexec --python-exit-code 2 \
trusted_template.blend \
--python build_scene.py -- --config room.json --out build/
```

`--disable-autoexec` must appear before the untrusted `.blend` path, because Blender processes command-line arguments in order. The flag is a necessary auto-execution control. It is not a Python sandbox, and it is not a complete security boundary. In particular, it does not restrict a --python build_scene.py specified explicitly on the command line, so that script itself must already have been reviewed. Blender's official security documentation explains that a `.blend` file can contain registered text blocks and Python driver expressions. The same documentation notes that Python itself does not limit what a script can do. Downloaded source files, add-ons, and build scripts must therefore first enter a worker with no credentials, no network, low privileges, read-only inputs, a bounded output root, and resource quotas. Auto-execution is disabled there by default. Blender Script Security [@srcS020]

CSG and OpenSCAD: Concise Rules Do Not Equal a Robust Boolean Kernel OpenSCAD uses declarative code to describe primitives and set operations, and it is especially suited to parameterizable enclosures, brackets, pipes, room modules, and printed parts. A solid set $S \subset \mathbb{R}^3$ can be represented by a CSG tree composed of unions, intersections, and differences:

$$S = (S_1 \cup S_2) \setminus \bigcup_k H_k,$$

Here S_1, S_2 are the main bodies, and H_k are doorways, holes, or slots. The code stores the tree and the parameters. A triangle mesh such as STL/3MF is merely the evaluation result at a particular resolution and kernel version. OpenSCAD's official command-line interface supports overriding variables with `-D` and exports according to the `-o` suffix, which makes it suitable for batch-building parameter combinations in CI. OpenSCAD Command Line [@srcOSC-CLI] Fast preview is usually an approximate display produced by OpenCSG/OpenGL. It cannot replace the final `render`/export evaluation and checking of the solid. The solid backend also varies with version and configuration, and the 2025 development snapshot has made Manifold the default while retaining the CGAL option. The manifest must therefore pin the OpenSCAD version, the actual backend, discrete parameters such as $fn/$fa/$fs , and the complete command. Neither an artifact-free preview nor a vague "using OpenSCAD" can be treated as reproducible evidence. OpenSCAD Backend Announcement [@srcOSC-MANIFOLD-DEFAULT]

Boolean failures cluster around geometric predicates and boundary representations. Coplanar overlaps, zero-thickness contacts, extremely thin slivers, excessive scale spans, near-duplicate faces, and self-intersecting inputs make inside/outside classification or topology splitting unstable. Expanding each cutting body by an arbitrary epsilon can sometimes avoid coplanarity, but it also changes dimensions and thin walls that closure should have preserved. A more reliable strategy has four parts. Specify minimum features and tolerances relative to the bounding box. Verify before the boolean that the inputs are closed oriented solids. Record component and volume changes at each step. Re-check boundaries, non-manifoldness, self-intersections, and target dimensions after export. A valid CSG tree proves only that the operation can be described. It does not prove that the discrete mesh is suitable for rendering or collision.

CadQuery and FreeCAD: Parametric B-rep, Feature History, and Topological Naming CadQuery models with `Workplane`, sketch, selector, feature, and assembly on the OpenCascade geometry kernel. Typical code first draws a two-dimensional profile on a workplane, then extrudes, revolves, or sweeps it into a B-rep solid. It then selects entities by the direction or geometric type of faces and edges, and performs hole, fillet, chamfer, or shell operations. The official documentation describes `Workplane` as the core object that carries the object stack, the modeling context, and a chained API. CadQuery Concepts [@web02108419a530] A B-rep is not merely a list of triangles. It stores curve and surface geometry — planes, cylinders, and B-splines/NURBS among them — as well as the topological relationships of edges, wireframes, faces, shells, and solids. That makes it suitable for exact dimensions, STEP exchange, and manufacturing constraints. Before entering a game engine it still must be tessellated and assetified according to OpenCascade's linear deflection, angular deflection, normals, and material grouping. OpenCascade Meshing Pipeline [@srcOCCT-MESH] Assembly constraint solving is essentially adjusting child object positions to minimize constraint cost. Under-constrained or multi-solution systems are also affected by initial positions, so "the solver returned success" alone cannot prove that the assembly is unique. CadQuery Assemblies [@srcCQ-ASSEMBLY]

FreeCAD's Python interface allows creating documents, Part/PartDesign objects, expressions, and parameter tables, and then evaluating the document via `recompute`. It is suited to turning interactive CAD workflows into auditable scripts, and it also carries the different object models of workbenches such as assembly and BIM. The most dangerous engineering misunderstanding here is to treat "parameters can be changed" as "references are always stable". Suppose a later feature references an earlier result through an anonymous Nth face or the topmost edge. After a boolean or fillet changes the topology, the selection may bind to a different face. This is the well-known topological naming problem. FreeCAD 1.0 has merged a topological naming mitigation mechanism based on modification/generation/deletion history mapping, and stability has improved somewhat. Its official positioning nevertheless remains a mitigation, not an absolute guarantee against arbitrary boolean and feature changes. A displayed Label is likewise not equal to a stable sub-shape ID. FreeCAD 1.0 release notes [@srcFC-TNP-1] Defense should preferentially use datum planes, datums, sketch constraints, object-level links, and semantic references. After a parameter sweep, re-check the count, area, normal, centroid, solid count, volume, key dimensions, and export read-back of the selected sub-shapes.

The truthfulness of parametric CAD comes from the design contract, not from photographic evidence. A digital twin of a real building or piece of equipment can use scans and point clouds as measurement constraints and CAD parameters as intent and an editable prior. Even so, post-fit residuals, unobserved surfaces, and human decisions must be retained. Treating CAD surfaces unconditionally as ground truth hides wrong drawing revisions or on-site modifications inside a ”precise model”.

Houdini and Geometry Nodes: Turning the Generation Process into a Dataflow Graph Houdini SOPs and Blender Geometry Nodes both organize geometry operations as directed graphs. Houdini can additionally scale to large-scale dependency scheduling through VEX, Python, HDA, and PDG/TOP. SideFX officially exposes each SOP's result as `hou.Geometry` containing point, vertex, primitive, group, attribute, and intrinsic. PDG/TOP goes further and represents tasks as work items with attributes and dependencies. Houdini Geometry API [@srcHOU-003] PDG/TOP overview [@srcHOU-005] Node graphs suit laying roads along splines, splitting building facades, scattering vegetation, terrain erosion, tiling large worlds, and batch LOD. The reason is that each node explicitly consumes upstream geometry and attributes. Houdini should distinguish point, vertex, primitive, and global/detail owners, where vertex corresponds to the face corner of each primitive. Geometry Nodes, in turn, distinguishes the point, edge, face, corner, and instance domains. Cross-domain reads, interpolation, or promote rules must be recorded separately per host. Otherwise materials, normals, IDs, or collision labels will quietly change.

The reproducible unit of a node graph is not the final `.blend`/`.hip` file. It is ”host version + node/HDA version + input schema + parameters + seed + external file hashes + cook result”. `hython` configures Houdini Python and imports `hou`, but licenses and site environment are also involved. Being syntactically correct does not mean the SOP has cooked. SideFX command-line scripting [@srcHOU-001] Random scattering should derive sub-seeds from a stable `entity_id` or spatial cell ID. Otherwise inserting one object upstream reorders all instances. Instancing can significantly save scene memory, but exporters, collision builders, or downstream engines do not necessarily preserve instance semantics. The contract must separately state when Geometry Nodes executes Realize Instances and when Houdini keeps or unpacks packed primitives. It must also state how each prototype and transform is serialized, and whether colliders are generated per instance, merged, or partitioned.

BlenderProc: Generating Images and Geometry Annotations from the Same Program State BlenderProc turns Blender scene scripts into a purpose-built synthetic data pipeline. Code loads or constructs assets, randomizes materials, lights, objects, and cameras, and performs collision checks or physical placement when necessary. It then outputs RGB, depth/distance, normal, segmentation, flow, and annotations such as HDF5 and COCO/BOP from the same scene state. BlenderProc official repository [@srcBPROC-001] BlenderProc2 JOSS paper [@srcBPROC-002] Its advantage is that images and labels share the same camera, object transforms, and occlusion state. Manual annotation therefore needs no secondary alignment. But ”same-source” only proves that labels are faithful to the synthetic scene. It does not prove that asset semantics, sensor models, or domain randomization are close to the real world.

The input contract must save camera intrinsics K, resolution, camera-to-world extrinsics, OpenCV/OpenGL coordinate conversion, near/far clipping, and object `category_id`/instance/name. It must also save material and pose distributions, renderer, physics step size, output channels, and writer version. `depth` and the `distance` from the optical center to a point are not the same array. The two cannot both be called ”depth”. If a visible background or object lacks a semantic attribute, a default must also be given explicitly, or the build must fail. The per-frame manifest should bind the scene hash, camera matrices, object poses, channel dtype/shape/units, instance mapping, and randomization rejection records to a common `frame_id`.

Fixing `BLENDER_PROC_RANDOM_SEED` unifies Python `random` and NumPy sampling, and it is one of the necessary conditions for reproduction. Yet it does not guarantee bit-level consistency across Blender/Cycles, GPU, physics, or plugin versions. The synthetic data gate should re-run on a fixed golden scene, compare camera projection, RGB/depth/mask/box cross-modal consistency and overall distributions, and then measure sim-to-real on a real validation set. ”generated one hundred thousand labeled images” cannot substitute for coverage, bias, and real task performance.

Open3D, trimesh, PyMeshLab, and CGAL: They Are Better Suited to Geometry Conversion and Acceptance Gates Geometry libraries and DCC/CAD hosts play different roles. Open3D provides point cloud, RGB-D, TSDF, normal, ICP, BPA, alpha shape, Poisson, and TriangleMesh operations, and suits connecting captured data to a surface pipeline. Its Poisson interface explicitly requires oriented normals and returns density to support trimming low-support regions. Open3D surface reconstruction [@srcO3D-SURFACE] `trimesh` focuses on loading, transforms, topology/watertightness checks, cross-sections, adjacency, sampling, collision, and format exchange, but it delegates boolean operations to the Manifold or Blender backend. Its collision and some acceleration capabilities depend on optional packages. PyMeshLab wraps MeshLab filters, and its results are affected by the current `MeshSet` layer, filter order, version, and parameters. CGAL provides polygon mesh algorithms with selectable numeric kernels, explicit topology concepts, and preconditions. Some configurations can use exact predicates, but not all constructions, simplification results, and outputs can be generalized as automatically exact or automatically free of self-intersections. trimesh boolean interface [@srcTM-BOOLEAN] CGAL mesh simplification [@srcCGAL-SIMPLIFY] These libraries can automate repair and conversion, but function names such as `fill_holes`, `make_watertight`, or `repair` cannot supply scene semantics. The manifest must record the actual backend, optional dependencies, and filter chain.

A reliable library-style pipeline should retain the immutable source mesh and then sequentially generate normalization, repair, simplification, visual, and collision derivatives. Each step should record input/output hashes, library and kernel versions, parameters, vertex/face/component counts, AABB, and area/volume. It should also record boundary/non-manifold/self-intersection, the mapping of removed or added elements, and bidirectional distance to the reference surface. Different libraries may handle the scene hierarchy, units, textures, and negative scale of the same OBJ/glTF differently. At least a second parser or the target engine must therefore read it back, and geometry queries must be compared, rather than merely comparing "export succeeded".

The Minimal Contract for Code-Generated Visual Surfaces, Collision Surfaces, and Scene Graphs The main function of a procedural build should be a pure parameter interface, with host operations converging into a boundary adapter. The pseudocode below deliberately separates visual from collision and reads back again before publishing:

Text workflow or pseudocode in the original manuscript

```
build(config, seed, toolchain_lock):
assert validate_schema_units_ranges(config)
intent = canonicalize(config, seed, toolchain_lock)
visual = build_visual_geometry(intent)
collision = build_collision_from_intent(intent) # do not blindly copy from the high-poly model
scene = attach_ids_materials_transforms(visual, collision)
assert geometry_gates(scene)
temp = export_to_isolated_directory(scene)
roundtrip = import_with_target_or_second_parser(temp)
assert query_equivalence(scene, roundtrip)
manifest = hash_inputs_outputs_and_metrics(intent, temp, roundtrip)
atomic_publish(temp, manifest)
```

`build_collision_from_intent` matters. The collision clearance of a door opening is best reconstructed from the same door width/height parameters, rather than by performing unconstrained simplification on the high-poly visual model. This way visual trim strips can exist while the collision proxy still maintains a clear passage contract. Object IDs should also be derived from semantic paths and stable parameters, not by relying on "which object in the current array". Otherwise a single insertion misaligns saved files, network replication, and differential updates.

This survey provides a minimal runnable mechanism that depends on no third-party libraries. `reproduction/programmatic_m` takes metric room and door-opening parameters and generates a visual OBJ, a collision OBJ, and a machine-readable manifest. The three files from two isolated builds are byte-for-byte identical. The visual mesh has 120 triangles and the collision mesh 84 triangles. The door-opening clearance, finite-value, index,

and degenerate-face assertions pass. This result only demonstrates the parameters–dual-mesh–manifest mechanism. It is not a reproduction on a Blender, CAD, Houdini, or engine host, and all examples whose host is not installed are marked in the status file as `SYNTAX_ONLY_NOT_HOST_EXECUTED`. Reproduction instructions (open material: `../reproduction/programmatic_modeling/README.md`) Status and boundaries (open material: `../reproduction/programmatic_modeling/STATUS.md`)

Choice Boundaries for Procedural Modeling Procedural DCC/CAD works best on targets that have explicit dimensions, repeated modules, assembly relationships or manufacturable constraints, or that need tens of thousands of controllable variants. Such targets need only a few inputs. Edits stay traceable. Semantic IDs can be generated from an explicit schema. Visual, collision and navigation parameters can be coupled. When the target is an unmodeled real site, reconstruction still carries the measurement work. When unobserved regions need visual richness, generative models can propose candidates. When runtime matters, the engine carries state and performance. Real systems commonly mix these routes. Scans calibrate dimensions, CAD solidifies structure, generative models supplement materials and content, and DCC/node graphs turn the result into assets. The engine then builds collision and navigation.

This hybrid route has a fixed acceptance order. First ask whether the rules satisfy the design constraints. Then ask whether the output fits the observations. Then ask separately whether the mesh, the collision, the navigation and the engine queries hold. Rules, images and runtime each carry independent evidence. No layer can self-certify the next one.

The Full Flow from Candidate Surface to Production Asset and Failure Propagation Indented flow or pseudocode in the original manuscript

```
Load the candidate mesh, the provenance mapping, and the coordinate contract
→ validate the parse budget, indices, finite numbers, units, handedness, and hierarchical transforms
→ remove exact duplicates and degeneracies by a scale threshold, preserving attribute seams
→ build half-edges and report connected components, boundaries, and non-manifold face fans
→ orient each component using adjacency and source views
→ repair only the holes and self-intersections allowed by policy, and tag newly added faces with provena
→ preserve hard edges and seams, and recompute normals and tangents
→ unwrap charts, pack UVs, bake by visibility, and record source image IDs
→ generate constrained QEM LODs and HLOD provenance mappings
→ run geometry, topology, rendering, attribute, and import round-trip tests
→ export the visual asset and a separate downstream collision candidate
```

In this chain, errors amplify rather than vanish on their own. A pose deviation in Chapter 6 makes MVS/TSDF produce double-layer surfaces. The color fitting or densification in Chapter 7 reads that double layer as a stable primitive. The Poisson step in Chapter 8 seals it into a shell. The welding and QEM steps in Chapter 9 may then join the inner and outer layers into self-intersections, or block the openings. Once UV baking covers geometric seams with blended colors, visual inspection becomes even harder. Conversely, chasing topological perfection too hard can also repair genuinely open structures into watertight fake objects. Every conversion should record its provenance mapping and its difference metrics. A problem can then be traced back to the stage that produced it.

Geometry gates and asset gates must be set separately. A geometry gate checks bidirectional surface distance, normals, boundaries, topology, observation coverage and completion surfaces. An asset gate checks units, coordinates, UVs, materials, LOD, import read-back and runtime budget. PSNR, Chamfer, non-manifold edge count and triangle count carry different units, so they cannot be summed into a single total score. A mesh can sit very close in geometric distance and still be unusable because its UVs are flipped. It can also be watertight and low-poly while sealing every door opening.

Chapter 9 Output Contract and the Connection to the Collision, Navigation, and Engine Layers A production visual mesh should deliver at least the following. Coordinate axes, handedness, units and origin must be explicit. The object hierarchy carries stable IDs, and the half-edge topology audit reports its results.

The asset must report statistics on open boundaries, non-manifold edges, self-intersections, degeneracies and connected components. It must log all automatic and manual repair events and the added-face masks. The reference surface and each LOD must give their error, triangle count and switching strategy. UV charts, texel density, padding, materials, tangents and texture color space must be recorded, along with source image, point cloud and field versions and their content hashes. Import read-back results must cover the target format.

Even then the result is only an "editable, renderable surface." The next stage must build a separate set of collision proxies from the visual mesh. It chooses primitives, convex hulls or convex decomposition, SDFs, or triangle meshes. The right form depends on whether the objects are dynamic or static, and on their convexity, velocity and error budget. It then bakes a NavMesh from agent radius, height, slope and step parameters. A common object ID and transform must link the visual LOD, the collision LOD and the navigation surfaces. They must not impersonate one another.

Only at this point does the logic of the method evolution become complete. SfM turns images into a camera-point observation graph. MVS and TSDF add depth, free space and fusible geometry, and feed-forward point maps lower the initialization barrier. NeRF provides continuous appearance queries, a neural SDF writes the zero level set into the representation, and 3DGS trades differentiable explicit primitives for real-time rendering. BPA, alpha shape, Poisson and Marching Cubes make different connectivity and closure assumptions about the samples. Half-edge, repair, QEM, LOD and UV then turn candidate surfaces into editable visual assets. Physical interactivity still requires a next, separate layer that builds collision, navigation, scene graphs and the engine runtime. That layer must be demonstrated through contact, reachability and performance regressions. It cannot be inferred from "the mesh has been exported."

L4: Collision Surfaces, Distance Proxies, and Navigation Surfaces

L4 delivers collision, distance or navigation queries natively, for a given proxy and runtime budget. A file named collider, a single bounding-box pass or a successful NavMesh bake are all only candidates. Each one still needs penetration, tunneling, stability, cost and reachability tests.

The meshes, Gaussians or implicit fields obtained earlier solve appearance replay and geometric representation. A collision system, by contrast, must answer a different set of questions. It reports whether two objects intersect, when they first come into contact, how deep the penetration is and what the contact normal is. It must also report whether that query completes stably within every physics step. Navigation asks more again. An agent has a body size, a slope capability and a step-over capability. Navigation must report which free spaces are connected for that agent and what the path cost is. It must also report which paths remain valid after dynamic obstacles change. The visual mesh, the collision proxy and the NavMesh must therefore be three artifacts in the same world coordinates, linked by version relationships. They are not three aliases of a single file.

Visual Meshes and Collision Proxies Optimize Different Errors Visual meshes usually target pixel error, silhouette, material continuity, normal smoothness and triangle budget. A window frame can carry a fine groove in a normal map. A leaf can be a zero-thickness face with a transparent texture. A distant wall can rely on occlusion to cull its back faces. Those tricks hold as long as rendering is correct. A collision proxy, by contrast, must define a spatial set. Let the reference entity occupy S and the proxy occupy C. The two most basic geometric errors are then

$$E_{\text{false}} = \mu(C \setminus S), \qquad E_{\text{miss}} = \mu(S \setminus C),$$

where E_{false} denotes obstacles that do not exist, such as a convex hull sealing off a cup handle, a doorway or a drawer handle. E_{miss} denotes real obstacles that are missed, such as a thin wall swallowed by voxelization. The first produces invisible walls and false unreachability. The second produces wall penetration and falls. Practical engineering should also weight the errors under a task-relevant sampling distribution $q(x)$.

Doorways, step edges, grasp slots and high-speed motion trajectories matter more than surfaces far from the interaction region.

$$L_{\text{proxy}} = \lambda_f \int_{C \setminus S} q(x)dx + \lambda_m \int_{S \setminus C} q(x)dx + \lambda_n E_{\text{normal}} + \lambda_c C_{\text{query}}.$$

Here E_{normal} measures the contact normal error, and C_{query} measures the number of candidate pairs, the narrow-phase time and the memory. Safety robots usually make λ_m larger, preferring to treat uncertain regions conservatively as obstacles. Competitive games may care more about λ_f, so that players are not blocked by invisible faces. Precision grasping must protect grooves and holes, and cannot optimize only the overall volume error. This difference in objectives explains why a visual LOD cannot automatically serve as a collision LOD. It also explains why "watertight" is not a sufficient condition. A proxy that is completely watertight but seals off real openings is topologically legal and functionally wrong.

The input contract for a collision asset should include at least the following. It names a normalized reference mesh and says whether the object is static, kinematic or dynamic. Mass and material belong here, together with the allowed maximum miss and false-hit distances and the query types (ray, overlap, sweep, simulation). The contract also states the speed limit, the minimum feature scale and whether inner-side contact is allowed. Observed-face and completion-face masks complete the list. The output contract is not a bare mesh. It is

 shape set + local transform + collision layer/mask + material + cooking parameters + error report + version hash .

If the upstream provides only NeRF or 3DGS, an acceptable reference surface or signed occupancy must be obtained first. If the upstream mesh contains self-intersections, non-manifold edges, open boundaries or an unknown scale, then "automatic convexification" will only freeze the uncertainty into the physics layer. The topology, scale, normal and observation coverage checks in Chapter 9 are therefore a prerequisite gate for this chapter. They are not optional cleanup.

AABB and BVH: Broad Phase Produces Only Candidates, Not Contacts A scene with n shapes needs $O(n^2)$ candidates if every pair is checked. Broad phase first uses cheap bounding volumes to exclude pairs that cannot possibly intersect. For a vertex set P_i, the axis-aligned bounding box is written as

$$B_i = [\ell_i, u_i], \quad \ell_{ik} = \min_{p \in P_i} p_k, \quad u_{ik} = \max_{p \in P_i} p_k.$$

Two AABBs overlap in three dimensions if and only if, for every axis $k \in x, y, z$,

$$\ell_{ik} \le u_{jk} \ \wedge \ \ell_{jk} \le u_{ik}.$$

Those six interval comparisons are very fast. They only indicate a "possible collision." An AABB overlap between a curved wrench and a sphere does not mean their real surfaces touch. The official PhysX documentation explicitly defines AABB overlap as broad phase, and computes real contacts only in the next stage. Its SAP, MBP, ABP, PABP and GPU broad phase suit different static/moving ratios, region partitions and parallel scales.[@srcC004]

Static triangle environments usually add a bounding volume hierarchy, BVH. Leaf nodes store triangles or small clusters. Internal nodes store the union of the left and right child bounding boxes. During construction, one may split at the median of the longest axis, or use the surface area heuristic:

$$C_{\text{split}} = C_t + \frac{A(B_L)}{A(B_P)} N_L C_i + \frac{A(B_R)}{A(B_P)} N_R C_i,$$

where $A(B)$ is the box surface area, N_L, N_R count the primitives of the subtrees, and C_t, C_i are the traversal cost and the primitive-test cost. A query starts from the root. If the query bounding volume does not overlap a node box, the node prunes. Only on overlap does the query visit the children, until a leaf node is handed to the narrow phase. Rays, overlaps and sweeps can reuse the same BVH, but their query bounding volumes differ. What is adopted here is the SAH form that MacDonald and Booth developed to approximate intersection probability by surface area. It is a construction heuristic, not a theorem that is optimal for every query distribution. That form comes from the original SAH paper[@srcSAH-1990].

Dynamic objects have three common update strategies. A bottom-up refit follows a rigid transform. A dynamic AABB tree suits frequent insertions and deletions. Asynchronous rebuild follows long-term degradation. To keep tiny motions from moving the tree every frame, an object can use a "fat AABB" expanded by the contact offset and the short-term velocity. High-speed objects use a swept AABB from t to $t+\Delta t$. Too small an expansion misses candidates. Too large an expansion makes narrow-phase candidates explode. For the PhysX MBP broad phase, a large-world partition must use regions that cover all active space. The official documentation explicitly says that only eMBP requires regions. Objects that fall outside all regions will not join the broad phase, and their collisions are disabled. SAP, ABP, PABP and GPU broad phase do not depend on this set of regions. The relevant PhysX type is PxBroadPhaseRegion[@srcPHYSX-MBP-REGION]. Therefore, only when MBP is selected must scene streaming update the visual cells, broad-phase regions and object proxies as a single transaction. This requirement must not be extrapolated into a fixed precondition for all PhysX broad-phase algorithms.[@srcC004]

The broad-phase data structure should record at least node_aabb, child_or_primitive, layer_mask, dynamic_flag, generation . Here generation prevents an old handle from being reused after asynchronous deletion. Layer mask filters out categories that need no contact before candidates are generated. NaN/Inf, reversed-order bounds caused by negative scale, an un-updated transform or an extremely large AABB will contaminate the entire tree. At the mildest, candidates surge. At the worst, objects are missed entirely. Before all shapes enter the broad phase, finite numbers, $\ell\le u$, a scale limit and world bounds checks should be enforced.

Convex Narrow Phase: Convex Hulls, GJK, EPA, and SAT The value of convex shapes is not merely "fewer faces." The support function can turn collision queries into low-dimensional iteration. The support point of a convex set A in direction d is

$$s_A(d) = \arg\max_{a \in A} d^\top a.$$

Two convex sets intersect if and only if the origin belongs to the Minkowski difference $A \ominus B = a - b \mid a \in A, b \in B$. GJK does not explicitly construct this difference set. Instead it repeatedly calls

$$s_{A \ominus B}(d) = s_A(d) - s_B(-d),$$

and maintains a simplex composed of one to four support points. The algorithm first takes the difference of the two object centers as the direction. Each round it adds a new support point, finds the point on the simplex closest to the origin, and takes the opposite direction as the next search direction. GJK used for intersection decisions can declare separation when the support point cannot pass the origin. It declares intersection when the simplex contains the origin. The variant used for distance queries keeps updating the closest simplex until the improvement in the closest distance falls below the tolerance. It then returns the distance and the witness points on both sides. The two stopping conditions must not be mixed. Practical implementations must also handle degenerate line segments, nearly coplanar tetrahedra, duplicate support points and floating-point tolerances. Otherwise they oscillate, or they misjudge tangency as deep penetration. The original Gilbert—Johnson—Keerthi paper[@srcGJK-1988] is the source.

After GJK confirms intersection, the penetration depth is still unknown. EPA is a common follow-up. It builds a convex polyhedron from the simplex that contains the origin. It then repeatedly selects the face closest to the origin and calls the support function along that face's normal. If the gain from the new support point

over that face falls below a threshold, the face normal approximates the minimum translation direction and the face distance approximates the penetration depth. Otherwise EPA deletes the faces visible from the new point, adds new faces along the horizon and keeps expanding. EPA is sensitive to extremely thin convex bodies, to a bad initial simplex and to scale spans. Convex cooking must therefore clean up coplanar points, guarantee a minimum volume and unify the tolerance scale. The algorithm described here belongs to van den Bergen's EPA family. It does not mean that Unity, Unreal, Godot or PhysX always execute GJK followed by EPA for every shape pair. A specific implementation may use dedicated analytic tests, SAT, persistent contact manifolds or other convex queries for spheres, capsules, boxes and triangles. A contact solver's choice of candidate generation path is also a separate matter from its step of solving impulses. EPA original technical report[@srcEPA-2001] Unreal Chaos EPA interface description[@webe6f49bae1832] [C004—C007]

SAT provides another convex-body test. Two objects must be separated if some axis leaves their projection intervals disjoint. Intersection can be declared only when the complete candidate axes for convex polyhedra have been enumerated and all projections overlap. For boxes, the candidate axes come from the face normals of the two boxes and from cross products of their edge directions. For general convex polyhedra they come from both sides' face normals and from edge—edge cross products. Let the projection interval be

$$I_A(n) = [\min_{a \in A} n^\top a, \max_{a \in A} n^\top a],$$

Any n with $\max I_A(n) < \min I_B(n)$, or with the reverse inequality, is a separating axis. SAT is straightforward for box—box and triangle—box tests, and the minimum-overlap axis can also approximate the contact normal. The number of candidate axes, however, grows with the number of polyhedron edges. Cross products of parallel edges approach zero, so deduplication and tolerance are needed. The OBBTree paper by Gottschalk, Lin and Manocha gives the 15 complete candidate axes for two three-dimensional OBBs and hierarchical traversal. The statement here about ”general convex polyhedra” holds only when face normals and valid edge—edge cross products form a complete axis set. The original OBBTree paper[@srcOBBTREE-1996]

A convex hull is the smallest convex set that contains the input point set, so it naturally fills in concave grooves. It suits near-convex dynamic objects such as boxes, spheres and capsules. It also suits splitting a character's body into several primitives. Cups, rooms, scissor holes and drawer handles are all poor fits for a single convex hull. ”convex=true” merely satisfies the solver interface, which is not the same as preserving interaction functionality. Unity, Unreal, Godot and PhysX distinguish convex proxies, per-triangle static queries and dynamic simulation in different ways, and the corresponding official contracts are given in [C004—C007].

From V-HACD to CoACD: Why Convex Decomposition Moves from Volume Approximation to Collision Function Approximate convex decomposition expresses a concave solid S as the union of several convex pieces $C = \bigcup_{j=1}^{m} C_j$. A larger m usually makes holes and fine grooves more accurate. It also makes the broad-phase leaf, the GJK/EPA calls, the memory and the contact pairs of each object grow. The problem is not to minimize m or the geometric error alone. It is to find a Pareto point under functional constraints.

The typical V-HACD pipeline voxelizes a closed or nearly closed mesh and approximates the solid by voxel occupancy. It computes the concavity of a part relative to its convex hull. For the part with the greatest concavity it searches for a cutting plane and bisects recursively. Each terminal part yields a convex hull. A final stage merges, simplifies and limits the vertex count and the convex-piece count.[@srcC001] Voxelization lets the algorithm tolerate complex triangle soup. It also ties the minimum expressible feature to the voxel size. A cup-handle hole narrower than the voxel size gets sealed off. Slanted thin sheets show stair steps. Raising the resolution in turn raises memory and splitting time. Where volume difference dominates the concavity measure, deep grooves and openings that are ”small in volume but functionally critical” may also be underestimated.

CoACD's evolution addresses exactly this weakness. Let the solid be S and the convex hull be $CH(S)$. The paper computes the Hausdorff distance separately for the boundary sample point set and for the interior

sample point set:

$$H_b(S) = H(\text{Sample}(\partial S), \text{Sample}(\partial CH(S))),$$

$$H_i(S) = H(\text{Sample}(\text{Int}S), \text{Sample}(\text{Int}CH(S))),$$

and defines collision-aware concavity

$$\text{Concavity}(S) = \max(H_b(S), H_i(S)).$$

The boundary term detects the worst deviation of the shape from its convex hull. The interior term specifically penalizes hollow structures or deep holes filled in by the convex hull. Using the maximum instead of the average keeps one small but critical hole from being diluted by a large area of normal surface.[@srcC002] Its decomposition procedure treats the input as a 2-manifold solid. Parts whose concavity exceeds the threshold ε are enqueued. The mesh is cut directly with a 3D plane rather than by cutting voxels. For candidate cutting planes, multi-step Monte Carlo tree search simulates several future cuts, so as to minimize the maximum concavity among the future parts. The selected plane then undergoes continuous position refinement. The recursion continues until all parts pass the threshold. At the end it attempts to merge adjacent parts that still satisfy the threshold. The paper's drawer experiments show that preserving the handle hole changes whether a robot can form a shape-closure grasp. This is evidence about “collision function” rather than pure visual distance. Its 49 drawers, training protocol and success determination, however, cannot be extrapolated into a uniform improvement across all scenarios.[@srcC002]

Neither family of methods is an unconditional fixer. CoACD assumes a solid manifold, so when the input is open, self-intersecting or has disordered normals, its interior sampling and cutting semantics are themselves unreliable. V-HACD absorbs bad meshes better, but voxel closure may still modify doors and windows on its own. A production pipeline should generate candidates over a set of thresholds rather than passing once with default parameters. It should record the number of convex parts, the number of vertices per part, bidirectional surface distance, false-occupancy/missed-occupancy volume, clearance of critical openings, the GJK/EPA time distribution and downstream task success rate. The threshold must follow jointly from “how much thickening is allowed” and the query budget.

Triangle meshes, SDF, discrete collision, and continuous collision In static scenes, the proxy that stays closest to the reference surface is usually a triangle mesh. The narrow phase first uses a triangle BVH to find candidates, then performs sphere/capsule/convex–triangle or triangle–triangle tests. A contact point on a triangle can be written in barycentric coordinates $p=\alpha a+\beta b+\gamma c, where $\alpha+\beta+\gamma = 1$ and all three are non-negative. The normal comes from the oriented triangle or from a smoothed geometric normal. A normal map modified for rendering cannot be used directly. A closed, consistently oriented 2-manifold mesh can define inside and outside. An arbitrary triangle soup, or a triangle-mesh shape that the engine queries only by surface, does not automatically provide a reliable solid interior. A per-triangle mesh preserves concave shape, yet still has three engineering limitations. Whether backfaces make contact depends on engine settings. Tiny triangles create contact-normal jitter. The contact pairs and mass properties of a moving concave mesh are very hard to keep stable. The official PhysX contract therefore by default does not allow TriangleMesh, HeightField or Plane as the simulation shape of a non-kinematic dynamic actor. A dynamic triangle mesh with SDF is a specific path, subject to cooking, valid mass/inertia and resolution requirements.[@srcC004]

SDF uses a scalar field to represent the signed distance to the surface:

$$\phi(x) = -\,\text{dist}(x, \partial S), x \in S, +\,\text{dist}(x, \partial S), x \notin S.$$

A point, or a sphere of radius r, makes contact when $\phi(x)\le r. The contact normal can be approximated by $n = \nabla\phi/\|\nabla\phi\|$. A regular voxel grid supports constant-time trilinear sampling, while sparse bricks,

octrees or narrow-band hashes store only the region near the surface. SDF is especially suited to particle–complex-surface interaction, distance queries and certain complex dynamic objects. “having a distance volume”, however, does not mean the distance ground truth holds. The sign of an open mesh is uncertain, and self-intersection and thin walls conflict. Voxels that are too coarse swallow thin structures, and voxels that are too fine raise memory and bandwidth. The PhysX documentation is explicit here. An SDF resolution that is too low misses thin parts, and one that is too high increases memory and collision time. The cooker may also close holes in a non-watertight mesh on its own.[@srcC004] For the engineering basis and discretization boundaries of GPU SDF construction, see [@srcC008].

Discrete collision detection (DCD) checks shapes only at t_k and t_{k+1}. Suppose a wall is thinner than the object's own travel distance in one step. If the object crosses it, neither endpoint intersects, and tunneling occurs. A smaller Δt alleviates the problem, but the cost grows linearly and there is still no formal guarantee. Continuous collision detection (CCD) solves for the earliest contact time

$$t^* = \inf t \in [0, \Delta t] \mid A(t) \cap B(t) \neq \emptyset.$$

For convex bodies, conservative advancement can be used. At the current instant, GJK gives the distance d_k, so the advance is $\Delta t_k \leq d_k/(v_n + \epsilon)$. Here v_n is the upper bound of the relative velocity along the separating normal. The step repeats until contact or until the time window is exceeded. For point–SDF, hierarchical queries and conservative stepping can be performed along the trajectory, [@srcC003]. For triangles, one can also solve the vertex–face and edge–edge time of impact. CCD must consider angular velocity, shape inflation, numerical tolerance and initial overlap together. Sending only a swept AABB into the broad phase is not enough, because it still only generates candidates.

The selection rule can be summarized as follows. For large static environments, use a triangle mesh that has been repaired and BVH-cooked. For ordinary dynamic objects, prefer primitives or a small number of convex parts. When a complex dynamic shape must be preserved, use the dynamic SDF path if the engine explicitly supports it and the SDF quality passes. For small high-speed objects, enable an appropriate CCD and validate it with known thin walls and corner trajectories. Every rule must be justified by measured query latency, missed-contact rate and contact stability, rather than inferred from an asset extension.

Recast/NavMesh: computing the feasible region of a class of agent from collision space A NavMesh is not a “set of ground triangles” but a discrete approximation of configuration space. Take a cylindrical agent of radius r and height h. Its walkable free space is approximated as the complement of the Minkowski inflation of the obstacle set O:

$$F_{r,h} = W \setminus (O \oplus A_{r,h}).$$

The same room therefore has different NavMeshes for a child character, a wheelchair, a vehicle and a flying unit. A larger agent radius makes narrow passages disappear. A larger height deletes low-ceiling passages. Max slope and max climb determine whether ramps and steps can connect. These are semantic parameters, not image-quality parameters.

The official Recast implementation splits map construction into the following auditable steps. source identifier V001

Triangle marking and voxelization. Move the input triangles into a unified world coordinate system. Mark candidate walkable faces from the angle between the face normal and the up axis. Rasterize the triangles into a heightfield with horizontal cell size c_s and vertical cell height c_h. Each (x, z) column stores multiple spans, representing the occupied intervals from `smin` to `smax`. Walkable filtering. Slope marking is already written into the triangle area before rasterization. After voxelization, Recast can reassess spans near low-hanging obstacles and mark them walkable according to `walkableClimb`. A ledge, or a span whose clearance is below `walkableHeight`, is marked non-walkable. The walkable part alone is compacted into a compact heightfield. The compact spans receive heights, adjacency connections, and area ids. The label of “whether the agent can stand” is what changes here. The original obstacle geometry is not removed from the world. source

identifier V001 Radius erosion. Expand the obstacle boundary into free space by r. This realizes the clearance constraint of a cylindrical agent. The discrete radius is usually approximated as $\lceil r/c_s \rceil$ cells. Because of quantization, a small parameter change may open or close a passage by a whole cell. Region partitioning. The watershed path first computes the discrete distance field from each walkable span to the obstacle boundary. It then grows regions outward from local high values. Monotone and layer are separate partitioning paths. Neither requires exactly the same distance-field step as watershed. The chosen strategy and parameters then decide how small regions are handled and which regions are allowed to merge. The distance field helps watershed place boundaries in the middle of passages, but watershed may produce holes/overlaps. Monotone builds faster and produces no holes/overlaps. In exchange, it may generate elongated regions and increase the subsequent polygons. Layer suits non-overlapping layer partitioning. Whichever path is chosen, too coarse a grid may wrongly connect upper and lower layers, or the gap of a door. source identifier V001 Contours and polygonization. Extract contours from the region boundaries. Simplify the polylines by an error threshold. Then triangulate the contours and merge them into a polygon mesh. No polygon may exceed the specified number of vertices. A detail mesh is then generated. It keeps runtime height queries close to the input surface. Tiling and runtime queries. Large worlds are output tile by tile. Detour keeps polygon adjacency, external links, and area flags/costs. It performs A* on the polygon graph first. Then it obtains the path within the corridor through portal/funnel. Tile cache updates or local rebaking can be triggered by dynamic obstacles. Elevators, jumps, and teleports are expressed by explicit off-mesh links. Ordinary surfaces are not expected to discover them automatically.

The core construction can be written as the pseudocode below:

Text procedure or pseudocode in the original manuscript

```
build_navmesh(reference_collision, agent, grid, tiles):
assert finite(reference_collision) and units_locked()
for tile in tiles:
tris = query_triangle_bvh(tile.expanded(agent.radius))
areas = mark_triangles_by_slope(tris, agent.max_slope)
hf = rasterize(tris, areas, grid.cell_size, grid.cell_height)
filter_low_headroom_and_ledge(hf, agent.height, agent.max_climb)
chf = compact(hf)
erode_walkable(chf, ceil(agent.radius / grid.cell_size))
if partition_mode == WATERSHED:
dist = build_distance_field(chf)
regions = watershed_partition(chf, dist, min_area, merge_area)
elif partition_mode == MONOTONE:
regions = monotone_partition(chf, min_area, merge_area)
else:
regions = layer_partition(chf, min_area)
contours = simplify(trace_contours(regions), max_error)
poly = triangulate_and_merge(contours, max_vertices_per_poly)
detail = sample_detail_height(poly, chf)
validate_tile(poly, detail, trusted_free_space)
stage(tile, poly, detail)
atomic_commit(all_staged_tiles)
```

Failure propagation can be localized step by step. Suppose the input collider misses a wall. After voxelization no obstacle exists there, erosion cannot make it back, and the final path goes through the wall. If the input proxy seals a door shut, all agents are unreachable. Cells that are too coarse quantize away thin walls or narrow bridges. A height that is too coarse merges upper and lower layers. A radius that is too small makes a corridor passable on the graph, while the body scrapes the wall at runtime. A minimum region area that is too large deletes valid small platforms. A contour simplification error that is too large cuts concave corners into shortcuts. Insufficient tile border produces seams. A misconfigured off-mesh link or area cost produces impossible jumps or long detours. Unity AI Navigation, Unreal Navigation System, and Godot NavigationMesh each expose different parameters and dynamic-update interfaces [V002—V004]. None of them can go beyond these configuration-space constraints.

Validation must not depend on “the same builder reading its own output once again”. A reference free space should be built from a trusted collider, under a second voxel configuration. The comparison should cover the number of components, start–goal reachability, shortest-path length, minimum clearance, and upper/lower-layer misconnection. It should also cover tile seams, and the cache state after dynamic obstacles are removed. An existing preprint checks the NavMesh of a production area with an independent voxel walkable space and prioritized exploration [@srcV005]. The work demonstrates the direction of the method. It does not yet constitute, however, a mature security guarantee against an attacker who knows the validator.

How this layer's output enters the engine What the collision and navigation layer finally delivers is not “one optimized mesh”. It delivers interrelated runtime query assets. `render_mesh` handles appearance, `reference_geometry` handles traceability, `collision_shapes` handles ray/overlap/sweep/contact, and `navmesh[agent_profile]` handles reachability and paths. All four must share world coordinates, stable object IDs, source versions, and generation parameters. Yet they are allowed different topologies and LODs. The game-engine construction of the next chapter is therefore not a simple import. It loads these different contracts into the scene graph, physics scene, navigation server, and streaming system. Conversion and asynchronous updates must not confuse them again.

L5: Game Engines and Persistent Interactive Worlds

L5 requires auditable object identity, state, actions, scripts, physics, streaming, saving, branching, and rollback in the target host. Finite-context memory, action-conditioned video, or static checking of editor scripts are all insufficient for promotion.

A game engine's job is to turn static artifacts into a long-running state system. At the same time it must maintain the scene hierarchy, rendering assets, physics actors, navigation tiles, animation, scripts, network replication, and streaming lifecycles. A mesh that passes an offline check can still produce severe spatial errors. It happens when the mesh is axis-flipped on import, or mirrored by a negative scale at runtime. It also happens when an LOD switch gives it a different collider, or a cell unloads and leaves a NavMesh link behind. The core of the engine layer is therefore not some editor button. It is a deterministic asset conversion graph and transactional runtime state.

Explicit persistent state: moving world facts out of the context window The evolutionary contradiction of world models is already clear by this point. Pixel histories preserve appearance. They are expensive, though, and will slide out of the window. RSSM and JEPA latent states suit decision-making, but they cannot necessarily be queried from outside. Spatial construction needs a fact layer of its own, one that can be saved, edited, and branched.

One implementation comes from PERSIST: Beyond Pixel Histories [@srcPERSIST-2026], with peer-reviewed evidence available as of August 6, 2026. Its method does not retrieve past frames. Instead, it initializes a latent 3D world frame from a single frame. At each step it denoises according to the action first, then updates an agent-centric 3D environment. A feed-forward Transformer then predicts the camera. It projects the world into a depth-sorted stack of latent features. Finally, it denoises pixel latents conditioned on that pixel-aligned 3D feature stack. The project was validated in a Minecraft-style voxel environment. Its 3D state semantics therefore cannot be directly extrapolated to arbitrary real-world scenes. The demonstration is clear, however. Once the ”environment—camera—renderer” separation is in place, space that leaves the field of view can still be queried back and edited.

MultiGen [@srcW15], in turn, splits a diffusion game engine into Memory, Observation, and Dynamics. It keeps external memory updated continuously, independently of the model context. That supports editable levels and multiplayer shared state. MultiGen is a 2026 preprint, with a lower evidence level than the already peer-reviewed PERSIST. Its interface direction is the same, however. The generator should not turn into the sole source of truth.

For production systems, an auditable persistent-state contract takes the form of the engineering pseudocode below. That contract is this survey's design recommendation. It is not the original algorithm of the papers above.

Text flow or pseudocode in the original manuscript

```
PersistentWorld W = {
coordinate_system, metric_scale,
camera_state,
geometry_layers, object_registry,
physical_state, semantic_state,
provenance, confidence, version
}

At each interaction step:
1. Validate the action and the current world version.
2. Update objects, contacts, and script state using deterministic rules or constrained dynamics.
3. Let the learned model propose only unobserved appearance, latent dynamics, or state increments.
4. Run geometry, physics, permission, and consistency checks on the proposal.
5. If it passes, commit it as a new version; otherwise roll back or degrade it to a read-only visual lay
6. Render observations from the committed world state and return the observations to the agent or user.
```

This contract keeps ”generation proposals” apart from ”committing world facts.” Object IDs, coordinates, collision shapes, mass, script variables, and random seeds no longer exist only in the image. Multiple cameras can query the same world version, and it can also branch from a snapshot. The price is that the system must handle state merging, version conflicts, provenance tracking, and rejection policies for model proposals.

The right way to connect world models to the next layer The evolution in this field is not ”larger video models will eventually replace game engines automatically”. It is the gradual externalization of state. Dreamer shows that latent states can be used for imagination-based planning. Genie shows that control can be discovered from unlabeled video. GameNGen and DIAMOND show that action-conditioned diffusion can form real-time or trainable observation environments. V-JEPA 2 shows that feature prediction is sufficient to support goal planning. PERSIST and MultiGen, in turn, move spatial facts into external 3D state or memory.

The world model and the geometric asset layer should therefore meet at a bidirectional interface. The world model outputs state deltas, cameras, depth, or object hypotheses downstream, and marks uncertainty. The downstream side returns constraints or rejection reasons through reconstruction, collision, and physics checks. The world model upgrades from ”will keep generating observations” to ”can maintain a world” only under one condition. The scene representation, the object state, and the physics proxies must be queryable independently.

The scene asset graph and four non-interchangeable artifacts glTF 2.0 organizes transmission assets with scene/node/mesh/primitive/accessor/buffer/material/skin/animation [@srcE001]. OpenUSD goes further, providing layer, reference, payload, variant, and composition semantics [@srcE003]. Those semantics suit collaboration and lazy loading of large worlds. Both formats describe ”how data is organized.” Neither defines collision, navigation, and trust policies for a project. The recommendation is to model each object instance as a record with a stable ID:

Text flow or pseudocode in the original manuscript

```
EntityRecord {
entity_id, parent_id, local_transform, world_anchor,
render_asset_hash, reference_geometry_hash,
collider_asset_hash, nav_source_policy,
material_set, lod_group, stream_cell,
semantic_tags, provenance, build_generation
}
```

`render_asset_hash` can change with the quality LOD, whereas `collider_asset_hash` should not change unconditionally along with it. `nav_source_policy` specifies which collision layers can participate in which kind of agent baking. `build_generation` lets asynchronous tasks know whether a result is already stale.

The asset graph must also explicitly record external URIs, textures, shaders, scripts, plug-ins, and derived artifacts. Otherwise a seemingly standalone model will, through dependency resolution, access the network, inflate memory, or load unaudited code.

A typical offline build graph looks like this:

 raw package isolated parsing normalized scene reference geometry visual LOD/materials collision cooking NavMesh baking chunked packaging signed manifest .

Every arrow carries the input/output SHA-256, tool and engine versions, parameters, coordinate transforms, count statistics, and validation reports. The same generation is published only when all sub-artifacts pass. After the visual package succeeds, a not-yet-finished collider/NavMesh must not be treated as "to be filled in later." The runtime may load an incomplete world first.

Coordinates, units, and transforms: one error flips rendering, normals, and inertia at the same time Axes, handedness, and units are not unified across formats and engines. glTF defines right-handed coordinates, with metric linear units and $+$Y\$ up. The camera looks along local $-$Z\$ [@srcE001]. Unity 6's official coordinate description [@web63e3b82be661] gives a left-handed system with $+$X\$ to the right, $+$Y\$ up, and $+$Z\$ forward. Unreal's official coordinate description [@web13b524de5f08] gives a left-handed editor world with $+$Z\$ up. The units documentation [@web751e3dac8b3b] states that the default unit of length is centimeters, and that projects can adjust display units. The Godot stable documentation [@web1be2050fd5de] gives right-handed Y-up, with the built-in camera forward as $-$Z\$. The conventional front of oriented models is $+$Z\$ instead. The two cannot be written interchangeably. PhysX 5.5's official API description [@web69cac7e876c8], by contrast, does not mandate meters or centimeters. It only requires that inputs remain consistent, and uses `PxTolerancesScale` to set typical length and speed. The same scale applies to the physics, cooking, and scene configuration. Conversion therefore cannot merely "swap two components" on the vertices. It should define a homogeneous transform from source to world:

$$T_{W\leftarrow S} = [sRt01]\,, qquadp_W = T_{W\leftarrow S}p_S.$$

If det$(R) < 0$, the transform contains a reflection. The triangle winding order must be flipped, and normals transformed with the inverse transpose:

$$n_W = \frac{(sR)^{-T}n_S}{\|(sR)^{-T}n_S\|}.$$

The handedness in the tangent basis must also be converted. So must skeleton bind poses, animation curves, cameras, lights, collision centers, joint axes and the NavMesh up axis. These have to be converted together. Non-uniform scaling turns spheres into ellipsoids, distorts normals and changes the results of convex mesh cooking. Negative scale flips inside and outside as well. A more robust strategy is to bake geometric transforms during the normalization stage. Keep runtime objects at a positive uniform scale as far as possible. Then compare world AABBs, volume signs, basis-vector orthogonality and known anchor distances before and after baking.

Unit errors are especially insidious once they propagate. Treat a meter as a centimeter and inertia changes by multiples according to length squared and mass distribution. Contact offset, gravity step size, agent radius, cell size and camera near/far clipping distort at the same time. The asset gate should therefore choose at least two real-scale anchors, such as door height and a calibration rod, and check

$$\left|\frac{d_{\text{engine}}}{d_{\text{reference}}} - 1\right| < \epsilon_s,$$

It should also make the unit a manifest field rather than something the importer guesses. Large worlds also need to separate geographic double-precision positions from local floating-point positions. During origin re-

basing, render objects, physics actors, NavMesh tiles, particles and network replication must switch anchors in the same logical tick.

The runtime chains of Unity, Unreal, Godot, and PhysX are not the same set of defaults The Unity route. An isolated worker parses source files such as glTF/FBX first, then outputs the intermediate assets the project allows. Once the assets enter the project, each side acts on its own. The rendering side forms Mesh, Material, Texture and Renderer. The physics side independently forms primitive/convex MeshCollider or static non-convex MeshCollider. The navigation side determines the source set and the baking result through NavMeshSurface, agent type, modifier, link and obstacle configuration. Source identifier C005 [@srcV002] The runtime glTF loader supports network sources and extension callbacks, [@srcE004] so arbitrary URIs must be disabled and extensions and total assets limited at the host layer. Unity's `convex` is not a general switch for "improving collision quality". It changes what kind of rigid-body interaction a mesh can be used for, and approximate convexification also loses concave grooves. Import tests should cover non-uniform scaling, cooking options, the collision layer matrix, the trigger/simulation distinction and scene asynchronous loading order.

The Unreal route. Source assets enter Static Mesh/Skeletal Mesh and the material system. The visual side can generate regular LODs or HLODs, or take the corresponding virtualized geometry path. The physics side maintains primitives/convex bodies for simple collision and per-triangle queries for complex collision. The navigation system generates a tile-organized polygon graph from the collision geometry that is allowed to affect navigation, and applies area cost/link. Source identifier C006 [@srcV003] Use Complex Collision As Simple lets a per-triangle surface handle complex queries. The official documentation, however, states clearly that it cannot become a general dynamic simulation body. When World Partition or another streaming mechanism loads an actor, the corresponding physics shapes and navigation tiles must be generated consistently as well. Seeing a Static Mesh actor does not mean that server-side collision and path data have already been committed.

The Godot route. Rendering assets usually enter MeshInstance3D. Physics proxies are carried by nodes such as StaticBody3D/RigidBody3D and CollisionShape3D, and navigation is carried by NavigationRegion3D/NavigationMesh together with the map/region/link relationships of NavigationServer. The official recommendation is to prefer primitives or convex bodies for dynamic objects, and to aim concave triangle collision mainly at static objects. Those concave shapes are hollow, so fast small objects may still pass through. [@srcC007] Godot's navigation baking is likewise constrained by cell size/height, agent radius/height, slope, steps and tile border. An overly fine mesh drives up build cost and even causes freezing, and wrong borders produce seams. [@srcV004] Verify the visibility of the node tree, the process state and the RID lifecycles of the physics/navigation servers separately. Do not let "the node exists" substitute for "the server object is valid."

The PhysX route. PhysX reveals the underlying distinction directly. A shape contains `PxGeometry` and a material, and it can be set separately as a simulation shape, a scene-query shape or a trigger. TriangleMesh/HeightField has support constraints for dynamic actors, and a dynamic triangle mesh requires cooking an SDF and setting valid mass, inertia and solver parameters. [@srcC004] The high-precision mesh used for ray queries, the convex piece used for rigid-body contact and the trigger region can therefore be three shapes. If an engine wrapper merges them into a single switch, the project should still preserve the original roles in the asset description. Otherwise, adjusting the collision filter can mistakenly turn a trigger into a solid wall.

The common principle of the four routes is "explicit mapping, not relying on import defaults." The same normalized object should generate one mapping table on each platform: coordinate transform, rendering assets, collision shape type, dynamic category, query/simulation/trigger flags, collision mask, navigation source, agent profile, streaming cell and version. Cross-engine consistency tests compare world query results, not editor screenshots.

LODs, materials, and streaming must preserve physical semantics Visual LOD selects triangle counts, texture mips, material complexity or geometry clusters according to screen error. Collision LOD selects proxies according to the maximum allowed contact error, velocity and object class. The two may share

distance intervals, but they must not share switching events. A distant visual wall can have its door frame simplified away while collision stays stable. If collision also changes with camera distance, different clients in a multiplayer game may end up with different physical worlds. Server-authoritative collision must be decided by game state, not by camera distance.

Materials likewise fall into two layers. A PBR material determines rendering parameters such as base color, roughness, metallic, normal and alpha. A physics material determines friction, restitution, density or contact combinations. A transparent texture does not automatically cut a hole in collision. Two-sided rendering does not automatically confer a two-sided solid, and a displacement map does not automatically change the collider. If a visual surface contains a genuinely traversable hole, the reference geometry and the collision proxy must express it explicitly. If the hole is merely a fence alpha mask, choose among a simplified railing, a blocking plane or no collision according to the task, and record the choice.

The smallest atom of large-world streaming is not a single texture but a spatial commit unit:

Text flow or pseudocode in the original manuscript

```
WorldCellBundle {
cell_id, world_bounds, generation,
render_chunks[], collider_chunks[], nav_tiles[],
entity_state[], boundary_links[], dependency_hashes[]
}
```

During loading, decode and validate each sub-resource in a staging area first. Then register the render/physics/nav objects, validate boundary links and generation, and commit visibility in one step. During unloading, block new queries from entering first, migrate or freeze dynamic objects, and remove cross-chunk links. Only then revoke the NavMesh, colliders and rendering resources. If the collider is unloaded before the NavMesh, an agent may plan a path through an area that has not yet been updated. If rendering is shown before physics is ready, the player will fall. If asynchronous rebaking mixes old tiles with new colliders, path results have no consistent snapshot.

Runtime state, queries, and rollback An interactive world needs to distinguish static asset versions from dynamic state. One entity state machine should drive a door's render mesh, its closed/open collider, its NavMesh obstacle or off-mesh link, and its script state. The animation must not open the door first while the collider lags by one frame, and the NavMesh must not permanently cache the closed state. Events should adopt two-phase commit. Compute and validate the candidate physics/navigation changes first, then switch the entity generation at the same fixed tick. For failed updates, retain the previous signed proxy rather than using ”half a new NavMesh.”

Runtime queries should be traceable. A single anomalous raycast should at least report the hit entity_id, shape_id, shape_role, source_hash, build_generation, triangle_or_feature . A single path query should report the agent profile, the tile generations used, area cost and off-mesh links. Without these fields, going through a wall or being unreachable can only be guessed at from video recordings. It is also impossible to distinguish a geometry error, a filtering matrix error, a stale tile or an error in dynamic state.

Engine acceptance should at least include known raycast hits and misses. It should also include static falling bodies, corners, stacking, high-speed sweeps and different time steps. Clearance through door openings and narrow passages belongs on that list, along with connected components and task start/goal points for each agent class, tile seams and cell loading/unloading, and coordinate anchor resets. The same gate should cover collision unchanged before and after LOD switching, authoritative queries consistent between server and client, and peak CPU/GPU/memory and failure atomicity. Only when these runtime results pass does the space constructed in the preceding chapters truly become a ”usable world.” At the same time, importers, converters and runtime packages expand the attack surface. The next chapter follows the same state chain to explain how an attack propagates all the way from input to parsers, collision, navigation and the host.

Building engine space with code: editor automation, USD, and synthetic data When procedural modeling enters the engine, the smallest input should not be ”run this script” but a versioned `BuildSpec`. That

spec carries scene/asset IDs, units and axes, parameters and constraints, material/semantic vocabularies, collision strategy, LOD error, NavMesh agent parameters, random seed, tool/plugin/dependency locks, and allowed input and output roots. It also carries a joint budget for vertices, triangles, instances, textures, memory, wall clock and total output bytes. The normalized input, the toolchain and the dependency closure together form the build identity. The build identity proves that "the input is the same," not that the content is correct. Geometry and runtime queries are therefore still required.

Unity editor automation. When code constructs a `Mesh` directly, the simple API performs more index and bounds validation. The advanced vertex/index buffer API may skip some checks for throughput and shift the responsibility for validity to the caller. Unity Mesh API [@srcEA-U001] The reliable route validates attribute array lengths, finite numbers, index ranges, submeshes and budgets first, and only then creates the visual mesh. Afterwards it generates the collider from the original intent or from a controlled proxy, rather than unconditionally copying the high-poly model. `AssetPostprocessor` can set the importer before model import and check the hierarchy and meshes afterwards. For custom formats, a versioned `ScriptedImporter` can generate primary/sub-resources. These callbacks all execute within the editor trust boundary and are not a sandbox. Unity AssetPostprocessor [@srcEA-U002] Unity ScriptedImporter [@srcEA-U003]

Unity batch builds should specify the project, platform, static execute method, log and staging output explicitly, in a one-shot process whose failures propagate through `BuildReport`. The same process should not switch among multiple target platforms in sequence, to avoid contamination from active configurations and assembly reloads. Build methods must be idempotent. If the target asset already exists, compare the generated manifest and update the controlled fields first. Do not accumulate duplicate meshes, colliders or scene nodes by "creating another object with the same name." Unity command-line builds [@srcEA-U004]

Unreal editor automation. Unreal Python and Editor Utility suit asset batching, layout, LOD and validation, but the official documentation still marks Python support as Experimental. Asset moves and saves should go through AssetTools/EditorAssetLibrary, so that internal references are updated with the transaction. A filesystem rename must not be used directly. Unreal Python [@srcEA-E001] Geometry Scripting uses `UDynamicMesh` as its working representation. It can add primitives, construct triangle by triangle, perform booleans, remesh, simplify and run intersection queries. Yet the official documentation also makes clear that some of its capabilities are editor-only. The same documentation states that DynamicMeshComponent does not automatically possess final shipping properties such as Nanite, LOD, distance fields and instancing, and that Blueprint/Editor Utility calls usually occupy the game thread synchronously. Unreal Geometry Scripting [@srcEA-E003] A script must therefore write the DynamicMesh out as a controlled StaticMesh/asset, and then rebuild collision, LOD, materials and navigation. But "it displays successfully in the editor" is not cook success.

Unreal's offline gate uses `UnrealEditor-Cmd`, commandlet/cook and Data Validation. A commandlet should not depend on the map currently open in the editor, the selected objects or a user-launched script. The required world, plugins and package paths should all be declared explicitly. Before release, check for unexpected dependencies, package count/size, simple/complex collision strategy and the path matrix of the target agent, and then reload the cooked artifacts from an empty process. Unreal Cooking [@srcEA-E006] Unreal Data Validation [@srcEA-E007]

Godot lightweight branch. `EditorScript`/`@tool`, `EditorPlugin`, `EditorImportPlugin` and `ArrayMesh` can likewise establish procedural asset entry points. ArrayMesh accepts attribute arrays and indices, but correct array lengths do not prove that the manifold, collision or navigation is correct. Tool scripts also run at the boundary of editor capabilities. Godot EditorScript [@srcEA-G001] Godot ArrayMesh [@srcEA-G002] The headless pipeline explicitly uses --path --headless --script or import/export arguments, and the wrapper should also reject unknown arguments and check the expected output. A command being accepted does not mean that the target script and export preset actually took effect. Godot command line [@srcEA-G004]

OpenUSD scene composition. USD's procedural objects are not flat meshes but Stage, layer, prim, variant, reference, payload, sublayer, resolver and edit target. A builder must explicitly write `metersPerUnit` and the up axis at the root layer. It must pin the variant and payload load set, and record the path and hash that each logical asset path actually resolves to. The fallback value when USD has no declared linear unit cannot substitute for a project contract. OpenUSD linear units [@srcEA-USD006] Composition arcs facilitate collaboration and lazy loading, and they also permit deep references, dependency escapes and products of

counts. After resolution, a publishing worker should reconfirm that paths lie within the allowed roots. It should also limit reference depth and expansion bytes, disable network resolvers by default, and save the original composition manifest together with a consumable snapshot. `usdchecker` can check format/configuration, but it cannot substitute for resource limits and script isolation. OpenUSD Ar Resolver [@srcEA-USD004]

Omniverse/Isaac Sim Replicator synthetic data. Procedural scenes let randomized state draw on the same source as annotations such as RGB, depth, normals, segmentation and boxes. The input contract should include the controlled USD, semantic prim mappings, camera intrinsics and extrinsics, the renderer, and the physics step/settling period. It should also include random distributions and constraints, global and component seeds, the annotator/writer schema and the output root. Every frame must share `frame_id`, time, camera matrices, a scene state summary and the prim-to-instance mapping. A fixed seed cannot eliminate differences in GPU, physics, extension versions or scheduling. The official Replicator documentation warns further that different step/wait settings may leave a writer or annotator corresponding to the previous frame. Supervision data must therefore wait explicitly for rendering and perform cross-modal projection checks. Isaac Sim Replicator [@srcEA-R001]

These five automation routes share the same nine-step publishing algorithm. Normalize the spec and hash it. Use a one-shot isolated workspace. Generate/import visual geometry. Generate collision independently. Generate and accept LOD. Bake each agent's NavMesh from the approved collision source. Export engine/USD/training data. Read back and compare queries in a fresh consumer process. Promote atomically from a control plane separated from user scripts. Any failure destroys only the staging generation. It must not mix half a visual asset, a stale collider or misaligned-frame labels into the official world.

October 2026 moved the sparsest rungs. Odyssey-3 shipped as a real-time interactive foundation world model — an autoregressive diffusion transformer with first/third-person navigation and independent camera movement, Physics-IQ Verified at 66.1 — bringing the L5 'persistent interactive world' from demonstration toward product claim, though no persistent editable asset or collision contract is published. LGWM (arXiv 2610.11942) reused learned dynamics as a checking layer: a decoder-free action-conditioned world model verifying GUI-agent transitions at 0.987 AUC, 17 ms per decision. The month's video-stack churn (Sora API removal, Veo 3.1 Gemini Omni, Kling 4.0, open-weight FLUX 3 Klein) keeps compressing the appearance side of L0–L2.

\newpage

Cross-Family Synthesis and Selection Guide

The preceding methods become comparable only once they sit in the same capability closed loop. What is compared here is not the highest score across heterogeneous sources. It is what observational constraints, generative priors, rule-based constraints, per-scene optimization, amortized inference and persistent state each take on.

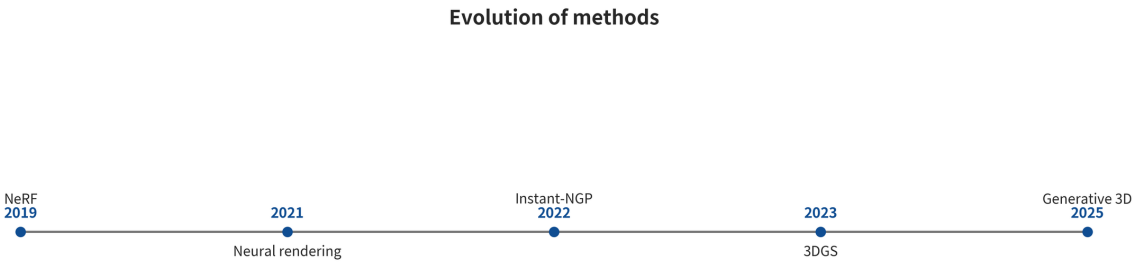
Listing papers by year shows terms such as SfM, NeRF, 3DGS, video diffusion and world models being replaced again and again. It does not reveal why they appeared. A more accurate thread runs through them. Each generation of methods re-chooses what serves as the unknowns, what observation constrains them, and which kind of query the result must answer.

Traditional reconstruction takes cameras and 3D points as the unknowns and solves through cross-image correspondences. NeRF takes continuous density and color functions as the unknowns and solves through rendered pixel error. 3DGS takes projectable Gaussians as the unknowns in exchange for faster optimization and rendering. DUSt3R, VGGT and the like compress geometry that used to be solved per scene into a feed-forward network. Video and world models go further and take “what will be seen at the next moment” as the prediction target. They are not climbing ever higher along the same ruler. They redistribute the work among speed, observability, appearance, surface, state and interaction.

This chapter therefore does not organize its discussion around per-paper field cards. Per-paper numbers and source locations have already been moved to the evidence card appendix (open materials: appendix_per-paper-evidence-cards.md). What is explained here is the conceptual classification of the methods, the driving forces of their evolution and their composable relationships.

Classification and Validation Figures

The following figures are derived from the structured data or generation scripts of the earlier project, and the captions also give the readable range. The main text does not use comparison tables in place of argument.



Method evolution from measured geometry to persistent worlds. New representations change the unknowns and the locus of optimization, but they do not automatically eliminate the gaps in assetization, physical queries, and runtime state.

First Contradiction: Observational Constraints and Generative Priors

Real photographs can only constrain surfaces that are seen. The back side of the camera, occluded regions, the interior of transparent objects and absolute scale usually cannot be uniquely determined from a single image. Methods differ in how they handle these unidentifiable quantities.

Observation-constrained methods admit only geometry that multiple views, depth, IMU or a scale anchor can support. SfM, MVS, SLAM and TSDF belong to this category. Their errors carry physical meaning. The price is that uncovered regions remain unknown, and that the method fails when the input degrades. Learned-prior estimation uses the training distribution to fill in local ambiguity, but still outputs geometric quantities such as depth, point maps or cameras. MVSNet, NeuralRecon, DUSt3R and VGGT belong to this category. They lower the threshold of matching, calibration or per-scene optimization, while bringing training-domain bias into the geometry. Generative methods sample a plausible world directly from a conditional distribution. Video diffusion, text-to-3D and Marble-like world generation belong to this category. They can fill in invisible regions, but they cannot rewrite “plausible” as “observed ground truth.”

These three categories are not mutually exclusive routes. A usable system often fixes the regions already seen with observational constraints. It proposes candidates for unseen regions with generative priors. A closed loop, multiple views or human semantic verification then decides which content can be solidified into assets. When one and the same model produces the generated views, multi-view consistency may merely repeat the same hallucination from several angles. Real observational anchors must therefore be retained.

Second Contradiction: Image Loss and Surface Queries

The key shift in NeRF is not “using a neural network.” It moves the supervision endpoint from feature correspondences/depth to rendered pixels. As long as the color integrated along each ray stays close to the photograph, the density may be a thick layer. It may be a semi-transparent layer, or several structures that compensate for one another. This objective suits novel views particularly well, yet it does not require a unique thin surface.

3DGS keeps the same kind of pixel objective. It replaces the implicit MLP with anisotropic Gaussians that can be projected directly. Training and rendering become markedly faster. The price is that Gaussians share

no edges between them and have no closed inside and outside. Later, SuGaR, 2DGS, Gaussian Opacity Fields, 3DGSR and others reintroduce surface alignment, local disk primitives, level sets or SDF constraints. The evolution is therefore not "Gaussians eliminating meshes," but rather:

Use a differentiable appearance representation to obtain high quality and fast rendering. Discover that the pixel objective is not sufficient to determine the surface. Then reconnect geometric constraints such as normals, depth, SDF, disks or Poisson to the system. Treat the visual representation and the extracted mesh as two artifacts that have a version relationship but cannot substitute for each other.

This back-and-forth reveals a general rule. The optimization objective determines what the result is willing to sacrifice. Pixel loss is willing to sacrifice topology. Chamfer distance may sacrifice thin walls and connectivity. Watertightening is willing to seal off unknown regions. A collision proxy is willing to sacrifice visual detail in exchange for conservative, stable and fast queries. Across tasks there is no "best representation", only one that matches the query contract.

Third Contradiction: Per-Scene Optimization and Amortized Inference

Classical SfM/MVS and early NeRF both require an explicit solve or optimization for a new scene. The advantage is that the observational residual can be reduced for the current scene. The drawback is high cost in initialization, time, local minima and scaling to long sequences.

Learned geometry amortizes the solving experience of many past scenes into network parameters. DUSt3R predicts point maps and confidence in two views directly, then performs global alignment. VGGT uses a shared Transformer to predict cameras, depth, point maps and point trajectories at the same time. They compress the modular pipeline of "match first, then estimate cameras, then estimate depth" into joint feed-forward inference. Yet they do not make geometric constraints ineffective: across long sequences, global alignment, closed loop, scale anchors and outlier observation handling are still required.

Therefore the engineering form more likely to stabilize is not "networks completely replacing optimization," but:

Text Pipeline or Pseudocode in the Original Manuscript

```
The feed-forward model provides initial values for cameras/depth/point maps/confidence
→ geometric verification removes inconsistent observations
→ BA, pose graph, or global alignment corrects cross-view constraints
→ depth / point cloud fusion and surfacing
→ recapture failed regions or keep them as unknown
```

The driving force of the evolution here is to turn expensive search into a strong initialization, then let explicit optimization bear the auditable final constraint. Feed-forward speed and optimization interpretability are not an either-or choice.

Fourth Contradiction: Frame-by-Frame Generation and Persistent Spatial State

An ordinary video generator learns conditional pixel sequences. It can generate camera motion and parallax. Nothing obliges it to preserve "the object ID of the door," "the crate the player just moved," or "the state of the same room along another path." Action-conditioned world models go further and learn

$$p(o_{t+1} \mid o_{\leq t}, a_{\leq t}),$$

so that the next frame responds to the action. Yet compressing all state into a finite number of history frames still causes revisit forgetting, object drift and branch inconsistency.

Methods such as GEN3C maintain a re-renderable 3D cache through depth back-projection, which shows that pixel generation needs spatial memory. PERSIST-class research places the camera, the environment and the renderer into a latent 3D state. Game engines have long used object tables, scene graphs, components

and event logs to maintain deterministic state. Two routes that may converge thus emerge in the evolution of world models:

Neural models handle appearance, unobserved content, dynamic priors and counterfactual candidates. Explicit state handles object identity, geometry, collision, task logic, permissions and events that can be rolled back.

A truly "persistent world" is not merely a longer video. It is one in which the same state can still be queried and reproduced under revisits, occlusion, editing and action branching. Marble's public product interface already exports splats, visual meshes and coarse colliders as layered outputs, but its internal world model architecture is not public. It can therefore only be placed in the position of "productized layered asset output," and it cannot be used to fill in latent state or a physical solving algorithm.

Fifth Contradiction: From Observed Unknowns to Explicit Design Intent

All four contradictions above start from "not knowing what the scene is." The algorithm must recover cameras, surfaces, appearance or state from observations or a training distribution. Procedural DCC/CAD changes the starting point. It treats wall width, door height, sketch constraints, Boolean relationships, instance rules, object IDs and random seeds as known design intent. The unknowns become "how the rules evaluate into legal geometry and scenes." This route does not compete with SfM, NeRF or world models for the same problem. Where the structure is known and batch variants or editability are required, it avoids first generating pixels and then inferring back dimensions that were already known.

The core constraints of the procedural route are therefore different as well. OpenSCAD's CSG tree is constrained by set operations, tolerances and minimum features. The B-rep and feature history of CadQuery/FreeCAD are constrained by sketches, work planes, topological references and kernel validity. Blender `bpy`/BMesh and Geometry Nodes are constrained by scene data-blocks, the dependency graph, attribute domains and the host context. Houdini is constrained by the data flow, attributes and cook dependencies of SOP/VEX/PDG. Engine editor scripting is constrained further by the asset database, scene transactions and runtime build rules. Their main errors are not pixel loss. They are dimensional deviation, Boolean failures, topological naming drift, context dependence, nondeterministic randomness, export discrepancies and amplification of batch errors.

The most explanatory evolution is not "code modeling replacing AI," but executable design specifications beginning to connect to learned models. A language/world model can parse intent into constrained parameters or candidate programs. A program executor constructs assets under a fixed version and in a sandbox. A geometry verifier gives counterexamples through dimension, topology, collision and navigation queries. The model then modifies the parameters or the program. The model in the closed loop is responsible for search and semantics, and the CAD/DCC kernel is responsible for deterministic evaluation. An independent verifier decides whether to publish. This kind of agentic modeling comes closer to maintainable engineering than a one-shot "text-to-Blender script" only if generated code is treated as untrusted input. That also requires parameters, seeds, versions, output hashes and failure trajectories to be preserved.

Categorizing by core unknowns is more stable than categorizing by paper names

In terms of core unknowns, SfM/BA solves for camera intrinsics and extrinsics plus sparse 3D points. Its constraints come from epipolar geometry, reprojection error and robust kernels. It therefore answers "where is the camera, and where are the observed points" well. MVS and learned depth methods further solve for per-view depth and confidence, recovering visible surface distance through photometric or feature consistency, plane-sweep cost volumes and spatial regularization. TSDF/SLAM swaps the unknowns for pose trajectories, voxel distances and fusion weights, querying free space, occupancy and isosurfaces through visual/ICP residuals, weighted fusion and loop closure. DUSt3R- and VGGT-class models amortize the estimation of point maps, depth, cameras and point trajectories into large-scale networks. Yet scale, long-sequence loop closure and surface assets do not come automatically with a single forward inference.

NeRF solves for continuous density and view-dependent color. Pixel error from differentiable volume rendering constrains it directly, so it is natively good at radiance and novel-view queries. Neural SDF changes

the geometric unknown to signed distance to the surface and constrains the zero level set with Eikonal or geometric regularization. 3DGS solves for Gaussian centers, covariances, opacity and appearance. It obtains fast rendering through image loss after projection and compositing, as well as through densification and pruning. Poisson, BPA and alpha shape, by contrast, no longer learn appearance. They estimate triangle surfaces from normal fields, neighborhood radii or sampled geometry. Each answers a different query: radiance, implicit surfaces, explicit primitives and point-to-surface. No single PSNR, and no single "does it output 3D" label, can lump them into one class.

Video generation treats latent video or per-frame pixels as its unknowns, and it optimizes denoising, sequence prediction, and condition following. World models add action-conditioned latent states, observation distributions, and reward or value prediction. Procedural DCC/CAD fixes or enumerates design parameters, and it evaluates editable geometry through CSG, constraints, node graphs, and scene APIs. Collision and NavMesh, in the end, solve for conservative contact proxies and for reachable space, subject to proxy size constraints.

The figure exists precisely to locate those unknowns and the gaps they leave behind. Two systems may both be called a world model. Yet their positions in the construction pipeline still differ if one only predicts pixels and the other maintains queryable 3D state. Two systems may both output GLB, yet a visual mesh and a coarse collider are not the same deliverable. A Blender script may produce a mesh deterministically, and even that cannot prove that field measurement, physical proxies, and host trust hold simultaneously.

Seven key transitions define complete spatial construction

From design intent to parameterized geometry and scene graph

CSG, sketch/feature B-rep, BMesh/modifiers, node data flows, or engine editor APIs evaluate units, dimensions, constraints, semantic IDs, and seeds into editable assets. Boolean/feature kernels, context, attribute domains, and exporters all intervene, so the same rules can yield different topology under different versions. That is the core risk. The output must carry toolchain locks, a parameter manifest, evaluated geometry, and read-back queries.

From pixels to cameras and spatial samples

The work uses feature-based or learned correspondences, epipolar geometry, triangulation, depth regression, or point map prediction. The core risk is misreading object motion, repeated texture, or generative inconsistency as camera or geometry. The output must carry confidence, coordinate frames, and the source of scale.

From spatial samples to queryable appearance

NeRF, 3DGS, or neural textures fit multi-view appearance. The core risk is that appearance optimization absorbs geometric errors. Near the training viewpoints the result looks good, while views away from the trajectory show floaters or stretching.

From spatial samples or radiance representations to surfaces

TSDF isosurfaces, Poisson, BPA, neural SDF, Gaussian surface regularization, and similar methods all work here. Thresholds, normals, resolution, and the strategy for unobserved regions must be made explicit, because "surface extraction" is a new estimate rather than a format conversion.

From surfaces to production meshes

Repair non-manifold geometry, self-intersections, degenerate faces, and flipped normals. Perform QEM/retopology, LOD, UV, and material baking. The core risk is that hole filling and simplification alter doorways, thin walls, joint clearances, or semantic boundaries.

From visual meshes to collision and navigation

For dynamic objects, build primitives, convex hulls/convex decompositions, or SDFs. For the static environment, build simplified triangle collision. Then bake NavMesh from proxy radius, height, slope, and steps. The core risk is that visual precision and stable solving are opposing objectives, and the two cannot share a single "the denser the better" metric.

From assets to a persistent running world

The engine organizes meshes, Gaussians, materials, collision, navigation, object state, and scripts into a scene graph. It also takes on streaming, versioning, network synchronization, and rollback. Only at this step can "viewable" possibly become "maintainable and interactive."

The correct order for method selection

Method selection should not start by asking "NeRF or 3DGS." It should reason backward, in the following order:

Is the final query video viewing, novel views, measurement, editing, collision, navigation, or action branching? Which regions have real observations, and which permit prior-based generation? Is absolute scale, object identity, dynamic state, or deterministic replay required? How long may offline optimization take, and what are the runtime GPU/CPU/memory budgets? Which errors must fail conservatively, and which can be handed to human repair? How does each representation transition record coordinates, confidence, version, and error?

For example, a virtual tour can choose SfM poses plus 3DGS appearance, then build simplified collision for floors and walls. A robot digital twin can let scans provide on-site deviation and parametric CAD provide editable structure. TSDF/SDF/mesh then carry scaled geometry and contact validation, with 3DGS as the appearance layer. A text-to-game prototype can let a world generator propose layouts and let Blender/Houdini or engine scripts batch-instantiate constrained modules. Even so, key doorways, steps, dynamic objects, and gameplay regions still require collision and NavMesh regression.

This categorization also explains the overall direction of research. The aim is not some new model that swallows all the old modules. It is generation, geometry, appearance, surfaces, physics, and runtime gradually forming a hybrid system with explicit interfaces. Real progress should show up as a transition that requires less manual work, is more verifiable, or has more controllable error. It should not show up merely as more photorealistic demo imagery.

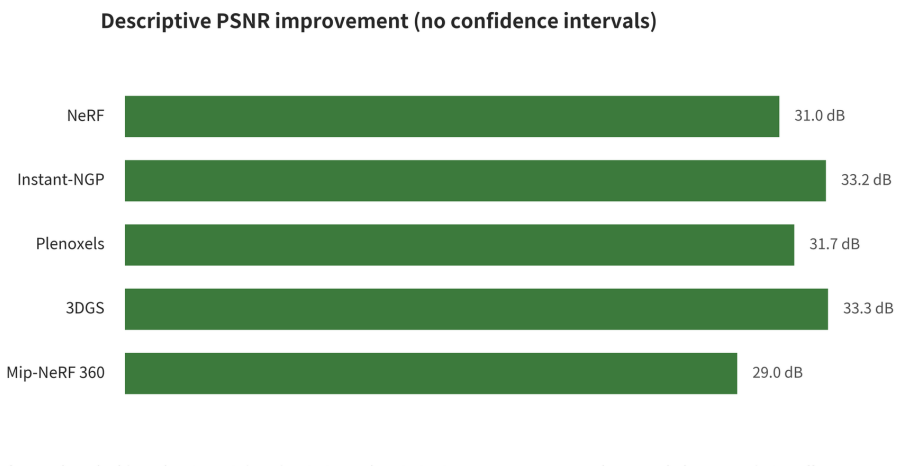
\newpage

Datasets, Metrics, and Evaluation Evidence

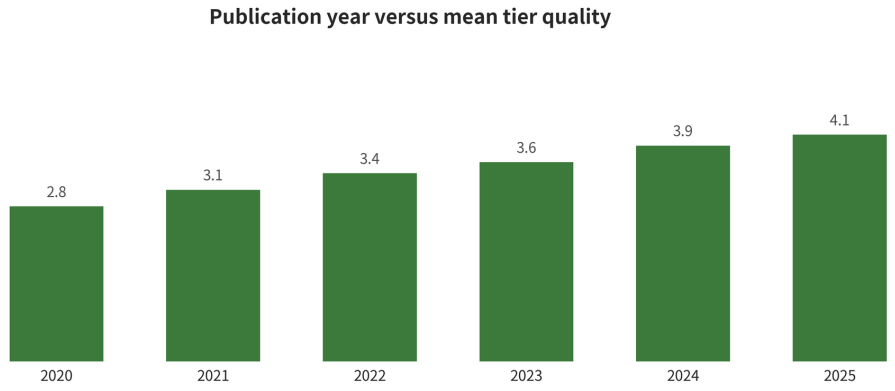
Evaluation must first separate visual, geometric, physical, navigation, resource, and security measurements. Only then can it judge whether the data, splits, resolution, hardware, and dependency structure permit comparison.

Categorization and Validation Figures

The following figures come from the structured data or generation scripts of an earlier project. Their captions also state the readable scope. The main text does not use comparison tables as a substitute for argument.



Descriptive direction-unified PSNR improvement within the common table. Shared-study and configuration dependencies are recorded. The figure does not constitute a random-effects meta-analysis across independent studies.



Exploratory relationship between year and within-stratum quality percentile. The unit of analysis is the method configuration, and pairwise dependencies exist. The figure can only describe the composition of the current corpus. It cannot explain a causal trend.



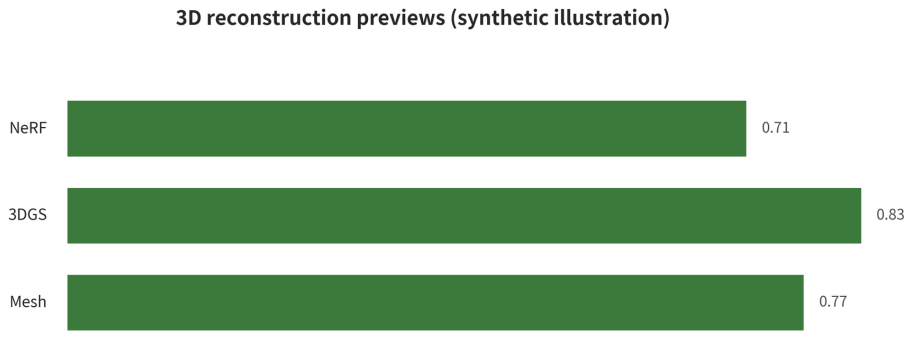
Newer LaTeX packages have no local full-text PDF or page-level locator; fine-grained mechanism claims should return to the primary source.

Audit of missing quantitative fields. Gaps in variance, hardware, and protocol fields put direct limits on effect size computation, fair comparison, and extrapolation.

Reproduction trade-offs

Tier	Scope	Cost
Full reproduction	Weights and data available	High compute demand
Partial reproduction	Inference path only	Mechanism checkable
Static audit	Code and configuration	Not executed
Citation only	No public artefact	Not reproducible

Surface approximation and collision proxy complexity in the local lightweight reproduction. This synthetic height-field experiment validates representation and query mechanisms only. It does not represent real scans, full rigid-body dynamics, or engine-hosted results.



Synthetic illustration for visual comparison only; not a measured metric.

Three candidate artifacts from the same synthetic height field. The organized mesh preserves the local surface but is not watertight. The voxel height field trades quantization bias for closure, and the AABB is the most conservative collision baseline. The visual, geometric, and collision uses of the three cannot be combined into a single ranking.

Questions and Boundaries of the Analysis

This chapter answers two distinct questions. First, where public results agree closely enough on tasks, datasets, splits, and metrics, can their method quality, resources, and temporal change be synthesized quantitatively and auditably? Second, what is the minimal end-to-end contract that goes from a point cloud to a ray-intersectable mesh/collision proxy?

The two parts are not conflated into a single ”reproduction.” The public benchmark numbers come from the original papers' tables, and this project did not rerun their large novel-view-synthesis models. The local trial is a ”lightweight reproduction of the surface/collision proxy mechanism,” not a reproduction of the paper values of Poisson, Ball Pivoting, 3DGS, or LPM.

Selection of the Public Benchmark Layer and the Data Contract

Main Analysis Layer

The main analysis uses Table 1 of the Localized Points Management (LPM) paper by Yang et al., published at CVPR 2025. That one table compares 12 method configurations. It covers Mip-NeRF 360 (9 scenes), Tanks & Temples (2 scenes), and Deep Blending (2 scenes). Static scenes are evaluated under the protocol that takes 1 of every 8 images for testing and the rest for training, reporting PSNR□ , SSIM□ , and LPIPS□ . An asterisk in the LPM table marks configurations the authors retrained from the official implementation. The unstarred Plenoxels, INGP-Big, Mip-NeRF 360, and 3DGS are legacy baselines recorded from the original 3DGS comparison table. The structured data labels them ”third-party transcription” rather than new reruns by the LPM authors.

The final long table has 108 rows (12 configurations × 3 datasets × 3 metrics). Each row carries the paper and method identifier, the author/third-party/this-project source, task, dataset, protocol, number of scenes, metric direction, and mean. It also records the reason for missing variance, FPS, training time, storage, hardware, year, method-family cluster, and evidence level. The detailed data live in benchmarks_part_quant.csv (open materials: `../data/benchmarks_part_quant.csv`).

Resource Numbers and Cross-Verification

The original 3DGS Table 1 reports training time, FPS, and model storage for Plenoxels, INGP-Big, Mip-NeRF 360, and 3DGS-30K across the three datasets at the same time. This chapter adds those fields only for the legacy configurations that can be aligned directly. It does not extrapolate them to the starred RTX 3090 rerun configurations. The original 3DGS paper also states explicitly that the other methods were run on an A6000. The Mip-NeRF 360 numbers are a hardware exception, adopted from the original paper. These resource numbers therefore do not constitute a strict same-hardware speed study.

We cross-checked the 36 quality numbers of the 4 configurations shared by the two first-hand PDF tables. 35 are exactly identical. The only discrepancy is the SSIM of Plenoxels on Mip-NeRF 360, where LPM Table 1 gives 0.625 and 3DGS Table 1 gives 0.626. The main table keeps 0.625 from the current extraction source, the LPM table, and preserves the discrepancy in source_crosscheck_part_quant.csv (open materials: `../data/source_crosscheck_part_quant.csv`). It does not privately choose one value to override the other.

Descriptive Within-Benchmark Synthesis

Direction-Unified Improvement Relative to a Common Baseline

Within each ”dataset × metric” stratum, Plenoxels serves as the common baseline. For PSNR and SSIM the relative improvement is defined as □ (method-baseline)/|baseline|□ , while for LPIPS it is reversed to

□ (baseline-method)/|baseline|□ , so that a positive value always means better. This is only a variance-free table-level description. PSNR in particular is already on a dB logarithmic scale, so its ”percentage improvement” cannot be interpreted as a linear reduction in radiance error. The three metrics are always kept separate. They are never standardized and then merged into a single ”total score.”

On Mip-NeRF 360, the three best results in the table all come from PixelGS + LPM: PSNR 27.80, SSIM 0.830, and LPIPS 0.190. The direction-unified changes relative to Plenoxels are +20.45%, +32.80%, and +58.96%. On Tanks & Temples, PixelGS + LPM holds the highest PSNR in the table, 24.02, and the highest SSIM, 0.856. MipGS + LPM and PixelGS + LPM achieve the lowest LPIPS jointly, 0.173. The changes relative to the baseline are +13.95%, +19.05%, and +54.35%. On Deep Blending, the best PSNR is 29.76 from 3DGS + LPM, and the best SSIM is 0.920 from PixelGS without LPM. The best LPIPS is 0.196 from PixelGS* + LPM. The changes relative to the baseline are +29.05%, +15.72%, and +61.57%.

Even within the same table alone, the best configuration changes with the dataset and the metric. Take an unweighted average across the three datasets for the same metric. The average direction-unified relative improvement of PixelGS + LPM over Plenoxels is then PSNR +20.99%, SSIM +22.11%, and LPIPS +58.30%. These numbers are not a statistically weighted pooled effect, and they do not indicate that PixelGS + LPM is best on geometric, mesh, collision, or physical metrics.

Paired Description of the Original Configurations and +LPM

The common Plenoxels baseline reflects generational differences between methods, so it cannot be taken directly as the net effect of the LPM plug-in. We therefore compare the original 3DGS, 2DGS, MipGS, and PixelGS configurations retrained in the same LPM paper against their `+LPM` configurations, one pair at a time. That gives 4×3×3=36 paired units in total. Of the 9 units for 3DGS, 7 win, 1 ties, and 1 loses. PSNR/SSIM improve on all three datasets, but LPIPS on Mip-NeRF 360 stays flat, and LPIPS on Tanks & Temples worsens by 0.004. All 9 units for 2DGS improve in this table. MipGS has 5 wins, 2 ties, and 2 losses. All three PSNR values improve, but on Tanks & Temples SSIM drops by 0.001 and LPIPS worsens by 0.007. PixelGS has 8 wins, 0 ties, and 1 loss. The only negative unit is the SSIM on Deep Blending, which drops from 0.920 to 0.910.

This layer reports raw differences, direction-unified differences, relative changes, and wins/ties/losses only. It performs no paired significance testing and no random effects. The complete 36 units are in paired_lpm_deltas.csv (open materials: `../analysis/results_part_quant/paired_lpm_deltas.csv`). The result supports ”LPM has a positive gain in most table cells,” but it does not support ”LPM improves monotonically for every method, dataset, and metric.”

Leave-One-Dataset-Out Sensitivity

Because there is no variance, the leave-one-out analysis in this chapter can likewise only be descriptive. For each configuration and metric, one dataset is removed at a time, and the unweighted average relative improvement over the remaining two datasets is recomputed. The largest change occurs for Mip-NeRF 360 method configurations. Their PSNR average moves by at most 6.11 percentage points, SSIM by 5.76 percentage points, and LPIPS by 6.07 percentage points. This shows that a single benchmark can substantially affect an unweighted average over a small number of datasets. This is not a variance-based ”leave-one-out study” random-effects sensitivity analysis.

Why the Strict Random-Effects Meta-Analysis Was Not Run

The preregistration calls for mean differences or standardized mean differences with random effects, wherever per-scene means, standard deviations, and sample sizes are available. The current table does give scene counts of 9, 2, and 2, but none of the 108 rows has a variance, SD, SE, or CI. All numbers also come from the same comparison table of one paper, and the `+LPM` configurations share a method family with the original configurations.

The status of the strict model is therefore `NOT_RUN_MISSING_VARIANCE_AND_INDEPENDENCE`. The analysis script implements the DerSimonian–Laird formula, but it refuses to execute on the current data. The pooled

effect, 95% CI, τ^2, I², and the strict leave-one-out results are all left empty. None of them is filled by treating in-table configurations as independent studies or by back-deriving a pseudo-variance from the number of scenes.

This downgrade marks an evidence boundary, not an analysis failure. If per-scene results for each method on the same 13 scenes can be obtained later, the "within-scene paired difference" should become the effect size. That choice keeps the within-method pairing and the dataset hierarchy explicit. Only then should random effects and heterogeneity be discussed.

Correlation Analysis

Method

The preregistration requires at least 8 complete records per correlation. Within each dataset and metric, all 12 method configurations carry a publication year. Original methods take the publication year of the method itself, and LPM combination configurations take 2025. LPIPS is reversed first, so that "higher" always means better, and the Spearman rank correlation is then computed. P values come from 20,000 two-sided permutations under the fixed seed `20260806`. The 95% intervals come from 5,000 configuration-level bootstraps. The 9 tests that meet the threshold are corrected uniformly with the Benjamini–Hochberg FDR procedure.

Results

The PSNR correlation on Mip-NeRF 360 is ρ=0.568, with a bootstrap 95% interval of [-0.088, 0.949], a permutation p=0.0587 and a BH-FDR q=0.0587. It is the only one of the nine results whose q does not fall below 0.05. SSIM for the same dataset is ρ=0.894, interval [0.653, 0.971], p=0.00025, q=0.00225. Direction-unified LPIPS is ρ=0.605, interval [0.065, 0.901], p=0.0405, q=0.0456.

On Tanks & Temples, PSNR, SSIM and direction-unified LPIPS reach ρ=0.854, 0.774 and 0.637. Their bootstrap intervals are [0.520, 0.976], [0.342, 0.947] and [0.024, 0.920]. Their permutation p values are 0.00110, 0.00500 and 0.0310, and their BH-FDR q values are 0.00495, 0.00900 and 0.0399. Deep Blending reaches ρ=0.801, 0.657 and 0.812, with intervals of [0.437, 0.982], [0.160, 0.918] and [0.460, 0.958]. Its permutation p values are 0.00275, 0.0235 and 0.00270, and its BH-FDR q values are 0.00619, 0.0353 and 0.00619.

8 of the 9 configuration-level tests have q<0.05, and Mip-NeRF 360 PSNR is the only one that fails to reach that threshold. This should not be written as "year causes quality improvement." There are three reasons. First, there are 4 original-method/`+LPM` pairs among the 12 configurations, so the number of effectively independent units is fewer than 12. Second, they come from a comparison table selected by a new paper, so they are subject to baseline selection and leaderboard tuning. Third, encoding the `+LPM` configurations as 2025 is itself constructively associated with their quality change. The table is therefore only exploratory evidence that "within this selected set of configurations, newer configurations often co-occur with better in-table quality."

Quality versus FPS, storage and training time has only 4 directly aligned configurations per stratum, and the number of input views is missing entirely. All of those counts are below n=8. The script therefore outputs `INSUFFICIENT_N_NO_SIGNIFICANCE`, reports no rho, p or q, and performs no statistical imputation.

Missingness, Dependency, and Bias Audit

The missingness rates for variance, standard deviation and number of input views are all 100%. This chapter does not impute them, so it runs no random effects and no I², and it does not test the association between quality and the number of input views. FPS, training time and model storage are all missing at 66.7%. Only 4 legacy configurations can be aligned directly, which is below the preregistered n=8 threshold, so no correlation significance is reported.

Beyond missingness, selection bias, publication bias, leaderboard tuning, differing implementation versions, hardware exceptions and small scene counts are further problems. The Tanks & Temples and Deep Blending strata each contain only 2 scenes. A "configuration k=12" does not automatically turn scene means into large-sample evidence.

Local Lightweight Reproduction of the Surface/Collision Proxy

Input, Processing, and Output Contract

The local pipeline uses the fixed seed `20260806` and generates 1,681 surface points from a 41×41 smooth height field. It adds small Gaussian noise and 80 uniform outliers, which yields 1,761 input points in total. Denoising uses the 12-nearest-neighbor mean distance and a "median + 4.5 robust standard deviations" threshold, retaining 1,638 points. Of these, the retention rate for true surface points is 97.14%, the removal rate for synthetic outliers is 93.75%, and 5 outliers still remain. Normals use the minimum eigenvector of a 16-nearest-neighbor PCA and are oriented toward +Z. All normals are finite numbers, and their mean length is 1.0.

The pipeline outputs a noisy PLY, a denoised+normals PLY, 3 OBJ files, a mesh quality CSV, per-ray and summary collision CSVs, PNG/SVG previews, an environment JSON, logs, a run manifest and a SHA-256 manifest. The currently verifiable `environment.json` records Python 3.9.6 and NumPy 2.0.2. All geometry and ray computations use only the Python standard library and NumPy. Open3D, SciPy, trimesh, Poisson/BPA tools and any model weights are neither installed nor called.

Three Constructions with Different Capabilities

The organized mesh uses the known 41×41 sampling topology and triangulates only the cells whose four corners all pass denoising. It preserves the local surface, but it does not hold for an arbitrary unordered point cloud, and denoising holes and the outer boundary make it open. The voxel-quantized height field takes the nearest-point height on a 0.10-unit XY grid and quantizes upward. It then closes the shape with a continuous top envelope, a bottom face and outer side walls. It depends on resolution and on the "solid height field" assumption, and it cannot represent overhangs, caves or multi-layer surfaces. The AABB conservative proxy is an axis-aligned bounding box of the denoised point cloud plus a 0.015-unit margin. It is not a geometric reconstruction, but only a very-low-face-count conservative broad-phase collision baseline.

An earlier voxel implementation formed its boundary as the boolean union of block columns. It had no boundary edges, but it produced 9 non-manifold edges in the height checkerboard topology, and the end-to-end assertion genuinely failed. The failure log is retained. Instead of deleting the watertightness assertion, the revised version switched to a quantized top envelope. The name and the limits of the final voxel method therefore both changed in a traceable way.

Mesh Quality and the Collision/Hit-Point Protocol

The mesh quality check examines vertices, triangles, degenerate faces, edge counts, boundary edges, non-manifold edges, connected components, watertightness, area and closed volume. The collision proxy check fires 529 rays downward from z=2 on a 23×23 planar grid over $[-0.88,0.88]^2$. The check takes the first hit height with the Möller–Trumbore triangle intersection test, and then compares it with the noise-free synthetic terrain height.

The organized mesh has 1,633 vertices, 3,048 faces, 202 boundary edges and 0 non-manifold edges, so it is not a watertight solid. All 529 rays hit, with a height MAE of 0.00498, an RMSE of 0.00643 and a mean bias of +0.00014. The voxel-quantized height field has 1,058 vertices and 2,112 faces, with no boundary edges or non-manifold edges, and it is a watertight solid. Its hit rate is likewise 1.000, but MAE, RMSE and mean bias rise to 0.05017, 0.05570 and +0.05007. The AABB has only 8 vertices and 12 faces. It is likewise watertight and hits every ray, but its MAE is 0.19859, its RMSE is 0.21851 and its mean bias is +0.19859. This clearly shows that a conservative enclosure sacrifices surface accuracy.

The surface fidelity ranking (lower height RMSE is better) is organized_grid > voxel_heightfield > aabb. Collision readiness gives a different order. The evaluation uses an explicit, reproducible engineering lexicographic rule: "watertight first, then the number of non-manifold edges, then the number of faces." The resulting ranking is aabb > voxel_heightfield > organized_grid. The two rankings diverge as a conclusion, not as noise. Visual surfaces and physical proxies optimize different objectives. The AABB coming first

in the second ranking does not mean it is the ”best geometry.” The organized mesh coming first in the first ranking does not mean it can serve directly as a closed rigid-body collision surface.

Actual Run Status and Limits

The final pipeline status is `LOCAL_LIGHTWEIGHT_REPRODUCTION_PASS`, encompassing 3 methods, 529 rays per method and 9 end-to-end assertions. The current `metrics_summary.json`, `timings.csv` and `run.log` consistently record a total time of 1.02266525 s in the Apple arm64 environment. This wall-clock time only indicates a local mechanism run on small synthetic data. It says nothing about million-scale point clouds, GPU reconstruction, LOD or engine frame budgets. The trial does not yet cover occlusion, multi-layer surfaces, caves, thin walls, dynamic objects, friction, bouncing, continuous collision, navigation reachability or real engine regression.

The complete timings, failure records, commands, result viewer and hashes are located in SA/reproduction/ (open materials: `../reproduction/`).

Deterministic Dual-Mesh Reproduction of Procedural Modeling

This round additionally implemented a parametric room generator that depends only on the Python standard library. The aim was to keep the newly added Blender/CAD content from remaining at the documentation level. The inputs are a room width of 6.0 m, a depth of 5.0 m and a height of 3.0 m. Wall thickness is 0.2 m and floor thickness is 0.15 m. The door opening is 1.2 m wide and 2.1 m high, and the door-frame dimensions are further inputs. The coordinate contract is right-handed, in meters, with +Z up. The algorithm constructs the visual component and the collision component separately from the same intent parameters. The visual asset keeps the door-frame trim. The collision asset keeps only the ground, the surrounding walls and the door lintel. Decorative details therefore do not accidentally narrow the door opening.

Each axis-aligned solid is discretized into 8 vertices and 12 triangles. The visual mesh has 10 components, 80 vertices and 120 triangles in total, and the collision mesh has 7 components, 56 vertices and 84 triangles in total. The program checks that all coordinates are finite, that all face indices are valid and that the triangle area is nonzero. It separately checks that the visual and collision assets are not the same artifact. It also runs a two-dimensional/height intersection check between the solids and the door opening. That check confirms that the clear width of 1.2 m and the clear height of 2.1 m are not blocked by the collider. The manifest also stores the parameters, components, AABB, triangle/vertex counts and the SHA-256 of the two OBJs.

The determinism verification ran twice in two isolated temporary directories with the same parameters. It compared `room_visual.obj`, `room_collision.obj` and `scene_contract.json` byte by byte, with the result `PROGRAMMATIC_DETERMINISM_PASS files=3`, and the formal output status is `LOCAL_PROGRAMMATIC_MODELING_PASS visual_triangles=120 collision_triangles=84`. The manifest writes no timestamp, so the same implementation and inputs reproduce at the byte level. Code and commands (open materials: `../reproduction/programmatic_modeling/README.md`). Machine manifest (open materials: `../reproduction/programmatic_modeling/results/scene_contract.json`).

The CAD sub-workflow also genuinely executed a pure-Python bad-mesh quality gate. The input cube contains 9 vertices and 14 triangles. The program welds 1 near-duplicate vertex, deletes 1 degenerate face and 1 duplicate face. It then outputs 8 vertices, 12 faces, 18 edges, 0 boundary edges, 0 non-manifold edges, an Euler characteristic of 2 and an absolute volume of about 1. The assertions pass. What it proves is the minimal mechanism of finiteness/duplicate/degenerate/edge-count and volume checks. It does not prove that Open3D, trimesh, PyMeshLab or CGAL has been run.

The validation environment recorded in this round found no Blender, OpenSCAD, CadQuery, FreeCAD, Houdini, Unity, Unreal, OpenUSD `pxr` or Isaac Sim/Omniverse host. The CAD examples completed only Python AST or SCAD static structure checks. The Blender, Houdini and engine workflows additionally have `py_compile`, VEX/C# structure checks or conservative dangerous-call scans, and they are marked respectively as `SYNTAX_ONLY_NOT_HOST_EXECUTED` or `HOST_ENGINE_TESTS=NOT_RUN`. These checks cannot prove that BMesh, modifier/dependency graph, CSG/B-rep booleans, SOP/VEX cook, engine collision/NavMesh, USD composition or the Replicator writer has been executed. Status and boundaries

(open materials: ../reproduction/programmatic_modeling/STATUS.md). Example index (open materials: ../reproduction/programmatic_modeling/EXAMPLES.md).

Integrated Interpretation

The quantitative table and the local reproduction together reveal an engineering fact that rendering metrics easily obscure. "Looking better in the view" and "the space can serve as a legitimate, stable, low-cost collision proxy" are not the same objective. The LPM table can support comparison at the PSNR/SSIM/LPIPS level, but it cannot automatically prove watertightness, stable topology or physical availability. The local pipeline, in turn, proves that rankings invert even on a simple height field. The most accurate open surface and the simplest watertight collision proxy come out in opposite order.

An evidence chain aimed at real spatial construction should therefore be divided into at least three layers. The first layer reports appearance/novel-view metrics. The second layer reports geometry, topology, scale and surface error. The third layer separately verifies collision, navigation, rigid-body stability and runtime cost. The step from descriptive synthesis to a strict random-effects meta-analysis should wait for one condition. The evidence must sit on the same task, the same dataset and the same protocol, and the same metric and usable variance must be available.

Auditable Entry Points

Data generation uses build_benchmarks_part_quant.py (open materials: ../analysis/build_benchmarks_part_quant.py). Quantitative analysis uses run_meta_correlation.py (open materials: ../analysis/run_meta_correlation.py). The quantitative method notes are results_part_quant/□ □ □ □ .md (open materials: ../analysis/results_part_quant/□ □ □ □ Geometry reproduction code is lightweight_geometry_pipeline.py (open materials: ../reproduction/lightweight_geometr Complete execution commands are in COMMANDS.md (open materials: ../reproduction/COMMANDS.md), and stage timing is in STAGE_SUMMARY.md (open materials: ../reproduction/STAGE_SUMMARY.md). The reproduction viewer is index.html (open materials: ../reproduction/index.html). Artifact hashes are in artifact_hashes.sha256 (open materials: ../reproduction/results_part_quant/artifact_hashes.sha256). Programmatic dual-mesh commands are in programmatic_modeling/run_smoke.sh (open materials: ../reproduction/programmatic_modeling/run_smoke.sh). Programmatic reproduction status is recorded in programmatic_modeling/STATUS.md (open materials: ../reproduction/programmatic_modeling/STATUS.md). The cross-tool static example index is programmatic_modeling/EXAMPLES.md (open materials: ../reproduction/programmatic_modeling/EXAMPLES.md).

Primary Sources for This Chapter

Yang H, Zhang C, Wang W, et al. Improving Gaussian Splatting with Localized Points Management. CVPR 2025, Table 1, PDF p.6. Kerbl B, Kopanas G, Leimkühler T, Drettakis G. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM TOG 42(4), 2023, Table 1, PDF p.8.

\newpage

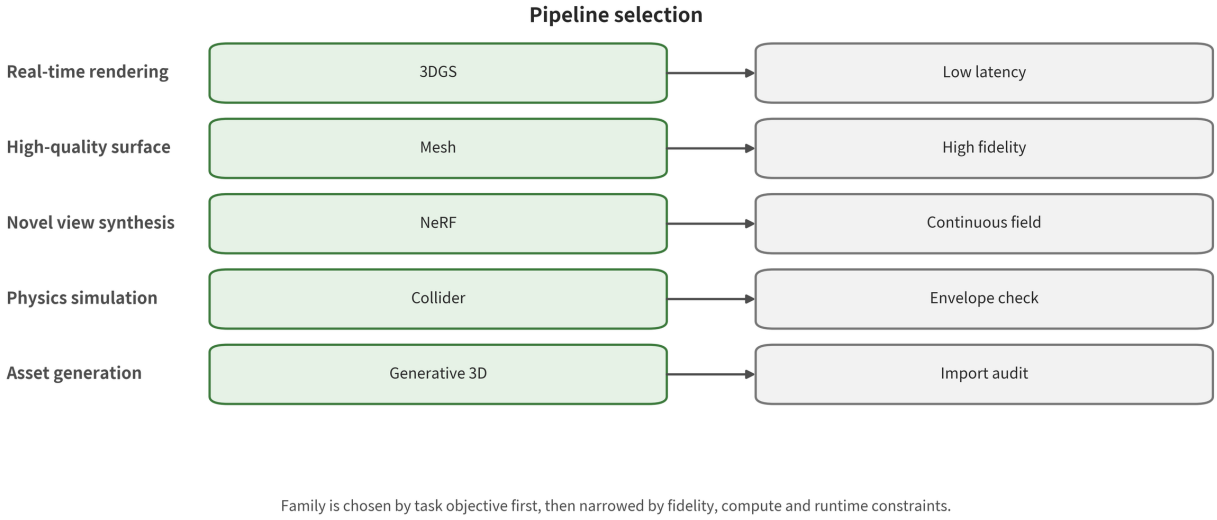
Applications and Deployment Mapping

Application mapping starts from the final query and works backward to the construction pipeline. Whether one needs to view, measure, edit, collide, navigate, plan or roll back determines the choice of a different primary representation and verification gate.

No single optimal pipeline applies to all inputs and applications. Selection should work backward from the query that ultimately needs to be answered: viewing images only, free viewing, precise measurement, editing surfaces, real-time collision, agent navigation or long-horizon action response. Six common construction tasks are given below.

Classification and Verification Figures

These figures derive from the earlier project's structured data or generation scripts, and their captions also give the readable range. The main text does not use comparison tables as a substitute for argument.



Classification figure that back-infers the construction pipeline from the final query contract. The selection order fixes the query that needs to be answered first. Only then does it choose the representation, assetization, collision and navigation, and engine runtime layers.

High-Fidelity Digitization of Real Scenes

Recommended pipeline: planned capture camera/color preprocessing SfM/SLAM poses MVS or feed-forward point maps/depth 3DGS/NeRF appearance layer + TSDF/neural SDF/mesh geometry layer mesh repair and materials independent collision and navmesh engine verification.

Why keep two representations? A 3DGS/NeRF layer usually fits complex appearance and view-dependent effects better near the training viewpoints. The geometry layer is better suited to scale, measurement, editing and collision. A project that only needs a virtual tour can put more budget on the appearance layer. A project for robots or digital twins is different. There, geometry and scale verification take priority over PSNR.

Key checks are the revisit loop, reprojection, depth and point-cloud error, uncovered regions, thin or transparent objects, and coordinates and scale. They also cover mesh self-intersections and holes, collision penetration, navigation connectivity, and peak engine resources.

Rapid World Generation from a Single Image or Text

Recommended pipeline: text/image layout or coarse 3D conditions multi-view/panoramic/3D world generation feed-forward geometry or scene representation object decomposition separation of the visual mesh/Gaussians from collision proxies manual semantic correction engine baking.

There is one fact that has to be accepted. Invisible content has no unique ground truth. A generative system should output uncertainty, version and provenance. It should not describe prior-based completion as reconstruction. In gameplay and art prototypes, plausibility matters more than observational fidelity. When the goal is to replicate real locations, generated regions and observed regions must be labeled in separate layers.

Services such as Marble should be used in one specific way. Their splats can serve as a high-fidelity visual layer, and their exported coarse collider as a prototype physics layer. Collision for key gameplay regions can then be rebuilt or manually corrected. The official documentation already states that high-quality visual

meshes can also fail at uncovered regions, reflections and thin structures. Asset inspection and game testing therefore cannot be skipped. Marble mesh export [@srcWL-MARBLE-MESH]

Interactive Video Worlds and Agent Training

Recommended pipeline: action-conditioned world model explicit state/memory interface pixel or 3D state rollout safety shell/action gating explicit maps or object tables for key states comparison with conventional physics/task logic agent evaluation.

Where is the application boundary? Pixel world models are suited to generating diverse, continuous visual experience and counterfactual scenes. A task that depends on precise distance, contact or reward, however, needs a verifiable state channel. One feasible hybrid scheme lets the generative model handle appearance and unseen content. An explicit simulator then handles collision, dynamics, task termination and safety constraints.

Key checks cover the replay of the same actions, branch consistency, revisit memory and object permanence. They also cover state leakage, long-horizon drift, action latency, closed-loop task success rate and safety constraint violations. Short-video preference scores cannot substitute for these metrics.

Games, XR, and Content Production

Recommended pipeline: generated/scanned assets object and semantic separation mesh repair/retopology UV/PBR materials/baking multi-level LOD/streaming simple/complex collision navmesh and interaction markers platform builds and frame budget testing.

How do the representations combine? Static backgrounds can use Gaussian or neural rendering plug-ins. Close-range interactive objects use conventional meshes, skeletons and materials. Vision and collision stay separate. Distant scenery, sky and non-interactive regions should not pay the cost of high-precision collision.

Key checks cover GPU, CPU and memory budgets for desktop, mobile and XR. They also cover transparent and reflective materials, lighting and shadows, and LOD popping. Then come collision layers and triggers, continuous collision and high-speed objects, and navigation agent size. Finally, asset loading and unloading, and failure fallback.

Robot Simulation and Digital Twins

Recommended pipeline: calibrated sensor capture reconstruction with scale semantics/objects and affordances contactable surfaces and physical parameters robot collision model sensor simulation task/safety constraints back-labeling with real data.

The priority here is different. Geometry, scale, reachability and sensor error rank above photorealistic appearance. Generated content can serve domain randomization and long-tail scenarios. But the project must make explicit which variables are synthetic, how the distribution is sampled, and whether task difficulty is changed.

Key checks cover corresponding real and simulated coordinates, joint and contact parameters, occlusion and sensor noise, trajectory executability, and collision margin. They also cover reset repeatability, simulator determinism and sim-to-real deviation.

Rule-Based Spaces, Modular Levels, and Batch Synthetic Data

Recommended pipeline: freeze the unit/coordinate/parameter schema construct geometry with OpenSCAD, CadQuery, FreeCAD, or Blender/Houdini nodes/scripts semantic IDs and material/instance rules generate the visual, LOD, and collision proxies separately, each from the intent idempotent import and layout via Unity/Unreal/Godot editor scripts NavMesh/sensor/annotation generation export read-back, query regression, and atomic release.

When should the procedural route win? When the target has repeated modules, explicit dimensions, assembly relations or manufacturability constraints, or when it needs large-scale controlled variation. In that case one should not generate images first and then reconstruct structure that is already known. Architectural rooms, roads, pipe networks, industrial parts, warehouse shelving, mazes, training obstacle courses and parametric cities can all encode design intent directly as rules. A real digital twin can use scans to estimate on-site deviation. Parametric CAD then solidifies walls, doors, equipment and editable structures. The generative model handles materials, furnishings or unobserved candidates, but it must not overwrite measurement residuals.

Key checks are these. The parameter range and its constraints must be satisfiable. The output must be deterministic once the seed and the toolchain are fixed. Booleans and B-rep must be valid, and tolerance must not swallow the smallest feature. Object IDs must remain stable after an insertion or a deletion. Visual, collision and navigation derivatives must share coordinates, yet each must be accepted independently. The hierarchy, units, transforms and materials must read back consistently from the exporter and from the target engine. Untrusted scripts, plug-ins and source files must run in a worker with no network access, no credentials and limited resources. When only Python AST or textual bracket checks are available, the status must be written as ”syntax/static check”. One must not claim that it has already run in a Blender, CAD or engine host.

Choosing the Primary Representation by the Final Query

Suppose the task is only film and television, or shot previsualization. The primary representation can then remain a video or an action-conditioned pixel world. The entry point is text, images and shot conditions. The physics proxy can be omitted or kept minimal. Acceptance focuses on condition adherence, cross-frame continuity and replayability of the same shot. The main risks are long-horizon drift and camera trajectories that cannot be reproduced. A virtual tour calls for a dual representation of 3DGS/NeRF appearance plus simplified geometry. The entry point is multiple images, video or panoramas, and coarse collision is built for the ground and the obstacles. Novel views, revisits, floaters, coordinates and uncovered regions form the minimum threshold.

When the result must support real measurement or a digital twin, the primary representation has to shift to scaled point clouds, TSDF, SDF or meshes. Parametric CAD can serve as design intent and editable structure. On-site deviation, though, must be verified with calibrated RGB-D, LiDAR or multi-view data with sufficient coverage. Collision proxies can be refined in layers. Scale, registration, thin surfaces, reflective surfaces, topology and version must still be verified item by item. Game prototypes and XR are better served by a hybrid of procedural meshes and Gaussian/neural appearance. Generation or scanning handles the content entry point. Blender/Houdini/engine scripts handle constrained assets and layout. Primitives, convex decomposition and NavMesh handle interaction. Acceptance must cover idempotent import, LOD, collision, navigation, device frame budgets, narrow passages and occlusion at the same time. It cannot look only at editor screenshots.

Robot training requires meshes/SDF with true scale, semantics and affordances. It must explicitly maintain robot and environment collision, contact and sensor models. Controlled generation can extend long-tail scenarios, but it cannot replace checks on contact, reachability, reset determinism and sim-to-real. Online streaming of large worlds turns the problem into spatial partitioning. Chunked meshes, hierarchical Gaussians or voxels handle vision and geometry. Partitioned collision and a tiled NavMesh handle local queries. Memory peaks, coordinate precision, world origin, LOD, dynamic re-baking and load-failure fallback must all be tested.

Decision Rules

If the output must support collision, require a surface/distance query contract first. When there is only video, novel views or Gaussians without topology, the plan must include a mesh/SDF/proxy construction step. If the output must be editable, require object, semantic, topology and material layering first. If the whole scene is baked into one indivisible large mesh, later editing costs move forward as technical debt. If the output must be measurable, require scale, coordinates and uncertainty first. Single-image generation and

purely appearance-based optimization cannot carry this responsibility alone. If the output must run in real time, budget rendering, physics, navigation and streaming as separate items. Rendering at 60 fps does not mean that collision or replanning is also within budget. When the output feeds safety-related decisions, refuse to rely on a demo alone. Data, protocol, failure samples, variance, adaptive attacks and independent reproduction must be explicit. If the structure is already known and batch variants are needed, prefer writing design intent as parameters and constraints. At the same time, lock the host/kernel/plug-in versions, the random seeds and the export contract. Validate generated code as an untrusted supply chain input.

October 2026 supplied adversarial context for L4/L5 deployment claims: a public mapping organised the humanoid attack surface into eight entry layers with real advisories per layer, and the month's embodied-agent incidents documented agents reaching live systems. The host environments that would run such systems are the same package and transport layers that spent October under attack — the LiteLLM compromise, the tensorlake worm, GhostAction's 346-repo wave, and dozens of harness/SDK CVEs.

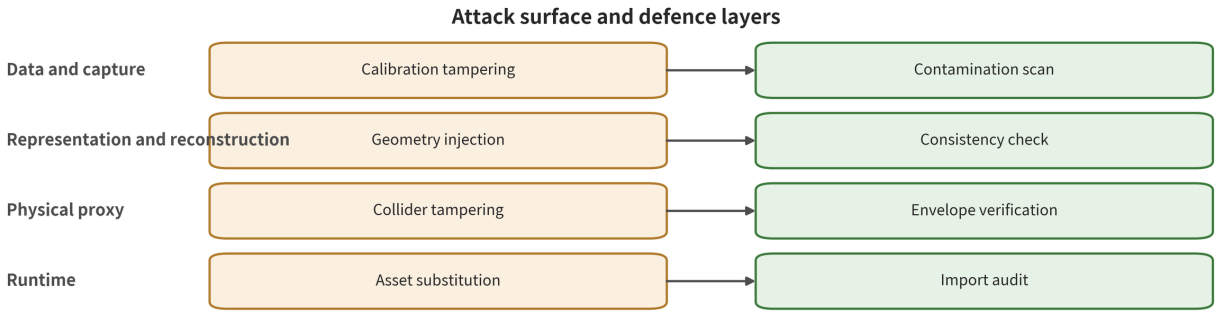
\newpage

Attacks, Defenses, Future Trends, and Limitations

The security problem is not isolated model robustness. It is whether errors can pass through interfaces and solidify into assets, physical proxies or runtime state. This section keeps attacks, defense in depth, the future agenda and evidence limitations inside one propagation chain.

Classification and Verification Figures

The figures below come from structured data or generation scripts of prior projects. Their captions also give the readable scope. The main text does not use comparison tables to replace argumentation.



Attack propagation and defense-in-depth gates for 3D spatial assets. Arrows indicate that errors can propagate from the observation and coordinate layers to representation, geometric assets, collision navigation, and runtime; defenses must be accepted separately at the interfaces.

Attack Methods: From Observation Poisoning to Geometric Assets, Collision Navigation, and Engine Hosts

A list of attack names cannot capture the security problem of 3D spatial construction. An attacker may only paste a piece of texture into a real-world scene. Or the attacker may upload a source image or model, control conversion parameters, or already sit inside the construction supply chain. These capabilities lead to completely different conclusions. An effective analysis must state several things at once. What does the attacker control? What does the attacker know? What does the attacker optimize? What budget constrains the

attacker? At which layer does the error first appear? How does it propagate? Which gate has independent observation? And can the attacker still succeed after knowing that gate?

Unified Threat Model and Cross-Layer Attack Objectives The entire construction pipeline is written as

$$z_0 \xrightarrow{f_1} z_1 \xrightarrow{f_2} \cdots \xrightarrow{f_K} z_K,$$

Here z_0 is the image/video/depth/point cloud and calibration. Later states may include pose, depth, NeRF/3DGS, a reference mesh, a collision proxy, a NavMesh and the engine package. The attacker selects a perturbation δ within the allowed set Δ to maximize the final task loss:

$$\max_{\delta \in \Delta} L_{\text{task}}(f_K \circ \cdots \circ f_1(z_0 + \delta)) - \lambda L_{\text{visible}}(z_0, z_0 + \delta) - \rho L_{\text{cost}}(\delta).$$

An attack that optimizes only the first-layer classification error is local. An attack that optimizes the final reachability difference, the missed-collision distance or the resource peak is an end-to-end spatial attack. Attacker knowledge falls into five levels. First, black-box queries. Second, possession of a same-family surrogate. Third, knowledge of weights and gradients. Fourth, knowledge of the verifier and its thresholds. Fifth, control of build nodes. The last level is no longer an adversarial example but a supply chain adversary. A security claim must be bound to one of these levels, and "effective against non-adaptive white-box samples" cannot be abbreviated as "secure".

Cross-layer propagation should be recorded as a Jacobian or as a discrete sensitivity. Some perturbations are averaged out at the next step; others are amplified through accumulation. A small depth error in one frame, for example, may be suppressed by multi-view fusion. A persistent same-direction pose deviation fuses a TSDF wall into a double-layer shell. A small doorway may disappear completely after convex decomposition. A NavMesh link changes only a few bytes, yet it can span an entire wall. The attack budget cannot use only the input L_p norm. It must also report semantic changes, physical realizability and downstream impact.

Input, Camera, and Pose Attacks: Corrupt the Coordinates First, Then Let Every Surface Inherit the Error Image and depth perturbations. For monocular depth or a generic NeRF with an L_∞ pixel budget, the common sign-gradient projection update is:

$$x^{t+1} = \Pi_{\|x - x_0\|_\infty \le \epsilon} \left(x^t + \alpha \text{sign}(\nabla_x L) \right).$$

If the target is a wrong surface at some spatial location, L should not be only the image classification loss. It should pass through depth back-projection, fusion and surface extraction. It should then take the gradient with respect to the depth, occupancy or collision difference in the target region. Printed patches, in turn, use expectation over transformation to sample over distance, pose, exposure, blur and print color gamut. The perturbation then stays effective under physical viewpoints. Existing evidence for digital monocular depth attacks, printed patches and 3D object texture attacks appears in [S011—S013] in that order. That evidence shows that the depth chain can be manipulated. It does not directly show that any SfM/MVS or collision system fails under the same budget.

A defense gate cannot look only at the pixel spectrum. It should compare several signals at once: stereo or active depth, cross-frame optical flow reprojection, contours and depth normals, and the multi-view consistency of static surfaces. It should also retain the raw sensor frames and the calibration hash. Once the attacker knows these gates, the detection loss should be added to the objective as well:

$$L_{\text{adaptive}} = L_{\text{task}} - \beta D_{\text{detector}} - \gamma D_{\text{crossmodal}}.$$

Here D_{detector} is the anomaly score, and $D_{\text{crossmodal}}$ is the cross-modal inconsistency score. The attacker suppresses both observable signals while amplifying the task error. If a defense is effective only when the attacker does not know the threshold, it should be labeled explicitly as a non-adaptive result.

Features, correspondences, and pose. Attacks on SfM/SLAM need not make pixels change noticeably. Stable wrong correspondences will do. So will control of camera metadata, repeated textures exploited until a false model holds a majority, or a physical patch that attracts features. RANSAC holds up only while the true inlier ratio and the sampling assumptions are met. If an attacker builds a set of geometrically self-consistent but semantically wrong correspondences, the wrong model may end up with more ”inliers”. AoR has already used a low-visibility road patch to amplify trajectory error in a visual SLAM setting, [@srcS014] and SLAMSpoof picks scan-matching-fragile locations and injects LiDAR points there. [@srcS016]

The propagation chain of a pose attack is: wrong matches $\rightarrow$ camera R,t$ deviation $\rightarrow$ MVS homography and depth cost volume misalignment $\rightarrow$ TSDF multi-layer shells/thin walls disappearing $\rightarrow$ mesh and collider offset $\rightarrow$ NavMesh connectivity change. Before fusion, defenses should check three things: cycle consistency in the match graph, cut edges in the view graph, and how reprojection residuals are distributed in space. The same check should cover IMU/GNSS/wheel-speed residuals, scale anchors, and the stability of re-integration after loop closure. After fusion, key wall surfaces must still be verified with independent ranging. These checks rest on correspondence and noise assumptions. The first-hand pipelines of COLMAP/MVSNet and the robust-estimation bounds of RANSAC/TEASER++ make those assumptions explicit. [@srcG003][@srcG004][@srcG012][@srcG013] Adaptive evaluation must go further and use structured, majority, within-threshold wrong correspondences, not merely random outliers.

Physical point removal. PRA exploits LiDAR signal and framework filtering pipelines to get real obstacle points deleted. [@srcS015] Its danger goes beyond a detector miss. When the mapping system treats ”no echo” as free space, the attack accumulates transient absences into persistent holes. Defenses must retain pre-filter echoes, distinguish unknown from free, and require cross-frame, multimodal evidence before clearing occupancy. A safe robot should slow down or stop on conflict. It should not let a generative prior automatically fill in a road that ”looks plausible”.

Point Clouds, NeRF, and 3DGS: Geometric Objectives and Resource Objectives Are Two Attack Lines
Point cloud manipulation and training backdoors. White-box point cloud classification attacks can move the origin, add scattered points or a small structural cluster, and iteratively change the label via ∇_{p_i}L$. Existing PointNet/ModelNet evidence shows that this digital input is fragile. source identifier S002 Training-set backdoors add a trigger point cluster to a small number of samples, so clean samples behave normally and triggered samples are classified to the target class. [@srcS003] If semantic labels directly determine ”walkable/non-walkable” or the collision layer, classification errors propagate to the physical layer. That path is conditional, though, because system design creates it, and it does not mean that the original paper has already attacked NavMesh. The correct defense is to let semantics serve only as candidates, with trusted geometry still determining physical occupancy.

SOR, radius filtering, and DUP-Net share one assumption: malicious points lie further out than the true surface. [@srcS004] IF-Defense takes another route and recovers the point set with an implicit shape prior. [@srcS005] Thin structures that really are rare, such as wires, leaves, and poles, may be deleted by both. An adaptive attack should pass through the filter via a differentiable approximation or BPDA while also constraining the recovered surface. Defense evaluation must move past classification accuracy and report bidirectional Chamfer/Hausdorff, normals, observation coverage, thin-structure recall, and missed collision faces as well. PointGuard's certificate protects classification labels only under a specified point-operation budget. source identifier S006 Nothing in it certifies pose, surface distance, or reachability.

NeRF source-view attacks. NeRFool modifies one or more source views of a generalizable NeRF, and that degrades the color/structure of multiple target views which were not provided. [@srcS007] Targeted work has also studied low-intensity perturbations and printable patches. [@srcS008] When surfaces are later extracted from density or an SDF, cross-view errors may become spurious surfaces. How far they propagate depends on the specific representation, the thresholds, and additional depth constraints. Defenses should randomly select subsets of source views and compare reconstructions from different subsets. They should cross-validate with real depth/contours and re-optimize for the case where the attacker knows the subset strategy

and the purifier. Adversarial training against same-family attacks alone cannot cover per-scene NeRF or other surface-extraction pipelines.

3DGS resource poisoning. Poison-splat's goal is not to quietly move a wall. It exploits the 3DGS densification mechanism to inflate the number of Gaussians, GPU memory, training time, and rendering latency. [@srcS009] It can be abstracted as a bilevel problem. The outer level selects a constrained input perturbation. The inner level performs normal training $\theta^*(x+\delta)$. The outer level maximizes the resource function

$$\max_{\|\delta\|\le\epsilon} R(\theta^*) = \alpha N_{\text{gauss}} + \beta M_{\text{peak}} + \gamma T_{\text{train}}, \quad \theta^* = \arg\min_{\theta} L_{\text{render}}(\theta; x+\delta).$$

The attack uses gradients/errors to trigger splitting and cloning, so training keeps optimizing rendering while the representation keeps expanding. The detection signal is not the final PSNR. Watch the number of Gaussians per iteration instead, along with the number of densifications, the GPU-memory and wall-clock slopes, and the mismatch between quality improvement and resource growth. RemedyGS uses detection and purification and reports white-box, black-box, and defense-aware adaptive attacks, [@srcS010] and that is relatively complete defense evidence. A purifier still cannot replace hard quotas on Gaussian count, GPU memory, iterations, and wall clock enforced before OOM. Tenant isolation and atomic failure are system guarantees. They do not depend on the attack distribution.

Geometrically targeted poisoning and resource DoS must be separated. One sample may increase the number of Gaussians without changing collisions. Another may move a specific surface while resources remain normal. Future evaluation should adopt a dual objective. Where the change in visual metrics stays below a threshold and resources do not trip the circuit breaker, maximize the surface or collision difference in the target region. Otherwise, ”defending against resource attacks” cannot imply ”geometric safety”.

Malicious Meshes and Asset Parsers: Geometry Files Are Both Complex Data and an Executable Trust Boundary Malicious meshes come in two classes. The first class is valid in format yet exploits geometric semantics. Nudging the vertices slightly makes a recognizer misclassify after differentiable rendering, and MeshAdv demonstrated this cross-view persistent carrier. [@srcS001] Further engineering attacks can add extremely thin long triangles, duplicated overlapping shells, flipped normals, NaN/Inf, extreme coordinates, massive numbers of connected components, or hidden internal geometry. Such a file is visually almost unchanged, yet it makes BVH, convex decomposition, SDF cooking, and NavMesh baking time out or produce incorrect inside/outside. The attack objective can be written as

$$\max_{M'} D_{\text{physics}}(B(M'), B(M)) \quad \text{s.t.} \quad D_{\text{render}}(M', M) \le \epsilon,$$

where B is the asset builder. A truly adaptive sample must know the repair step, the simplification step, and their thresholds. Its attack effect must survive all of them.

The second class directly attacks the parser or the host. glTF's buffer/accessor/extension/URI widens the attack surface, and so do USD's resolver/plugin, FBX/SketchUp/MDL, and DCC source files. The Unity 2023 security advisory confirms the memory-corruption/RCE risk for affected editors that import FBX/SketchUp. source identifier S017 Assimp's specific HL1 MDL loader heap overflow shows that unified format libraries can fail at length/index parsing too. [@srcS019] Blender officially states that `.blend` can contain Python, drivers, and other scripting capabilities, so automatic execution must be treated as a trust decision. [@srcS020] The Unity 2025 host-loading vulnerability further shows that clearing geometry schema validation cannot replace engine patches and application rebuilds. [@srcS018]

The parsing defense line should count in streaming fashion and check for integer overflow before allocating large buffers. It should also cap file bytes, decompression ratio, nodes, vertices, indices, texture pixels, bones, animations, extensions, recursion depth, and external URIs. The parsing process should use low privileges, read-only input, temporary output, no network, and no credentials, and it should run under CPU/memory/wall-clock limits. It should also disable formats, resolvers, plugins, and scripts that are not needed. Specification conformance is something glTF Validator can check. [@srcE002] Resource quotas,

image-decoding safety, and the sandbox stay outside its reach. Security testing requires format-aware fuzzing, complexity fuzzing, ASan/UBSan corpus regression, and a fixed-version inventory. Uploading a few corrupted files to see whether they crash will not do.

Collision Surfaces and NavMesh Manipulation: Minimal Geometric Changes Can Cause Maximal Reachability Differences An attacker who can modify colliders, collision masks, agent profiles, area costs, off-mesh links, or tile data does not need to break the reconstruction model. Deleting an invisible collider can let a character cross the boundary. Adding a thin box can form an invisible wall. Changing a certain layer from block to ignore can make rays and rigid bodies see different worlds. Shrinking the agent radius by one step can open a passage that was originally closed. Adding an ultra-low cost link can make paths pass through walls. Deleting a tile border link can split a large region apart. Mature peer-reviewed benchmarks for such active attacks remain scarce. The current evidence should therefore be labeled an engineering threat model. It should not be cited as an unverified "success rate." [C004—C007][@srcV005]

Navigation attacks can be formalized as

$$\max_{\delta \in \Delta} D_R(R(N), R(N \oplus \delta)) - \lambda \|\delta\|_0,$$

Here N is the construction input and parameters. $R(N)$ is the reachability matrix or shortest-path distribution of a chosen set of start and end points. δ may be a small number of triangle, link, area, or parameter modifications. The attacker wants the fewest modifications that maximize the connectivity difference, and holds the visual mesh unchanged. Against collision, the attacker can instead maximize missed hits or false hits across a set of sweeps, keeping the rendering distance below a threshold.

Defense gates must deterministically rebuild the proxy from trusted reference geometry. They must lock down the construction parameter policy and the collision matrix, and a second implementation must establish a free-space reference. The test set should include positive queries and negative queries alike. Walls that must block, doors that must be passable, upper and lower floors that must not be erroneously connected, and critical platforms that must not be lost all belong in it. High-speed sweeps, different agent clearances, tile seams, dynamic door opening and closing, and resource caps belong there too. An adaptive red team re-solves the above objectives once it holds all the thresholds. If the verifier only samples fixed paths, the attack will avoid the samples. Random seed rotation, coverage-driven exploration, and exhaustive small regions for critical tasks are therefore needed.

Engine Runtime and Supply Chain: Signatures Guarantee "Who Built It," Not That It Was "Built Correctly" Replace a build node, a plug-in, model weights, an importer, or a release image, and the attacker can generate a malicious world. It is well-formed and passes tests selectively. C2PA can bind media assertions, signatures, and hashes, and suits tracing the input image editing chain. [@srcS021] SLSA provenance describes how the source, the builder, and artifact generation relate. [@srcS022] NIST SSDF provides organization-level secure development practices. Source identifier S023 All three address provenance and build-chain integrity. None proves that the content is authentic, the license valid, the geometry correct, or the collision reasonable. None proves that the signer is trustworthy. A legitimate signing key in the attacker's hands can still sign a malicious collider.

A supply chain gate should require several things at once: pinned dependency hashes and an SBOM, isolated reproducible builds, two-person approval of critical parameters, artifact signing, and mandatory verification on the deployment side. At the content layer it should add independent geometry/physics/navigation tests. Runtime needs least privilege, network and file access control, crash isolation, and safe fallback. When a quota is exceeded or the state generation is inconsistent, the last verified cell should be retained or the problematic region disabled. A half-parsed resource must never be registered into the global scene and then left running.

Procedural Modeling Attacks: Correct Code Execution Can Still Build Wrong Worlds in Bulk Once code drives Blender, CAD, node graphs, and engine editors, the attack surface shifts. It moves forward from

the ”malicious mesh” to the executable build specification. The first category is host code execution. Blender registering text blocks/drivers, plug-in Python, Unity import callbacks and C#, Unreal `init_unreal.py`/Editor Utility, and Godot `@tool` and Omniverse extension all qualify. Any of them may access files, the network, and the project under the editor account's privileges. Blender's script security documentation states directly that Python does not restrict script capabilities and that automatic execution is a trust decision. [@srcS020] Unity's official 2023 advisory once confirmed that specially crafted FBX/SketchUp caused memory corruption and possibly code execution in affected editor versions. [@srcS017-EA-S001] That evidence covers only the historical versions/formats listed in the advisory. It must not be written as Unity 6 still having the same vulnerability today. Other host chains should be labeled an engineering threat model in the absence of a specific exploit paper, without giving fabricated incidence rates.

The second category is dependency, startup-context, and cache pollution. Loose plug-in versions, user-level Python paths, writable package caches, unknown startup files, the current editor selection, and the active scene all play a part. The same script can end up loading different modules or acting on the wrong object. An attacker does not necessarily need an obviously malicious statement. A plug-in with the same name on a higher-priority search path can change what loads. A changed Geometry Nodes/HDA reference can do the same, as can a modified Unity importer version or a modified Unreal startup object. The build output may drift consistently as a result. Several signals mark it. The dependency closure changes, scripts execute without being declared, and a large portion of the project turns dirty. The object count jumps, and the canonical geometry signature of the same BuildSpec changes.

The third category is path and reference escape. Scripts that output `../`, absolute paths, or symbolic links can resolve outside the approved root after a string-prefix check. So can USD `resolver/reference/payload`, texture URIs, nested archives, and Replicator writer file names. The attack may aim to read credentials or overwrite the project. It may write training labels into a different generation, or mix network resources into an offline package. Validation must judge the canonical path as it actually resolves, along with symlinks and mount boundaries. Rejecting `..` in text is not enough.

Geometry and combinatorial complexity bombs form the fourth category. A single item may look compliant on its own count. Yet the product of array × subdivision × boolean × instance × USD reference × frame count can exhaust CPU, GPU, memory and disk. The victims are the dependency graph, the B-rep kernel, convex decomposition, NavMesh, rendering or the writer. The more covert samples hide their blow-up elsewhere. They use near-coplanar booleans, extremely small slivers, deep node graphs, circular dependencies, massive component counts, or a high texture decompression ratio. Parameters that ”generate a few dozen objects” then yield hundreds of millions of triangles once evaluation finishes. Defense must estimate the joint budget before that expensive evaluation, and set growth rates and hard aborts during the run. Checking only the final file bytes is already too late.

Determinism and semantic attacks form the fifth category. Delete one upstream instance and the global random sequence shifts backward, which can change every object. Modify the unit or the up axis and collision, NavMesh, cameras and depth break together. Reorder objects and the array-index-based instance IDs and labels drift. Replicator's asynchronous step can even mismatch RGB against the previous frame's annotations. None of this has to make the build fail. The assets may be well-formed, visually plausible, and consistently passing erroneous tests. Attack goals should therefore be written directly as differences in size, occupancy, reachability matrix, label projection, or cross-generation object identity. Looking only at the script exit code is not enough.

Adaptive evaluation of procedural attacks should hand the attacker everything: the public schema, the allowlisted API, path roots, budgets, seeds and geometry gates. It then searches for the minimal program or parameter patch that satisfies the syntactic and visual constraints, does not trigger the gates, and still maximizes physical, navigation, resource or label differences. After success, delta debugging should shrink it to the fewest nodes, instances, paths or parameters and fix it as regression corpus. Only then can one test whether the defense line truly limits capability. Blocking the obvious `os.system` string is not that test.

The Real Through-Line of Attacks Is Failure Propagation, Not Paper Names All attacks can be grouped into six propagation mechanisms. First comes coordinate pollution, which shifts all subsequent surfaces

together from calibration or pose. Second comes observation pollution, which makes depth, points or radiance fields create spurious geometry locally. Third comes representation inflation, which exhausts resources through densification, triangles, instances, references or texture counts. Fourth comes contract substitution, which changes inside/outside, clearance or connectivity at the mesh collider NavMesh conversion points. Fifth comes execution-boundary breakout, which uses parsers, modeling scripts, node graphs, plugins or the supply chain to obtain host privileges. Sixth comes reproduction-identity pollution, which changes the evaluation result of the same specification through versions, dependencies, seeds, context and asynchronous frames. Defenses must likewise be placed before the corresponding propagation edge. Pose errors are caught before fusion. Malicious combinations are caught before expensive evaluation. Collision differences are caught before release. Script capabilities are restricted before the host starts. Supply chain substitutions are caught at deployment signature verification. The next chapter no longer lists papers one by one. It follows these mechanisms instead, to explain why methods move from single-point robustness toward multiple contracts, independent validation, and an adaptive closed loop.

Evolution of defense approaches and methods: from ”cleaning the input” to ”certifiable world contracts”

The evolution of 3D construction techniques is not a monotonic rise in rendering metrics. Each generation of methods adds one kind of query capability, and also introduces a new unverifiable state. Image generation adds view-dependent appearance but lacks persistent geometry. Reconstruction adds cameras and surfaces but lacks physical inside/outside. 3DGS adds explicit differentiable primitives and real-time rendering but lacks a unique topology. Meshes add editing and interchange but lack collision semantics. Convex pieces/SDF add contact queries but still cannot decide whether a given agent is reachable. NavMesh adds a task-feasible region but depends on engine state, dynamic updates and the supply chain. Defense methods therefore gradually evolve from input sanitization into cross-layer output contracts and independent verification.

Three logics of method evolution: query capability, determinism boundaries, and state persistence
The first logic runs from appearance similarity to answerable spatial queries. Pixel video can answer only ”what is seen from this viewpoint”. NeRF/3DGS can answer ”given a camera, how colors are synthesized”, but density, Gaussians and surfaces still require additional interpretation. Point clouds/TSDF/neural SDF add depth, occupancy, or a zero level set. Oriented meshes add local surfaces and topology. Convex pieces, triangle BVHs and SDF add overlap/sweep/contact. NavMesh adds connectivity and cost for a specific agent. The engine scene graph further adds object identity, dynamic state and streaming lifecycle. Each upgrade must declare the new queries and their error units. It must not pass off the previous layer's metrics as the next layer's guarantees.

The second logic runs from a shared representation to separation of responsibilities. Early prototypes often used the same mesh for rendering, collision and navigation at the same time. That is simple, but every simplification or hole-filling then changes all three systems. Production routes gradually split into reference geometry, visual assets, physics proxies, and agent-specific NavMesh. Hashes and generation link them together. Separation of responsibilities is not redundant storage. It makes the objective functions interpretable. The visual surface may allow material approximation. The reference surface preserves observational evidence. The collision surface controls false/missed occupancy, and NavMesh controls clearance and connectivity. One low-privilege visual file is also a harder route by which an attacker can directly change the physical world.

The third logic runs from one-time construction to persistent, transactional state. An offline scene only needs to be loaded once. A dynamic world has to handle doors, destruction, streaming, relocation and multiplayer synchronization. ”Rebaking and then replacing the file” is the simple approach, and it produces mixed snapshots of old and new collider/NavMesh. The evolution direction is stable object IDs, chunked generation, incremental BVH/refit, tile cache, two-phase commit, and rollback-capable artifacts. A world model that is to enter games or robotics must generate the next frame, but that is not all it must do. It must also output object and geometry state that is branchable, revisitable, queryable, and capable of atomic updates.

Together these three logics explain why one single representation will not swallow all layers in the future.

Gaussians suit appearance, meshes suit editing, SDF suits distance, convex pieces suit dynamic solving, NavMesh suits paths, and scene graphs suit identity and state. Hybrid representations are more feasible. They share coordinates, provenance and uncertainty, and clear conversion boundaries organize them into a system.

Six defense gates: each gate intercepts one kind of propagation rather than rerunning the same detector Gate one: capture and calibration. Raw frames, depth, IMU/LiDAR, timestamps and calibration files enter an append-only capture package and are hashed item by item. Check frame rate, exposure, time synchronization, camera intrinsic drift and multimodal residuals. When a texture patch or laser attack causes a modality conflict, label the region as unknown. Do not write missing measurements directly as free. The output of the gate is "observations with provenance and uncertainty", not cleaned data pretending to be clean.

Gate two: estimation and fusion. SfM/MVS/SLAM check match loops, reprojection, view-graph connectivity, scale anchors and loop closure. The TSDF/neural state keeps per-voxel observation counts, weights and observed/inferred labels. 3DGS monitors the Gaussian count, densification, VRAM, time and quality gain at each iteration. When the resource slope is exceeded, abort immediately at a safe boundary instead of waiting for OOM. Reconstruct suspicious views separately, and compare the geometry between subsets rather than only the final render.

Gate three: reference geometry and mesh. Validate finiteness, AABB, indices, duplicate/degenerate faces, boundaries, non-manifoldness, self-intersections, normals, connected components, volume and minimum thickness. Perform sensitivity sweeps over the surface-extraction threshold, voxel resolution and Poisson/hole-filling settings. Only regions that are stable under different settings and supported by observations may enter collision automatically. Completed and capped surfaces must be labeled and allow rollback. Mesh repair outputs a difference report and must not silently overwrite the input.

Gate four: physics proxies. Accept primitives, convex decompositions, static triangles and SDFs separately. Compare the signed distance between proxy and reference surface, and measure the clearance of key holes/slots. Run ray/overlap/sweep, initial overlap, corners, thin walls, high-speed CCD, stacking and different time steps. Record the counts of convex pieces, BVH nodes, SDF voxels and candidate pairs, plus narrow-phase time. All collision layer/mask and query/simulation/trigger flags enter a policy lock. Configuration differences are reviewed as strictly as geometry differences.

Gate five: navigation and tasks. Each agent profile is built and versioned independently. Check components, clearance, upper and lower layers, shortest paths and tile seams against a second-implementation free-space or high-resolution local reference. Dynamic doors, elevators, links, area cost and obstacle removal all run state sequences. The release gate asks not only "is there a path". It also asks "does a path appear where there should be none, does a path cross reference obstacles, and does the narrowest point satisfy body and error margins".

Gate six: runtime and supply chain. All untrusted formats are parsed in sandbox workers, with pinned versions/hashes/SBOM, network and scripts disabled, and resources limited. Build results carry provenance and signatures. At runtime, render/collider/nav are loaded atomically by generation, and query logs can be traced back to shape/tile/source. If verification fails, keep the last trusted world or enter safe states such as slowdown, stop, or area lockdown. Signature verification and content verification run in parallel. Passing either one cannot substitute for the other.

The key to the six gates is mutual independence of their inputs. Suppose one attacked mesh generates the collider, the NavMesh and the "reference answer" at once. All three may then agree perfectly, and still be wrong together. References may come from raw depth, a second reconstructor, measurement anchors, different voxel resolutions, or manually labeled critical regions. Independence must be written into the evaluation protocol.

Adaptive evaluation: let the attacker see the entire defense line, then search for the minimal modification Many defenses work on fixed attack samples, because the attack was not re-optimized against

sanitization, thresholds, or randomization. Deployable evaluation should adopt the closed loop of ”build--verify--attack--rebuild”:

The text workflow or pseudocode in the original manuscript

```
adaptive_red_team(clean_case, pipeline, gates, budgets):
baseline = pipeline.build(clean_case)
assert gates.accept(baseline)
attacker.learn(pipeline.public_config, gates.thresholds)
population = initialize_allowed_edits(clean_case, budgets)
repeat until wallclock_budget:
candidate = attacker.optimize(
maximize = task_difference(candidate, baseline)
+ resource_cost(candidate),
subject_to = input_budget
+ visual_similarity
+ gates.accept(candidate)
)
result = pipeline.build_in_isolation(candidate)
record(all_intermediate_states, result, gate_signals)
attacker.update(result)
minimize_successful_edit()
replay_on_other_engines_seeds_and_resolutions()
```

Attack objectives should correspond to layers. The input layer measures pose/depth/surface deviation. The resource layer measures peak VRAM and wall-clock. The collision layer measures false hits/misses and time of impact. The navigation layer measures the reachability matrix/shortest path/clearance. The parsing layer measures crashes/RCE/resource allocation. The supply chain tests whether unauthorized artifacts are deployed. Successful samples should also undergo delta debugging to find the minimal vertex, parameter, link or file-field modification. Minimal counterexamples are easier to fix and regress.

Randomized defenses must disclose their seeding policy and report confidence intervals. They must not pick a random subview only once. Non-differentiable filters must be tested with BPDA, EOT, or gradient-free search to avoid gradient masking. Threshold gates must test slow attacks near the threshold rather than looking only at obvious anomalies. Multimodal defenses must let the attacker optimize the controllable modalities jointly, and must state which modalities are still trusted. RemedyGS explicitly includes defense-aware adaptive attack, and [@srcS010] provides a relatively good paradigm in this field. Point-cloud sanitization, SLAM patches and navigation validation still generally lack closed loops of the same strength.

Evaluation can no longer be compressed into ”a single 3D total score” An evolved system has at least six ledgers. The visual ledger uses PSNR/SSIM/LPIPS, temporal consistency and novel views. The geometry ledger uses depth, Chamfer/Hausdorff, normals, completeness, topology and scale. The physics ledger uses false/missed collisions, penetration depth, missed CCD, contact stability and query latency. The navigation ledger uses components, reachability, path length, clearance and dynamic updates. The resource ledger uses Gaussian/triangle/convex piece/tile counts, peak memory and wall-clock. The security ledger uses attack budget, success endpoint, detection/rejection, adaptive status and worst-case loss. Different units cannot be summed into a single ”overall construction capability”.

When comparing two approaches, one can first define the hard contract that must be met, and then optimize cost within the contract. For example: zero missed collisions on critical walls, door clearance error below 2 cm, all task start and goal points reachable, and peak physics step below budget. Only after these are satisfied does one compare visual quality or triangle counts. A fast but wall-clipping proxy then cannot win the total score on FPS. A gorgeous but unreachable world cannot hide its failure behind PSNR.

Defenses should also report their costs. SOR/implicit recovery may lose thin structures. [@srcS004][@srcS005] Finer SDF increases memory. [@srcC004] Smaller Recast cells increase build cost and may create resource risk. [@srcV004] More convex pieces improve accuracy but increase contact pairs. Each gate should report

its rejection rate, wrongly deleted real geometry, latency, memory and manual review load, so that ”reject everything” is not treated as a security success.

Method directions for the next stage: hybrid worlds, certifiable error, and cross-engine differencing Hybrid world representation. The most promising production structure does not force Gaussians into a single mesh. It instead lets appearance Gaussians/radiance fields, occupancy or SDF with observational evidence, editable meshes, collision proxies and object scene graphs coexist. The shared layer provides entity_id + world_transform + time + provenance + uncertainty . Each query layer keeps its own error bounds. Visual completion may appear immediately. Before it enters collision, though, it must have geometric evidence or a conservative policy.

Contract-oriented learning. Today's training losses favor pixels, depth or Chamfer. The next step should fold in thin-wall recall, door-opening clearance, free-space consistency, SDF sign, contact distance and reachability directly. It should also stop treating the engine builder as an unauditable black box. Differentiable approximations may propose candidates, but final acceptance still comes from independent discrete geometry and runtime queries. Learner output confidence cannot self-certify. It should be calibrated across sensors, thresholds and implementations.

From label certificates to spatial certificates. PointGuard-style results protect classification labels under a given point budget source identifier S006 . Spatial systems need different certificates: pose error that stays within bounds, a surface Hausdorff distance in critical regions that does not exceed ϵ, occupancy that does not miss obstacles larger than a certain scale, and the path of a given agent that does not cross trusted obstacles. Proving the entire open world outright is very difficult. A first step is to give interval or conservative occupancy guarantees for local safety corridors, robot stopping zones and mission-critical door openings.

Transactional persistent worlds. World models and online reconstruction should write new observations into a candidate generation. They first complete incremental validation of geometry, collision and navigation, and only then commit to the running world. Object state, tile links and physics actors use stable IDs and event logs, so that one can replay why a given update changed a path. In this way, ”continuous generation” does not equal ”continuously putting unverified content into the physical world”.

Cross-engine differential benchmark. The same normalized glTF/USD/mesh assets and malicious variants should be compared across Unity, Unreal, Godot and direct PhysX integrations. That comparison should cover coordinates, triangle winding, convexification, SDF, collision mask, NavMesh and resource behavior. Differential comparison does not require byte-identical output. It requires that a set of world queries agree within an allowed tolerance. Parse crashes, excessive memory, invisible walls and cross-layer shortcuts should all enter the corpus and be regressed automatically after engine upgrades.

End-to-end adaptive red teaming. Future attacks will not stop at classification ASR. A more meaningful objective is this. The source image is almost unchanged, render quality passes, the resource quota is not exceeded and the geometry gate does not raise an alarm. Under those conditions, the attacker's goal is to make a task path pass through a wall, or to make a high-speed sweep go undetected. Defense research, in turn, must state which independent piece of evidence blocked this propagation and must release the failure samples. Standardized data that directly manipulates collider/NavMesh remains an obvious gap. The independent geometric verification represented by [@srcV005] can serve as a starting point, but it requires cross-engine comparison, third-party reproduction and attackers who know the verifier.

Dedicated Defenses for Procedural Construction: Capability Limits, Determinism, and Differential Queries For procedural modeling, the preferred security interface is a declarative, capability-limited specification, not arbitrary Python/C# handed directly to a high-privilege editor. A parameter schema or a small DSL allows only primitives, controlled Booleans, instances, material references and limited scene operations. The validator first checks units, numeric ranges, reference roots and the joint budget. A trusted adapter then translates the specification into `bpy`, CAD, Unity or Unreal APIs. If a task genuinely requires general-purpose code, the execution environment must be newly created for that task, with the network disabled and no user credentials or signing keys. Its inputs are read-only, and outputs may be written only to a staging root. The operating system must impose CPU, memory, GPU, file-count, byte and wall-clock limits. Dependencies,

plugins, startup scripts and host versions are allowlisted item by item. Blender disables automatic execution by default, and Unity/Unreal/Godot use clean projects and explicit entry points.

The path gate resolves canonical paths and symlinks before it actually opens them, and confirms that they lie within an allowed mount. USD references/payloads/sublayers, texture URIs, archives and writer outputs are all bounded by recursion depth, expanded bytes and target protocol. The resource gate must not limit only `N_faces`. It must estimate an upper bound on the product of arrays, instances, subdivision, Booleans, node iterations, references and frame count. During evaluation it must also monitor objects/triangles/prims, memory and the output growth rate continuously. Reaching any hard bound terminates the entire generation, rather than leaving half-imported objects in the main project.

The determinism gate fixes the schema, toolchain, kernel, plugins, dependency closure, random algorithm and seed-derivation rules. The random seed should be derived from global_seed + stable_entity_id + cell_id + operation_id . It should not consume a global sequence that shifts as a whole when an object is inserted. Scripts must not depend on the current selection, the active object, the open scene, user preferences or undeclared environment variables. After host evaluation, record the counts and hashes of the base mesh and the evaluated mesh. GPU/physics outputs for which full byte-level consistency is unrealistic should have field tolerances defined in advance. Object identity, labels, units, collision policy and manifests must not use fuzzy tolerances to conceal drift.

The differential gate is defined by queries, not by file format. Visual assets compare bounding boxes, bidirectional surface distance, normals, UVs/materials and semantic IDs. Collision assets compare pre-registered ray/overlap/sweep queries, door-opening clearance and conservative occupancy. NavMesh compares connectivity, path matrices and shortest paths for each agent. Synthetic data compares per-frame camera projection, mask/box/depth and the same scene state. After export, a second parser or the target engine must read back from an empty process, so that the generator and the validator do not share the same error. The validation, signing and release control plane is separated from user scripts. Only when everything passes is the result promoted atomically. On failure, the previous generation is retained.

The final red team should know all of the above policies and attempt to construct minimal counterexamples within the allowlisted APIs, path roots, budgets and geometric thresholds. The defense report simultaneously lists the clean-build pass rate, false rejections, resource overhead, output differences, worst-case query loss and adaptive state. The statement ”the host was not run, only AST/bracket checks passed” must remain in the results. It must not be packaged as an end-to-end defense success.

Evolution Conclusion: A Genuine 3D Construction Method Is a Set of Traceable Conversions From Chapter 10 to this chapter, the evolution of methods compresses into a single causal chain. Visual surfaces cannot reliably answer questions about contact, so primitives, convex bodies, triangle BVHs and SDFs arise. A single convex hull destroys concave grooves, so the voxel approximation of V-HACD evolves into the collision-aware concavity and multi-step cutting of CoACD. Discrete contact tunnels, so continuous time-of-impact is added. Collision space cannot directly answer whether an agent is reachable, so Recast voxelizes, filters, erodes, partitions and polygonizes it. Static assets cannot sustain a dynamic world, so engines introduce scene graphs, streaming cells, generation and transactional commits. Each conversion in turn becomes an attack surface, so defense evolves from input purification into six independent contract gates and an adaptive closed loop.

A qualified method therefore delivers a minimum set of artifacts, not a walkthrough video or a single mesh. Observations carry provenance, or the method ships an executable design specification. Reference geometry is auditable. Visual, collision and navigation artifacts each state an explicit intended use. Coordinate and agent contracts are locked, and manifests record parameters, seeds, the toolchain and dependencies. Engine packages can be rolled back. Error and resource reports are provided per layer. Failure records are collected once the attacker knows the verifier. Only when these interfaces hold continuously do the front-end video models, world models, reconstruction, 3DGS, procedural DCC/CAD, point cloud and mesh methods truly complete the leap from ”generating 3D content” to ”constructing an interactive space.”

Future trends: from ”can generate” to ”maintainable, verifiable, branchable”

World models are exploring turning persistent 3D state into an explicit interface Short-context pixel histories forget space during revisits, occlusion and long-horizon interaction. Methods such as PERSIST put the environment, the camera and the renderer into a latent 3D state, which indicates that future world models may consist of three parts: a neural model that generates appearance, a persistent state that maintains objects/geometry, and a simulation layer responsible for actions and constraints. What PERSIST [@srcPERSIST-2026] really needs to validate is not whether the model occasionally remembers a frame from a minute ago. It is whether the same object preserves identity, geometry and causal relations under different paths, different times and different action branches.

Generation and reconstruction may form a closed loop rather than a one-way pipeline The common practice today is to generate a video first and then reconstruct 3D from the video. A more effective system in the future would read geometric uncertainty during generation, actively choose the next viewpoint or generate supplementary views, and then feed reconstruction residuals back to the generative model. What warrants caution is that self-generated views are not independent evidence. If the same model repeats the same hallucination across multiple viewpoints, multi-view consistency does not equal reality. A reliable closed loop must retain real observation anchors and uncertainty propagation.

Hybrid representations are more likely to become the common production solution Meshes, SDFs, 3DGS, neural materials and latent states are each good at different queries. Future systems are more likely to divide the work as follows.

Meshes take on topology, rigging, collision and editing. Gaussians/neural textures take on complex appearance, reflections and detail. SDFs/voxels take on distance, contact and spatial occupancy. Object/scene graphs take on semantics, permissions and persistent state. World models take on unobserved content, dynamics and counterfactual generation.

The research question will shift. It will no longer be ”which representation replaces all representations,” but ”how to synchronize multiple representations under shared coordinates, LOD and an error budget.”

Feed-forward geometric foundation models are driving rapid capture, but optimization will not disappear entirely VGGT-style models put camera, depth, point maps and trajectories into a single forward inference pass. For VGGT [@srcVGGT-2025], a more realistic future form may look like this. ”Feed-forward inference provides a strong initialization and confidence, while lightweight geometric optimization handles the closed loop and refinement.” That form would not eliminate bundle adjustment or explicit constraints entirely.

”Assetization” is becoming an independent research goal Evaluation will expand from plain Chamfer/PSNR. The added metrics are topology usability rate, UV/material completeness, rig/joint plausibility, LOD fidelity, collision proxy error, navigation reachability, import success rate, runtime budget and manual repair time. WorldGen takes ”traversable and interactive” as its goal, and Marble directly provides different export layers for splats, visual meshes and collision meshes. Together they indicate that research and products are making the last mile explicit. WorldGen [@srcWORLDGEN-2026] Marble export specification [@srcWL-MARBLE-EXPORT-M04]

Physical consistency must be upgraded from ”looks like physics” to a verifiable state Video models can learn the visual patterns of gravity, fluids and collision, but closed-loop agent training requires readable state, action consequences and failure conditions. The next generation of benchmarks should publish video, geometry, contact/constraint state and task events at the same time. Then researchers can distinguish ”it looks like a collision happened” from ”the solver really satisfies the contact and energy constraints.”

The security scope is expanding from model robustness to the 3D asset supply chain 3DGS already has peer-reviewed evidence of computational-cost poisoning, Poison-splat [@srcS009], as well as a 2025 preprint on target-viewpoint-directed poisoning, GaussTrap [@srcGAUSSTRAP-2025]. Point clouds and meshes also have adversarial perturbations. On the other hand, production systems must also handle extremely large vertex/Gaussian counts, NaN/Inf, degenerate faces, self-intersections, malicious textures/shaders, scripts, path references and decompression bombs. In the future, 3D assets need provenance, hashes, signatures, format sandboxes, resource quotas and per-layer verification gates established for them, just as for the software supply chain.

Open formats and cross-engine validation will affect ecosystem usability Formats such as glTF/GLB, OpenUSD and PLY/SPZ need to carry coordinates, units, materials, LOD, semantics and possibly physics extensions. Exporting something that "can open" is only the minimum standard. The coordinates, transparency, collision, navigation and performance should also be verified automatically in targets such as Unity, Unreal and Godot/WebGPU. Marble's specification page retains the note that going from OpenCV to OpenGL requires flipping Y/Z. The changelog, however, shows that world generation has exported OpenGL by default since 2025-12-11, and since 2026-01-01 can choose either OpenGL or OpenCV. A fixed default value therefore must not be assumed. The actual export version and options must be read. Marble export specification [@srcWL-MARBLE-EXPORT-M04] Marble changelog [@webd179b99c4e6c]

Executable asset specifications and modeling agents will form a closed loop Future text-to-3D need not predict final vertices directly. It may instead generate restricted Blender Python, Geometry Nodes/Houdini graphs, OpenSCAD/CadQuery programs, USD scene operations or engine editor transactions. Intermediate results are then editable and parameterizable, and the geometry kernel can also report explicit failures. Code generation, however, simultaneously expands execution privileges and the geometric attack surface. A syntactically correct script can read credentials, access the network, overwrite paths, instantiate a billion objects, or exhaust resources with booleans/subdivision. A security architecture should restrict model output to a declarative schema, a whitelist API or a capability-limited DSL. If general-purpose Python/C# must be executed, then run it in a one-shot worker. That worker has the network disabled, no credentials, read-only inputs, a bounded output root, CPU/memory/wall-clock quotas and host auto-scripting disabled.

Closed-loop quality should not be judged only by the same model "looking at images and grading itself." The builder outputs a normalized mesh/scene and a manifest. An independent verifier checks dimensions, manifoldness, self-intersections, object IDs, collision clearance, NavMesh reachability, engine read-back and resource budget, and then returns a minimal counterexample to the agent. Each iteration saves the original program, patches, parameters, seeds, toolchain, stdout/stderr and asset hashes. Only generations that pass all verification are released atomically. Research evaluation can report the first-pass rate, the number of repair rounds, the final query pass rate, worst-case resource usage and unresolved failure types. It should not show only the prettiest successful samples.

A testable research agenda Build an end-to-end benchmark that includes observed images, true-scale geometry, topology, materials, collision/contact, and navigation tasks, all at the same time. Design "revisiting, branching, occlusion, and modification persistence" tests that specifically evaluate the long-term 3D state of world models. Build calibrated uncertainty maps for generated regions and for observed regions. Verify whether those maps can predict mesh/collision failures. Study reversible or controlled lossy conversions among Gaussians, meshes, SDFs, and scene graphs. Report the error propagation of each conversion. Incorporate manual asset repair time, import failure rate, and runtime budget into generative 3D evaluation. Establish a combined red-team benchmark for data poisoning, viewpoint backdoors, resource exhaustion, malicious assets, and navigation/collision manipulation. Require defenses to report clean quality, geometric cost, latency, memory, adaptive attacks, and recovery capability at the same time. Use real interaction tasks to verify whether "visual physics" translates into dynamics that can be verified, controlled, and safely stopped. Establish an agentic modeling benchmark for "intent or text---modeling program---visual/collision asset---engine query". It must publish host versions, sandboxes, failure programs, and repair traces.

Evidence and reproduction limitations

The first limitation concerns retrieval. Query and screening logs exist, but the process is not a database-recomputable two-person systematic screening. Conclusions about coverage can therefore only be interpreted as a distribution within the recorded corpus. A second limitation is the evidence base for methods. Much of it comes from official web pages, project pages, structured extraction, and author reports. The new LaTeX package preserves neither local full-text PDFs nor page-level locations, so fine-grained mechanisms should still be verified against the primary full text.

The third limitation concerns the quantitative record. It depends on the same study and on shared tables, and variance is generally missing. Descriptive improvements, year correlations, and missing plots cannot be upgraded into causal evidence for overall effects, for significant superiority, or for technological progress. The fourth limitation concerns the reach of local reproduction. That work covers only synthetic height fields, three kinds of surfaces or collision proxies, and deterministic dual meshes. Real scans, dynamic scenes, rigid-body contact, navigation, and cross-engine consistency have not been experimentally verified.

The fifth limitation concerns Marble. Its internal algorithms, paid-service behavior, and training data are not visible. The main text evaluates only public inputs, outputs, sample files, and failure descriptions. The sixth limitation concerns the hosting tools. Hosts such as Blender, Houdini, OpenSCAD, CadQuery, FreeCAD, Unity, Unreal, USD, and Omniverse were not executed in the recorded environment. Syntax checking and static scanning can prove only that examples can be parsed. They cannot prove that host APIs, boolean kernels, dependency graphs, collision cooking, or navigation baking succeed.

Finally, this survey uses a publisher-neutral Chinese generic template. It has not yet selected a specific journal or conference, and it has not verified page counts, anonymity policy, citation style, or official class files. Therefore, even if the PDF compiles, the status can only be compiled-draft.

Two October releases extend this chapter's frame. Towards Safe World Models (arXiv 2610.13372) organises a 163-work safety corpus around a six-stage lifecycle — a usable safety map over exactly the L4/L5 deployment zone. The Survey of Security in Real-World Agents (2026-10-14) audits plan-level versus physical-level evidence, applying the same discipline as this survey's host-deployment boundary.

\newpage

Conclusion

This survey does not offer a list of methods. It answers one question through a capability closed loop. How does 3D space move from imagery or design intent toward a world that can be queried, edited, collided with, and run?

3D spatial construction is not a single-model problem. It is a set of contracts that are connected to one another yet cannot substitute for one another. Video models are good at generating pixels and motion. World models add action and memory. Reconstruction algorithms recover cameras and geometry. NeRF and 3DGS provide high-quality novel views. Procedural Blender/DCC, CAD/CSG, and node graphs turn known design intent directly into editable geometry. Point clouds and implicit surfaces connect perception and surfaces. Meshes, colliders, and navigation meshes turn representations into assets. Game engines turn assets into runnable worlds. The verification and security layer determines whether this world can be trusted.

Therefore, the most reliable selection principle is this. Do not seek a ”universal 3D representation.”

First, define the query that ultimately needs to be answered. Choose the most suitable representation for each layer. Make explicit what is lost in each conversion. Measure the visual, geometric, physical, and security aspects separately. Finally, stay honest about conclusions that are unobservable, incomparable, and non-reproducible.

Generation, reconstruction, hybrid representations, assetization, physics, and runtime verification must become one system with an evidence closed loop. Only then does the field truly move from ”generating a result that looks like a world” to ”constructing a usable 3D space.”

\newpage

Open Materials and Migration Audit

Open materials preserve the records of retrieval, sources, screening, quantitative analysis, reproduction, failures, and migration. They are used to reconstruct the claims in the main text. They do not upgrade static audits or planned work into executed results.

The structured source master table is `data/papers.csv`. It holds 253 unique source ID/URL entry records and 247 normalized title keys. The retrieval and screening traces are located in `sources/search_log.csv` and `sources/screening_log.csv`. Per-paper research questions, inputs and outputs, key numbers, and evidence levels are stored separately in paper/ _ .md . They no longer interrupt the method text.

The attack-defense contracts are stored in `data/attacks_defenses.csv`, and the benchmark long table is stored in `data/benchmarks.csv`. The statistical entry point is `analysis/run_meta_correlation.py`, with outputs located in `analysis/results_part_quant/`. The original lightweight mechanism reproduction entry points are `reproduction/run_reproduction.sh`, `reproduction/STAGE_SUMMARY.md`, and `reproduction/index.html`. The programmatic dual-mesh reproduction and the host-not-executed boundary are located in `reproduction/programmatic_modeling/`. Evidence merging and the quality gate are given by `analysis/build_evidence_db.py` and `analysis/evidence_qa.json`.

Throughout the text, citations point preferentially to papers, official technical reports, project pages, standards, or official engine documentation. Product materials only attest to public interfaces and official statements. Where the model structure, training data, or independent benchmark is not public, those parts are not written up as verified performance.

Appendix — Post-cutoff update (2026-08-07 2026-09-26)

The body's material closes on 7 August 2026. This appendix registers later material; **the body text is unchanged.**

\newpage

A.1 — 3D spatial construction: no substantive progress on the agenda

The body sets out a testable research agenda (an end-to-end 3D benchmark; revisit, branch, occlusion and edit-persistence tests; calibrated uncertainty; error propagation across representations; manual repair time in evaluation). Nothing found since the cutoff changes it — all five directions remain **open**.

\newpage

A.2 — Related movement

Date	Movement
2026-09	Sony and Reuters demonstrated a near-live newsroom authenticity workflow; AFP and Dalet announced news-v

\newpage

A.3 — Cross-repository note: institutional consequence of the OpenAI–Hugging Face incident

On 2026-09-16 / 17 OpenAI published an account of the incident and announced a safety-incident disclosure process; reporting describes roughly 700 agents, dozens of third-party systems and 53 leaked user images; the US Senate opened an investigation (led by Hawley).

This is a methods survey with no attack/defence content, so the incident does not enter its argument. It is recorded here only to make the time window of the sibling documents explicit, and to note that this survey shares the ”actionable 3D world” layer with the embodied security survey in the same series — a layer that is now becoming a regulated object.

\newpage

A.4 — How to use this appendix

The L0–L5 capability loop, the cross-family synthesis and the evaluation boundaries are unaffected. What this appendix records is the fact that **nothing changed** — the agenda is still open, which is itself informative.

\newpage